Chair of Applied Mechanics
TUM School of Engineering and Design
Technical University of Munich

Development of an Open-Source 3D Visualization Framework for Structural Dynamics Compatible with Commercial and Academic FEM Tools

Entwicklung eines Open-Source-3D-Visualisierungsframeworks für Strukturdynamik, kompatibel mit kommerziellen und akademischen FEM-Tools

Scientific work for obtaining the academic degree
Master of Science (M.Sc.)
at the TUM School of Engineering and Design of the Technical University of Munich

Supervisor	Prof. dr.ir. Daniel J. Rixen Chair of Applied Mechanics
Advisor	Tiago Miguel Simoes Martins, M.Sc. Chair of Applied Mechanics
Submitted by	Caglar Ayberk Saglik
Submitted on	January 26, 2026 in Garching

Abstract

The fragmentation of simulation tools in computational mechanical engineering creates significant barriers to collaboration and data exchange. This thesis addresses the interoperability gap between proprietary simulation software, academic codes, and in-house solvers by developing a modular framework in the Rust programming language. The framework enables the conversion of proprietary mesh formats (such as COMSOL `.mphtxt` and Abaqus `.inp`) to the open-standard VTU format for visualization in ParaView, while providing extensible modules for user-defined pre-processing analysis. Key contributions include mesh quality assessment using metrics like Jacobian ratio and skewness, integration point optimization based on material properties and a user-defined error tolerance, and deformation visualization of raw result data from `.txt` files. By leveraging Rust's memory safety, performance characteristics, and growing industry adoption, this work establishes a foundation for reliable, high-performance engineering software tools that bridge legacy systems and modern development practices.

Kurzfassung

Die Fragmentierung von Simulationswerkzeugen im rechnergestützten Maschinenbau bildet erhebliche Barrieren für Zusammenarbeit und Datenaustausch. Diese Arbeit behandelt die Interoperabilitätslücke zwischen proprietärer Software, akademischen Codes und internen Solvern durch die Entwicklung eines modularen Frameworks in der Programmiersprache Rust. Das Framework ermöglicht die Konvertierung proprietärer Netzformate (wie COMSOL `.mphtxt`, Abaqus `.inp`) in das offene VTU-Standardformat zur Visualisierung in ParaView, unter Bereitstellung erweiterbarer Module für benutzerdefinierte Vorverarbeitungsanalysen. Zu den Kernbeiträgen zählen die Netzqualitätsbewertung mittels Metriken wie Jacobi-Verhältnis und Schiefe, die auf Materialeigenschaften und einer benutzerdefinierten Fehlertoleranz basierende Integrationspunktoptimierung, sowie die Verformungsvisualisierung von Rohdaten aus `.txt`-Dateien. Durch Nutzung der Speichersicherheit, Leistungscharakteristika und der wachsenden Industrieakzeptanz von Rust schafft diese Arbeit eine Basis für zuverlässige Hochleistungs-Ingenieursoftware zur Verbindung von Altsystemen mit modernen Entwicklungspraktiken.

Contents

List of Figures viii

List of Tables ix

Nomenclature xi

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 State of the Art . . . 2
  - 1.2.1 FEM Related Tools . . . 2
  - 1.2.2 Programming Language . . . 3
- 1.3 Objectives . . . 4
- 1.4 Thesis Outline . . . 5

2 Physical and Mathematical Basics 7
- 2.1 Shape Functions and Jacobian Matrix . . . 7
- 2.2 Mesh Quality Analysis . . . 8
  - 2.2.1 Mathematical Formulation and Data Extraction . . . 9
  - 2.2.2 Implemented Quality Metrics . . . 10
- 2.3 FEM Formulations and Derivation of Element Mass and Stiffness Matrices . . . 12
  - 2.3.1 Equations for Element Mass and Stiffness Matrices in Physical Domain 13
  - 2.3.2 Equations for Element Mass and Stiffness Matrices in Isoparametric Domain . . . 14
- 2.4 Derivation of Error Bound for the Gauss Quadrature . . . 15
  - 2.4.1 Derivation of the Gauss Quadrature Error Formula . . . 15
  - 2.4.2 Gauss-Legendre Quadrature Formula . . . 17
  - 2.4.3 Derivation of the Error Bound for 1D Case . . . 18
  - 2.4.4 Extension to Multidimensional Cases . . . 18
  - 2.4.5 Monomial Ordering . . . 22
  - 2.4.6 Implementation of Polynomial Operations . . . 22
- 2.5 Numerical Helper Functions . . . 23
  - 2.5.1 Inverse of a Matrix . . . 23
  - 2.5.2 Series Expansion for Reciprocal Polynomial . . . 24
  - 2.5.3 Series Expansion for Square Root . . . 25

3 Implementation of the Code 27
- 3.1 Code Overview . . . 27
- 3.2 Parsing Module . . . 28
  - 3.2.1 Parsing of COMSOL Multiphysics Mesh Output (.mphtxt) . . . 28
  - 3.2.2 Parsing of Abaqus Input Files (.inp) . . . 30
  - 3.2.3 Parsing of Nodal Results (.txt) . . . 31

3.3 Node Reordering and Shape Functions for VTK-Compatible Output 33
3.3.1 Element Connectivity Reordering and Topology Matching 33
3.3.2 Implementation of Shape Functions 35
3.4 Mesh Quality Analysis Module 36
3.4.1 Analysis Architecture and Data Flow 37
3.4.2 Jacobian Polynomial Construction 37
3.4.3 Algebraic Matrix Factorization 37
3.4.4 Quality Metrics 38
3.4.5 Utility Functions 38
3.5 Gaussian Quadrature and Numerical Optimization 38
3.5.1 Architectural Flow and Parallelism 39
3.5.2 Optimization Logic 39
3.5.3 Symbolic Integrand Construction 40
3.5.4 Gauss-Legendre Error Estimation 41
3.6 Writer Implementation for Visualization in ParaView 42
3.6.1 Geometry Serialization and Coordinate Padding 42
3.6.2 Topology and Connectivity Assembly 42
3.6.3 Nodal Attribute Export and Vector Padding 43
3.6.4 Cell Data and Geometric Analysis Reporting 43

4 Validation and Results **45**
4.1 Validation of Mesh Parsing 45
4.1.1 Validation of COMSOL (.mphtxt) File Parsing 45
4.1.2 Validation of Abaqus (.inp) File Parsing 47
4.1.3 Validation of Nodal Results and Mesh Deformation 49
4.2 Validation of Mesh Quality Metrics 52
4.2.1 Validation of 1D Elements in 1D Meshes 53
4.2.2 Validation of 2D Elements in 2D Meshes 53
4.2.3 Validation of 2D Elements in 3D Meshes 54
4.2.4 Validation of 3D Elements in 3D Meshes 56
4.3 Validation of Quadrature Optimization 59
4.3.1 Analysis of 1D Quadratic Line Element in 1D Mesh 59
4.3.2 Analysis of 2D Biquadratic Quad Elements in 2D Mesh 60
4.3.3 Analysis of 3D Hexahedron Elements in 3D Mesh 61
4.4 Verification and Error Analysis of Polynomial Approximations 62
4.4.1 Mass Integration: Non-Square Jacobian with Constant Metric Determinant 63
4.4.2 Mass Integration: Non-Square Jacobian with Non-Constant Metric Determinant 64
4.4.3 Stiffness Integration: Square Jacobian with Constant Determinant . . . 65
4.4.4 Stiffness Integration: Square Jacobian with Non-Constant Determinant 66

5 Summary and Outlook **69**
5.1 Summary 69
5.2 Future Developments 70

A Source Code Repository **73**

Bibliography **75**

List of Figures

2.1 Transformation of an element from the physical space to the isoparametric space. 7
2.2 Comparison of elements based on the volume metric. 10
2.3 Comparison of elements based on the Jacobian Ratio. 11
2.4 Comparison of elements based on the shape metric. 11
2.5 Comparison of elements based on the skewness metric. 12
2.6 Comparison of elements based on the length ratio metric. 12
2.7 Comparison of elements based on the orientation metric. 13

3.1 Execution sequence and system data flow. The pipeline is orchestrated by 'main.rs', beginning with argument validation. Depending on the input file extension, the system dynamically dispatches the appropriate parser ('MphtxtParser' or 'AbaqusParser') to unify legacy formats into a standardized 'MeshData' structure. The workflow features two conditional branches controlled by user flags: the **Mesh Quality** module, which computes geometric metrics such as skewness and aspect ratio, and the **Gaussian Optimization** module, which performs symbolic integration to determine optimal quadrature points. The process concludes with the 'VTUWriter' serializing the mesh topology and computed results into a '.vtu' file for visualization. 29
3.2 Abbreviated structure of a COMSOL Multiphysics .mphtxt file [9]. The parser processes topological data while ignoring auxiliary sections like Parameters, Domains, and Up/down pairs. 30
3.3 Structure of an Abaqus .inp file showing explicit node definitions (`*NODE`) and element connectivity (`*ELEMENT`) [13]. 32
3.4 Quadratic Edge Element comparison: Abaqus vs. VTK. 34
3.5 Biquadratic Quad Element comparison: COMSOL vs. VTK. 34
3.6 COMSOL pyr2 element. 35
3.7 VTK Quadratic Pyramid (Type 27). 35
3.8 Quadratic Pyramid Element comparison: COMSOL vs. VTK. 35

4.1 Visualization of the one-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. Labels 1, 2, and 3 correspond to vertex, linear edge, and quadratic edge elements. 46
4.2 Visualization of the two-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. The domain comprises the 1D elements (1–3) alongside the 2D triangular and quadrilateral elements (4–7). 46
4.3 Visualization of the three-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. The scene integrates 1D lines and 2D surfaces alongside the 3D volumetric elements labeled 8 to 15. 47
4.4 Visualization of the one-dimensional mesh topology parsed from the Abaqus `.inp` file. Labels 1 and 2 correspond to linear and quadratic link elements, respectively. 48

4.5 Visualization of the two-dimensional mesh topology parsed from the Abaqus `.inp` file. The domain comprises the 1D link elements (1–2) alongside the 2D triangular and quadrilateral elements (3–6). 48
4.6 Visualization of the three-dimensional mesh topology parsed from the Abaqus `.inp` file. The scene integrates 1D links (1–2) and 2D surfaces (3–6) alongside the 3D volumetric elements (7–15). 49
4.7 Visualization of 1D mesh deformation. The orange nodes (0a, 1a) represent the initial undeformed state of the `edg` element. The green nodes (0b, 1b) depict the deformed state after applying the displacement values parsed from the `.txt` file. 50
4.8 Visualization of 2D mesh deformation on a 9-node `quad2` element. The orange mesh (nodes labeled with suffix 'a') shows the initial unit square configuration. The green mesh (nodes labeled with suffix 'b') depicts the deformed state, characterized by symmetric horizontal expansion at the corners and vertical displacement of the horizontal mid-side nodes, while the vertical mid-side nodes (5 and 7) remain stationary. 51
4.9 Visualization of 3D mesh deformation on a composite domain containing a `pyr2` and a `hex` element. Subfigures (a) and (b) present alternate perspective views. Orange indicates the initial state, and green indicates the final deformed state. 52
4.10 Visual validation of quality metrics for 1D Quadratic Edge elements in 1D mesh. 54
4.11 Visual validation of quality metrics for a 2D Quadrilateral element within a 2D mesh domain. 55
4.12 Visual validation of 2D Triangular element quality metrics embedded in 3D space. 57
4.13 Visual validation of quality metrics for 3D Wedge (Prism) elements. 58

List of Tables

4.1 Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 1D Quadratic Edge). 59
4.2 Effect of geometric distortion on Gaussian quadrature requirements. The distortion is introduced by shifting the mid-side node, lowering the Jacobian Ratio. (Material: Constant $\rho(x) = 1.0$, Tolerance: 10^{-4}). 60
4.3 Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 2D Biquadratic Quad). 60
4.4 Effect of geometric distortion on Gaussian quadrature requirements for a Bilinear material property ($\rho = 1 + 2x - y$, Tolerance: 10^{-4}). 61
4.5 Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 3D Hexahedron). 61
4.6 Effect of geometric distortion on Gaussian quadrature requirements for a Quadratic material property ($\rho = 1 - x^2 + 10y + 2z$). Note that the parametric distortion in the Jacobian case significantly increases the polynomial degree from 8 to 12. 62
4.7 Gaussian point inflation due to Binomial Expansion vs. Ideal Exact function. . 65
4.8 Gaussian point inflation due to Taylor Series approximation vs. Ideal Rational function. 68

Nomenclature

Abbreviations

FE	Finite Element
FEM	Finite Element Method
PDE	Partial Differential Equation
VTK	Visualization Toolkit
VTU	Unstructured Grid (VTK XML format)
XML	Extensible Markup Language
JR	Jacobian Ratio
1D, 2D, 3D	One-, Two-, and Three-Dimensional
API	Application Programming Interface
MPHTXT	COMSOL Mesh Text Format
INP	Abaqus Input Format

Latin Symbols

a_k, a_{ij}, a_{ijk}	Polynomial coefficients
$\mathbf{A}$	Working matrix for mesh quality analysis (Metric Tensor or Jacobian)
A_n	Leading coefficient of the Legendre polynomial
$\mathbf{B}$	Strain-displacement matrix
$\mathbf{B}^{\text{iso}}$	Strain-displacement matrix in isoparametric coordinates
$\mathbf{B}^{\text{phys}}$	Strain-displacement matrix in physical coordinates
C_n	Error constant for Gauss-Legendre quadrature
$\mathbf{C}$	Elasticity matrix (Constitutive matrix)
$\mathbf{D}$	Length ratio matrix (Knupp's decomposition)
d, d_{poly}	Polynomial degree
d_{el}	Dimension of the element
d_{mesh}	Dimension of the mesh
$E_n(f)$	Error term for n-point quadrature
$f(\xi)$	General integrand function
$H_n(x)$	Hermite interpolating polynomial
$I_n(f)$	Numerical approximation of an integral
$\mathbf{I}$	Identity matrix
$\mathbf{J}$	Jacobian matrix
$\mathbf{J}^+$	Moore-Penrose pseudoinverse of the Jacobian
$\mathbf{K}$	Element stiffness matrix

$l_j(x)$	Lagrange basis polynomial
Len	Length ratio metric
$\mathcal{M}$	Metric Jacobian tensor ($\mathbf{J}\mathbf{J}^T$)
$\mathbf{M}$	Element mass matrix
n, n_{gp}	Number of integration points (Gaussian points)
$n_{gp,\text{theoretical}}$	Theoretical minimum Gaussian points for exact integration
$n_{gp,\text{optimal}}$	Optimal Gaussian points satisfying error tolerance
N	Number of terms in series expansion
$\mathbf{N}$	Vector/Matrix of shape functions
N_i	Shape function for node i
N_{nodes}	Total number of nodes in an element
Ori	Orientation metric
$p(\mathbf{x})$	Polynomial function
$P_n(x)$	Legendre polynomial of degree n
$\mathbf{Q}$	Skew matrix (Knupp's decomposition)
$Q_n, Q_{n,n}, Q_{n,n,n}$	Quadrature approximation in 1D, 2D and 3D
r	Term used in Taylor series expansion for reciprocal
$\mathbf{R}$	Orientation matrix (Knupp's decomposition)
s	Term used in Binomial series expansion for square root
$\mathbf{S}$	Shape matrix (Knupp's decomposition)
Shape	Shape metric
Skew	Skewness metric
$\mathbf{U}$	Volume matrix (Knupp's decomposition)
u, v, w	Nodal displacement components
V^{iso}	Volume of the element in isoparametric space
V^{phys}	Volume of the element in physical space
Vol	Volume metric
w_j	Gaussian quadrature weight for node j
x, y, z	Global (physical) coordinates
$\mathbf{x}^\alpha$	Monomial with multi-index α
$\mathbf{X}$	Matrix of global nodal coordinates
$\mathbf{X}_i$	Global coordinate vector of node i

Greek Symbols

α	Multi-index exponent (in polynomial context)
α	Determinant of the working matrix $\det(\mathbf{A})$ (in mesh quality context)
$\det(\cdot)$	Determinant operator
ξ, η, ψ	Local (isoparametric) coordinates
λ_{ij}	Components of the metric tensor $\mathbf{A}\mathbf{A}^T$
μ	Scale factor derived from $\sqrt{\lambda_{00}}$
ρ	Material density
$\varphi_n(x)$	Orthogonal polynomial
$\Psi_n(x)$	Nodal polynomial

Mathematical Operators

∇	Gradient operator
∂	Partial derivative
$\otimes$	Kronecker product
$\|\cdot\|_F$	Frobenius norm
adj$(\cdot)$	Adjugate matrix
trace$(\cdot)$	Trace of a matrix
$\binom{n}{k}$	Binomial coefficient

Chapter 1

Introduction

1.1 Motivation

The field of computational mechanical engineering exists within a complex and fragmented ecosystem of simulation tools. While commercial software suites are prevalent, they frequently suffer from significant interoperability issues, locking valuable simulation data, such as finite element meshes and field results, into proprietary, incompatible formats. This creates substantial barriers to collaboration between universities, research institutes, and industrial partners who may utilize different software environments. Furthermore, the specific demands of advanced research often necessitate the use of specialized academic or in-house developed codes, which, while powerful for their intended physics, typically lack robust visualization interfaces and standardized data exchange capabilities. The high cost of commercial licenses further restricts access and flexibility, particularly for smaller organizations and academic groups.

The core motivation for this work stems from several interconnected challenges observed in this landscape. First is the fundamental issue of data compatibility and visualization. The inability to seamlessly share and visually interpret results across different software platforms impedes collaborative progress and knowledge dissemination. A researcher using an in-house solver, for instance, may generate critical results but have no immediate means to visualize them effectively, hindering insight. Second is the critical need for flexible pre-processing and mesh analysis. Commercial and academic tools often provide limited capabilities for user-defined, programmatic analysis. For example, assessing whether a mesh uses an optimal number of Gaussian integration points for a given material model, based on user-defined error tolerance and material properties like density and elasticity, can prevent both numerical inaccuracy and computational waste. Performing such analysis before launching expensive simulations is vital, poor mesh quality, such as bad length ratio and non-positive determinant, leads to inaccurate results. Identifying these issues early saves considerable time and resources, while optimizing mesh and integration scheme efficiency addresses not only an economic concern but also an environmental one by reducing unnecessary computational load.

To address these needs in a sustainable and forward-looking manner, this thesis proposes the development of a new, modular framework. Beyond creating mesh file conversion tools, the aim is to establish an extensible architecture, enabling the seamless integration of new parsers, analysis modules, and output formats, accommodating future requirements. A strategic decision underpinning this framework is the choice of the Rust programming language for its implementation. Rust offers the performance necessary for handling large-scale engineering data while guaranteeing memory safety and enabling fearless concurrency for parallel processing tasks. This performance is further leveraged by employing efficient numerical

methods, such as monomial-based operations instead of heavier symbolic calculations, for critical analysis routines. Rust's growing adoption by major technology leaders signifies its long-term viability, making it an ideal foundation for a tool intended to bridge current gaps and remain relevant amidst future technological shifts.

Consequently, this project is motivated not only by the immediate goal of solving specific interoperability and pre-processing problems but also by the opportunity to create a robust, high-performance, and future-proof foundation for open engineering software development.

1.2 State of the Art

This section provides a critical review of the existing ecosystem for Finite Element Method (FEM) pre-processing and interoperability. It evaluates current software tools to identify the functional gaps that the proposed framework aims to address. Subsequently, it justifies the selection of the Rust programming language by analyzing its performance benchmarks, memory safety guarantees, and growing industrial adoption relative to traditional scientific computing languages.

1.2.1 FEM Related Tools

The current landscape of FEM helper tools encompasses a variety of mesh generators, format converters, and visualization engines. While robust solutions exist for individual stages of the simulation pipeline, there remains a lack of unified tools capable of handling proprietary formats, performing rigorous algebraic quality analysis, and optimizing numerical integration parameters simultaneously.

Gmsh 4.15.0 [16] operates as a high-level, general-purpose finite element mesh generator, excelling in the discretization of CAD geometry into conformal unstructured meshes using algorithms such as Delaunay and Frontal. It provides a robust suite of native geometric quality metrics, such as the Jacobian determinant, Gamma, and EquiAngleSkew, designed to detect element inversion or excessive distortion. Furthermore, Gmsh defines integration points based strictly on element topology, lacking mechanisms to ingest material properties such as density and elasticity, or to optimize Gaussian quadrature schemes based on user-defined error tolerances. Additionally, despite its extensive list of supported input options, it lacks native support for the COMSOL `.mphtxt` format.

FEconv [30] is a specialized utility for mesh interoperability, functioning primarily as a format translation engine written in Fortran. It facilitates the bidirectional conversion of various mesh formats, including COMSOL `.mphtxt` and ANSYS `.msh`. However, FEconv operates effectively as a "blind" translator; it preserves node coordinates and connectivity without verifying the mathematical validity or quality of the resulting elements. It lacks internal mechanisms for geometric diagnostics, such as Jacobian-based quality checks or orientation analysis, and possesses no capabilities for physics-coupling or numerical integration optimization. While Fortran remains dominant in high-performance computing, the legacy architecture of this tool lacks the modern memory safety features inherent in newer systems languages.

The meshio library [31] stands as the dominant Python-based utility for the input/output and conversion of unstructured meshes, supporting an extensive breadth of file formats including Abaqus `.inp` and ANSYS `.msh`. Its integration with the NumPy ecosystem allows for high-level scripting and data manipulation. However, the scope of `meshio` is strictly limited to data transport; it reads topology and geometry but performs no mathematical assessment

of the data. It does not implement geometric quality checks or orientation analysis and is entirely decoupled from simulation physics. Furthermore, as a Python-based tool, it incurs performance overheads in large-scale data processing compared to compiled languages, and its standard documentation does not list native support for the COMSOL `.mphtxt` format.

PyVista [34] functions as a high-level Python interface to the Visualization Toolkit (VTK), streamlining the visualization and geometric analysis of spatial datasets. It excels in rendering complex 3D geometries and includes geometric quality filters for standard metrics like scaled Jacobian and distortion. However, PyVista's scope is bound by the VTK backend, designed primarily for visualization rather than numerical verification. Its metrics are geometric approximations intended to detect visual artifacts rather than numerical suitability for solvers. Crucially, it lacks frameworks for optimizing Gaussian integration points based on constitutive equations. Like `meshio`, it relies on the Python runtime, which presents performance limitations for intensive pre-processing tasks relative to systems languages.

ParaView [3] is the premier open-source application for scientific data visualization, capable of handling extremely large datasets via distributed computing pipelines. It includes a `Mesh Quality` filter for various geometric metrics. Consequently, ParaView cannot natively optimize Gaussian quadrature points based on constitutive polynomials. Although it supports a vast library of input formats, it lacks a native reader for the COMSOL `.mphtxt` format. Despite these limitations regarding pre-processing verification, it remains a powerful tool for post-processing visualization and is utilized in this thesis for rendering the output of the developed framework.

1.2.2 Programming Language

The choice of Rust for the developed framework is motivated by its unique position in the modern software landscape, offering memory safety without garbage collection while maintaining performance parity with legacy systems languages. This section evaluates Rust's performance against traditional scientific languages, details its adoption by major industry stakeholders as evidence of its longevity, and addresses the mitigation of its learning curve through modern tooling.

Rust is a high-performance, low-level language designed to offer memory safety guarantees without garbage collection. Its key feature is an ownership-based system where the compiler detects data races and memory issues (e.g., use-after-free, buffer overflows) at compile time. This allows developers to write expressive, high-level code with zero-cost abstractions while maintaining fine-grained control over system resources [26].

In terms of raw performance, Rust significantly outperforms interpreted languages like Python, which face limitations in CPU-intensive tasks due to interpretation overhead. Benchmarks demonstrate that Rust can be approximately 30 times faster than standard Python implementations for algorithmic tasks such as summation operations [1].

When compared to legacy systems languages, Rust offers a modern alternative to C. Known primarily for its speed, the C programming language has defined the standard binary interface for bindings due to its prevalence and use in creating universal software packages. However, it suffers from outstanding undefined behavior issues, which have prompted generations of additional tooling to remedy its design faults. Rust emerged from the lessons learned from C, focusing on delivering similar performance using modern tooling [1].

In sequential execution benchmarks, Rust is highly competitive, performing approximately 1.23% slower than Fortran and 5.59% faster than C++ in scientific workloads. However, in parallel execution contexts (specifically using the Rayon library), Rust currently lags slightly behind, performing approximately 16% to 19% slower than mature OpenMP implementations in Fortran and C++ due to differences in scheduling and thread management

[26]. Recent evaluations using the NAS Parallel Benchmarks (NPB), a standard suite for evaluating high-performance computing, confirm Rust's viability for scientific applications. The NPB-Rust suite implementation demonstrates that while Rayon scales well with work-stealing strategies, it may hit scalability limits earlier than OpenMP in fine-grained computations [26].

Beyond performance, major industry players are shifting toward memory-safe languages such as Rust to mitigate security vulnerabilities. According to a Microsoft presentation by Matt Miller, approximately 70% of the vulnerabilities addressed through security updates each year continue to be memory safety issues, with the transition to safer languages identified as one of the mitigation tactics [27].

This trend is further evidenced by Google's adoption of Rust within the Android ecosystem. As the amount of new memory-unsafe code entering Android has decreased, the number of memory safety vulnerabilities dropped from 76% in 2019 to 35% in 2022, marking the first year where memory safety issues did not represent the majority of vulnerabilities. In Android 13, about 21% of all new native code is written in Rust, and as of December 2022, zero memory safety vulnerabilities had been discovered in Android's Rust code [35].

While the ownership-based type system provides robust safety guarantees and enables features like closures and pattern matching, it also presents a steep learning curve. However, recent advancements in Large Language Models (LLMs) are mitigating this barrier. A tool called `RUSTASSISTANT`, which leverages LLMs to suggest fixes for compilation errors, has achieved a peak accuracy of roughly 74% on real-world errors in popular open-source repositories. This demonstrates that modern AI tooling can effectively support developers in overcoming the complexities of the language [15].

1.3 Objectives

The primary objective of this thesis is the development of a high-performance, modular framework for finite element mesh conversion and analysis. The software utilizes the Rust programming language, which ensures high computational performance, memory safety, and provides a foundation for future-proof, modern software. The specific research goals are defined through four core functional components.

The first objective is to engineer a robust parsing architecture for distinct data sources. This module is designed to ingest COMSOL Multiphysics mesh files (`.mphtxt`) and Abaqus input files (`.inp`). It also processes nodal simulation results from text files (`.txt`), which allows the system to create a unified internal representation from varied external sources. Furthermore, the modular design facilitates the future inclusion of parsers for additional mesh and result formats.

The second objective is the rigorous conversion of proprietary mesh formats into the open-standard VTK Unstructured Grid format (`.vtu`). This objective extends beyond static geometry since the framework integrates parsed nodal results directly into the mesh topology. This data encapsulation enables scientific visualization tools, such as ParaView, to render the deformed mesh configuration and display simulation fields simultaneously.

The third objective is to implement an optional diagnostic module for geometric validation. This component performs an element-by-element assessment using Knupp's algebraic quality metrics, Jacobian ratio checks, and determinant evaluation. The calculated quality fields are packed directly into the output `.vtu` file to allow for the visual identification of distorted elements within the 3D domain. Additionally, the architecture permits the implementation of user-defined quality metrics.

The final objective is to develop a numerical optimization module for Mass and Stiffness matrix integration. The system determines the optimal number of Gaussian points that are

required to satisfy a user-defined error tolerance and specific material properties, specifically density or elasticity. The upper bound error formula is derived from the Gauss-Legendre rule. To maximize computational performance, the algorithm utilizes monomial polynomial arithmetic instead of standard symbolic calculations. The optimization results, specifically the theoretical limit for exact integration, the calculated optimal number, and the difference between them, are exported as scalar fields in the `.vtu` file for visualization. The current implementation supports the following VTK element types: Line, QuadraticEdge, Quad, QuadraticQuad, BiquadraticQuad, Hexahedron, QuadraticHexahedron, BiquadraticQuadraticHexahedron, and TriquadraticHexahedron. The framework design allows for future extensions such as damping matrix integration, alternative error bound formulas, and support for additional element topologies.

Complementary to the visualization outputs, the framework is designed to ensure rigorous data verification and accessibility. For every major processing stage, including mesh parsing, nodal value ingestion, geometric quality analysis, and Gaussian optimization, the software automatically generates structured text files (`.txt`). These raw data exports allow for granular numerical control, statistical data analysis, and precise verification in scenarios where visual inspection via ParaView is insufficient to detect minute discrepancies.

1.4 Thesis Outline

The remainder of this thesis is structured as follows.

Chapter 2 establishes the mathematical and physical foundation required for the software implementation. It defines the Finite Element Method (FEM) governing equations, specifically the transformation to the isoparametric domain and the derivation of element Mass and Stiffness matrices. The chapter details the algebraic quality metrics used for mesh assessment and derives the theoretical error bounds for Gauss-Legendre quadrature. Furthermore, it documents the critical numerical helper functions used in the codebase, including monomial basis arithmetic, matrix inversion techniques, and Taylor and Binomial series expansions required to approximate non-linear Jacobian terms.

Chapter 3 provides a comprehensive technical breakdown of the modular Rust framework. It describes the architecture of the dispatch-based parsing engine designed for COMSOL (`.mphtxt`) and Abaqus (`.inp`) formats, as well as the generalized ingestion of nodal result files. The chapter details the algorithms for topology standardization, including node reordering and coordinate padding, which ensure compatibility with the VTK visualization standard. Additionally, it explains the control flow of the mesh quality module and the symbolic polynomial engine that powers the adaptive Gaussian quadrature optimization.

Chapter 4 presents the validation and critical analysis of the developed tool. It verifies the robustness of the parsing modules by visualizing mixed-dimensional topologies and validates the accuracy of the geometric quality metrics through unit tests. The chapter evaluates the efficiency of the adaptive Gaussian quadrature, demonstrating computational savings for standard element geometries even with spatially varying material properties. Crucially, it includes a rigorous error analysis of the polynomial approximations, quantifying how the symbolic handling of non-square Jacobians and rational functions via Taylor and Binomial expansions artificially inflates the polynomial degree, leading to wrong Gaussian point estimates for distorted elements.

Chapter 5 synthesizes the research findings, discussing the trade-offs inherent in the exclusive use of symbolic monomial arithmetic. It concludes by outlining strategic avenues for future development, including the implementation of alternative approximation techniques to mitigate degree inflation, support for deformed mesh quality analysis, shape function in-

terpolation for topology matching, and the extension of the framework to include damping matrices.

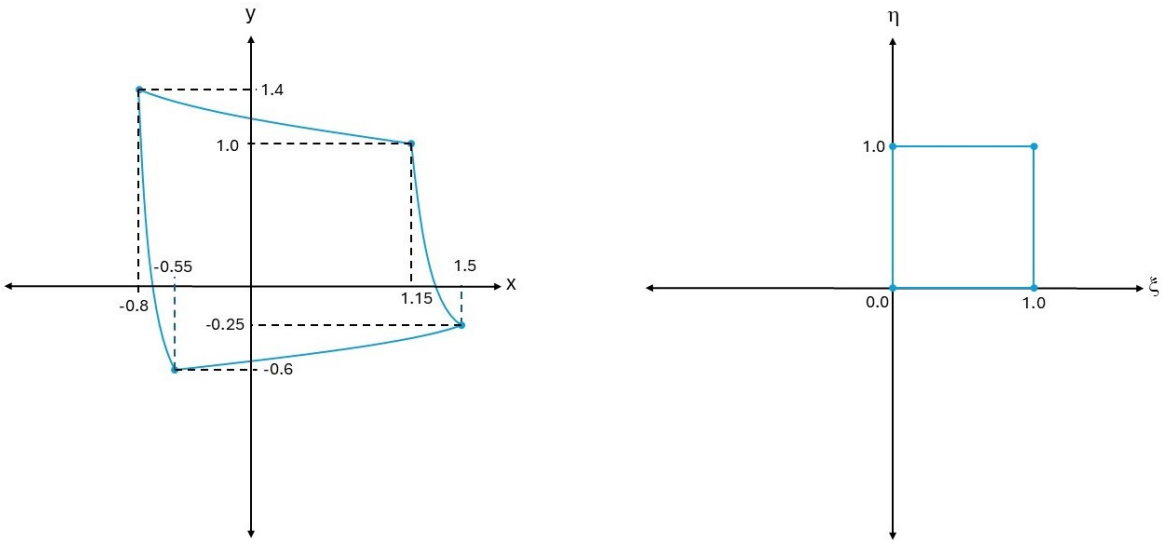
Chapter 2

Physical and Mathematical Basics

This chapter outlines the fundamental physical and mathematical concepts required for the thesis. First, essential terms and formulas for Finite Element Method (FEM) equations, such as shape functions and the Jacobian matrix, are explained. Second, the formulas utilized in mesh analysis are presented. Third, the specific FEM formulations and the derivation of element mass and stiffness matrices are detailed, followed by the derivation of error bounds for the Gauss quadrature. Finally, the mathematical foundations required for code implementation, including monomial calculations, inversion methods, and relevant series expansions, are discussed.

2.1 Shape Functions and Jacobian Matrix

In FEM calculations, computational complexity is significantly reduced by transforming elements defined in the physical space into a standardized isoparametric space where calculations are performed. While the standard isoparametric space in the literature is typically defined within $[-1,1]^d$ [36], this thesis utilizes the boundaries $[0,1]^d$, following the convention used in the VTU file format [21].



(a) Mesh element in real space.

(b) Mesh element in isoparametric space.

Figure 2.1: Transformation of an element from the physical space to the isoparametric space.

To perform this transformation, shape functions are required. The mapping from global

(physical) coordinates to local (isoparametric) coordinates is defined as [36, eq. 8.1]:

$$x(\xi) = \sum_{i=0}^{N_{\text{nodes}}-1} N_i^{\text{iso}} X_i, \tag{2.1}$$

where $\mathbf{N}^{\text{iso}}$ represents the vector of shape functions and $\mathbf{X}_i$ represents the global coordinates of the element's nodes. This interpolation also provides an approximation of the displacement field throughout the element. Shape functions must satisfy specific prerequisites, most notably the Partition of Unity [36, eq. 8.4]:

$$\sum_{i=0}^{N_{\text{nodes}}-1} N_i = 1. \tag{2.2}$$

The transformation is necessary not only for coordinate mapping but also for handling derivative terms. This is achieved using the Jacobian matrix. The relationship between derivatives in the local (ξ, η, ψ) and global (x, y, z) systems is established via the chain rule [36, eq. 9.10]:

$$\begin{bmatrix} \frac{\partial u}{\partial \xi} \\ \frac{\partial u}{\partial \eta} \\ \frac{\partial u}{\partial \psi} \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \psi} & \frac{\partial y}{\partial \psi} & \frac{\partial z}{\partial \psi} \end{bmatrix}}_{\mathbf{J}\ \text{(Jacobian Matrix)}} \begin{bmatrix} \frac{\partial u}{\partial x} \\ \frac{\partial u}{\partial y} \\ \frac{\partial u}{\partial z} \end{bmatrix}. \tag{2.3}$$

The entries of the Jacobian matrix, $\mathbf{J}$, are generally unknown directly but must be calculated using known quantities: the global coordinates of the nodes and the shape functions. The global nodal coordinates are obtained via parsing the existing mesh, while the shape functions are defined in the literature. Consequently, the entries of the Jacobian matrix are calculated as [36, eq. 9.12]:

$$\frac{\partial x}{\partial \xi} = \sum_{i=0}^{N_{\text{nodes}}-1} \frac{\partial N_i^{\text{iso}}}{\partial \xi} X_i, \tag{2.4}$$

where $\frac{\partial N_i^{\text{iso}}}{\partial \xi}$ is the derivative of the shape function with respect to the local coordinate, and X_i is the specific global coordinate component of the element node. The Jacobian matrix also contains information that is used for assessing mesh quality, which will be discussed in Section 2.2.

2.2 Mesh Quality Analysis

A critical criterion for ensuring the reliability and efficiency of numerical simulations is the quality of the computational mesh [7]. Although automatic mesh generators are now widely utilized, the initial discretizations they produce are often prone to a small number of poor-quality elements [7]. These deficient elements can result in an ill-conditioned stiffness matrix, which can severely compromise the stability and convergence of the Finite Element (FE) solver and undermine the accuracy of the resulting Partial Differential Equation (PDE) solutions [7].

To address this, the `geometric_analysis` module developed in this thesis provides tools to analyze the quality of each element individually. This analysis relies solely on the element's

geometry, independent of material properties or physics, enabling the identification of specific elements that may degrade simulation performance. Formally, an element quality metric is a scalar function of node positions that quantifies a specific geometric property of the element [22]. To effectively evaluate the mesh, well-designed metrics are required. Ideally, these metrics should be scale-invariant to provide a consistent overview of quality across the entire mesh, independent of element size. The notable exception is the determinant of the Jacobian, which is retained as a size-dependent metric to explicitly quantify element volume. The framework adopted in this thesis is based on the algebraic mesh quality metrics proposed by Knupp [22], Jacobian ratio, and the determinant of the Jacobian matrix [6, 24].

2.2.1 Mathematical Formulation and Data Extraction

The Jacobian matrix $\mathbf{J}$ contains the necessary information regarding the mesh quality, such as volume, shape, and orientation of an element [22]. First, the Jacobian matrix must be checked for element validity at all evaluation points, which in this work include the element nodes and the centroid [33]. A non-positive Jacobian determinant ($\det(\mathbf{J}) \leq 0$) indicates a degenerate or inverted element, rendering further geometric quality analysis redundant as the element is already numerically invalid [33]. For valid elements, the Jacobian is evaluated at the evaluation point where the determinant reaches its minimum value. This minimum value is then utilized as the baseline for subsequent mesh quality analysis [33].

When the dimension of the mesh element differs from the mesh dimension (e.g., a 2D shell in 3D space), the Jacobian matrix $\mathbf{J}$ is non-square. To generalize the quality analysis for both square and non-square cases, we define a working matrix $\mathbf{A}$ to be used in the algebraic decompositions. For elements where the element dimension d_{el} equals the mesh dimension d_{mesh} (e.g., a 2D quad in 2D space), the Jacobian is square, and we set $\mathbf{A} = \mathbf{J}$. However, for elements where $d_{\text{el}} < d_{\text{mesh}}$, we construct the square metric tensor $\mathbf{A} = \mathbf{J}\mathbf{J}^T$.

While the element volume (or area) is defined as $\sqrt{\det(\mathbf{A})}$, purely geometric properties must be isolated from size and orientation using the matrix $\mathbf{A}$ directly. A QR factorization of $\mathbf{A}$ is performed [22]:

$$\mathbf{A} = \mathbf{R}\mathbf{U} = \mu\mathbf{R}\mathbf{S} = \mu\mathbf{R}\mathbf{Q}\mathbf{D}, \tag{2.5}$$

where $\mathbf{R}$ is the orientation matrix, $\mathbf{U}$ is the volume matrix, $\mu = \sqrt{\lambda_{00}}$ is the scale factor, $\mathbf{S}$ is the shape matrix, $\mathbf{Q}$ is the skew matrix, and $\mathbf{D}$ is the length ratio matrix. The auxiliary tensor terms are denoted as $\lambda_{ij} = [\mathbf{A}\mathbf{A}^T]_{ij}$, and the determinant for Knupp's metrics is given by $\alpha = \det(\mathbf{A})$. It is important to distinguish this from the Jacobian Ratio calculation: while Knupp's metrics utilize α (which equals $\det(\mathbf{J}\mathbf{J}^T)$ in non-square cases), the Jacobian Ratio metric utilizes the volume measure $\sqrt{\det(\mathbf{A})}$. The specific matrix decompositions for different element dimensions are presented below.

For 1D elements, the decomposition is trivial as skewness and orientation are not defined. The decomposition yields scalars where $\mathbf{R} = [1]$, the shape components are identity $\mathbf{Q} = [1]$ and $\mathbf{D} = [1]$, and the scale factor is simply the matrix entry $\mu = A_{00}$.

For 2D elements, the working matrix $\mathbf{A}$ is 2×2. The decomposition is defined as [22]:

$$\mathbf{R} = \frac{1}{\sqrt{\lambda_{00}}}\begin{bmatrix} A_{00} & -A_{10} \\ A_{10} & A_{00} \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} 1 & \lambda_{01}/\sqrt{\lambda_{00}\lambda_{11}} \\ 0 & \alpha/\sqrt{\lambda_{00}\lambda_{11}} \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{\lambda_{11}/\lambda_{00}} \end{bmatrix}. \tag{2.6}$$

For 3D elements, let the columns of $\mathbf{A}$ be $\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2$. The decomposition is derived as follows

[22]:

$$\mathbf{R} = \begin{bmatrix} \frac{\mathbf{c}_0}{\sqrt{\lambda_{00}}} & \frac{\lambda_{00}\mathbf{c}_1 - \lambda_{01}\mathbf{c}_0}{\sqrt{\lambda_{00}}|\mathbf{c}_0 \times \mathbf{c}_1|} & \frac{\mathbf{c}_0 \times \mathbf{c}_1}{|\mathbf{c}_0 \times \mathbf{c}_1|} \end{bmatrix},$$
$$\mathbf{Q} = \begin{bmatrix} 1 & \frac{\lambda_{01}}{\sqrt{\lambda_{00}\lambda_{11}}} & \frac{\lambda_{02}}{\sqrt{\lambda_{00}\lambda_{22}}} \\ 0 & \frac{|\mathbf{c}_0 \times \mathbf{c}_1|}{\sqrt{\lambda_{00}\lambda_{11}}} & \frac{\lambda_{00}\lambda_{12} - \lambda_{01}\lambda_{02}}{\sqrt{\lambda_{00}\lambda_{22}}|\mathbf{c}_0 \times \mathbf{c}_1|} \\ 0 & 0 & \frac{\alpha}{\sqrt{\lambda_{22}}|\mathbf{c}_0 \times \mathbf{c}_1|} \end{bmatrix}, \tag{2.7}$$
$$\mathbf{D} = \mathrm{diag}\left(1, \frac{\sqrt{\lambda_{11}}}{\sqrt{\lambda_{00}}}, \frac{\sqrt{\lambda_{22}}}{\sqrt{\lambda_{00}}}\right).$$

2.2.2 Implemented Quality Metrics

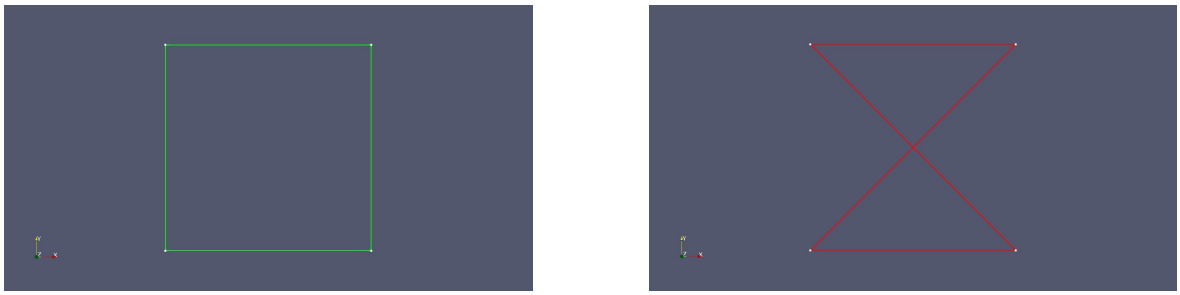
The following metrics are derived from the Jacobian matrix and its decompositions.

Volume Metric

The volume metric represents the physical space occupied by an element at the evaluation point [22]. For a square Jacobian matrix, this is simply the determinant. For non-square matrices (e.g., surfaces in 3D), it is the square root of the determinant of the metric tensor:

$$\mathrm{Vol}(\mathbf{J}) = \begin{cases} \det(\mathbf{J}) & \text{if } d_{\mathrm{el}} = d_{\mathrm{mesh}} \\ \sqrt{\det(\mathbf{J}\mathbf{J}^T)} & \text{if } d_{\mathrm{el}} < d_{\mathrm{mesh}} \end{cases}. \tag{2.8}$$

Figure 2.2 illustrates elements with optimal and invalid volume metrics.



(a) Good volume (Vol = 1.000). **(b)** Bad volume (Vol ≤ 0.000).

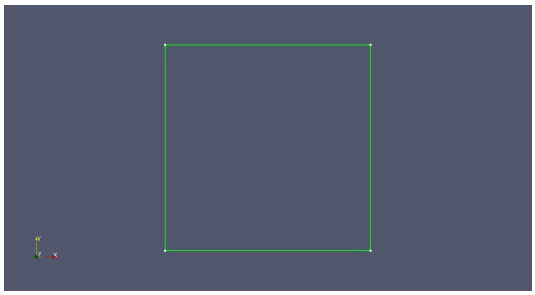
Figure 2.2: Comparison of elements based on the volume metric.

Jacobian Ratio

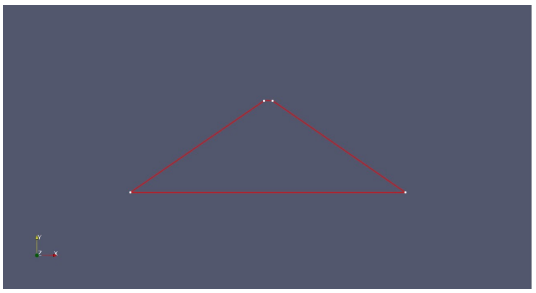
The Jacobian Ratio (JR) measures the distortion of the mesh element relative to its reference configuration. It is defined as the ratio between the minimum and maximum determinant of the Jacobian measured across all evaluation points [6]:

$$\mathrm{JR} = \begin{cases} \frac{\min(\det \mathbf{J})}{\max(\det \mathbf{J})} & \text{if } d_{\mathrm{el}} = d_{\mathrm{mesh}} \\ \frac{\min\left(\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\right)}{\max\left(\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\right)} & \text{if } d_{\mathrm{el}} < d_{\mathrm{mesh}} \end{cases}. \tag{2.9}$$

According to commercial FE analysis software ANSYS, a good quality element has JR values within the interval $[0.0\overline{3}, 1]$ [6]. The visual comparison is presented in Figure 2.3.



(a) Good Jacobian Ratio (JR = 1.000).



(b) Bad Jacobian Ratio (JR = 0.030).

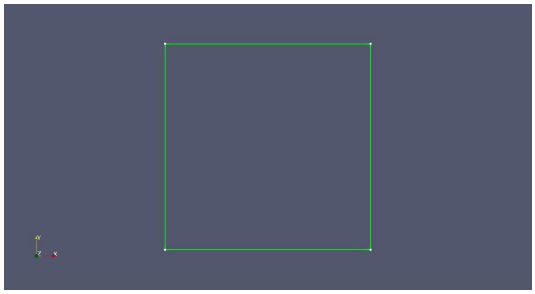
Figure 2.3: Comparison of elements based on the Jacobian Ratio.

Shape Metric

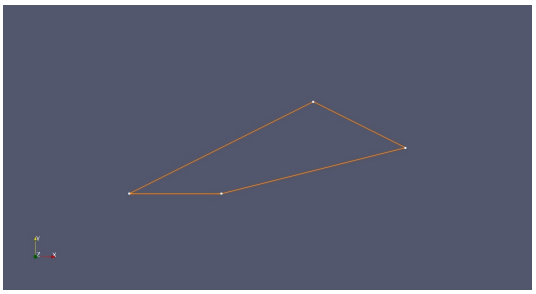
The shape metric is sensitive to element skew and length ratio. It is derived from the Frobenius condition number of the working matrix $\mathbf{A}$ [22]:

$$\text{Shape}(\mathbf{A}) = \frac{n}{\|\mathbf{A}\|_F \, \|\mathbf{A}^{-1}\|_F} \in [0,1], \tag{2.10}$$

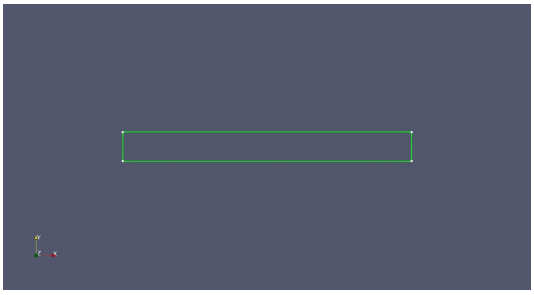
where n is the dimension. A value of 1.0 represents an ideal equilateral shape, whereas values approaching 0.0 indicate a degenerate element. Figure 2.4 displays three elements with varying shape quality to illustrate this sensitivity.



(a) Good shape (Shape = 1.000).



(b) Bad shape due to skew (Shape = 0.191).



(c) Bad shape due to length ratio (Shape = 0.198).

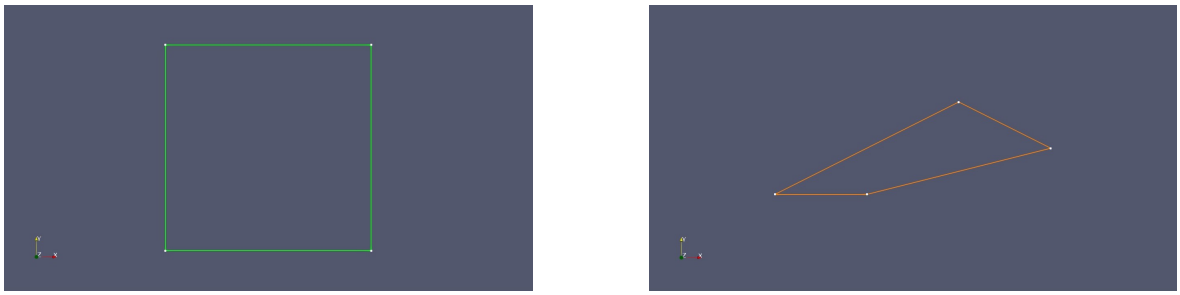
Figure 2.4: Comparison of elements based on the shape metric.

Skewness Metric

The skewness metric isolates angular distortion using the skew matrix $\mathbf{Q}$ [22]:

$$\text{Skew}(\mathbf{Q}) = \frac{n}{\|\mathbf{Q}\|_F \, \|\mathbf{Q}^{-1}\|_F} \in [0,1]. \tag{2.11}$$

A value of 1.0 signifies perfect orthogonality, while 0.0 indicates collapsed corners. This effect is shown in Figure 2.5.



(a) Good skewness ($\mathrm{Skew} = 1.000$). **(b)** Bad skewness ($\mathrm{Skew} = 0.243$).

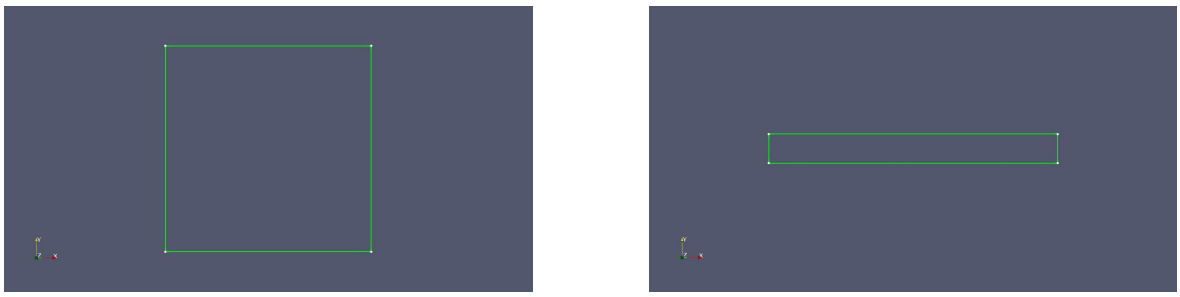
Figure 2.5: Comparison of elements based on the skewness metric.

Length Ratio Metric

Derived from the length ratio matrix $\mathbf{D}$, this metric measures the stretching of element edges [22]:

$$\mathrm{Len}(\mathbf{D}) = \frac{n}{\|\mathbf{D}\|_F \, \|\mathbf{D}^{-1}\|_F} \in [0,1]. \tag{2.12}$$

A value of 1.0 indicates that edge lengths are in proportion to the reference element. Examples of good and poor length ratios are provided in Figure 2.6.



(a) Good length ratio ($\mathrm{Len} = 1.000$). **(b)** Bad length ratio ($\mathrm{Len} = 0.198$).

Figure 2.6: Comparison of elements based on the length ratio metric.

Orientation Metric

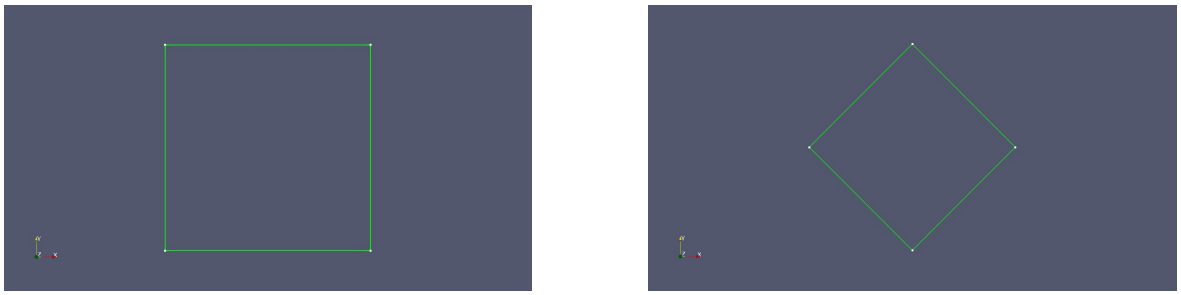
The orientation metric measures the spatial alignment of the element's local coordinate frame relative to a global frame using the orientation matrix $\mathbf{R}$ [22]:

$$\mathrm{Ori}(\mathbf{R}) = 1 + \frac{\mathrm{trace}(\mathbf{R}) - n}{4} \in [0,1]. \tag{2.13}$$

The metric is maximized at 1.0 when the frames are perfectly aligned ($\mathbf{R} = \mathbf{I}_n$). The impact of misalignment is illustrated in Figure 2.7.

2.3 FEM Formulations and Derivation of Element Mass and Stiffness Matrices

This work does not address the solution of the global FEM equations. However, the integrands of the element mass and stiffness matrices are required to analyze the optimum number of Gaussian points for computing these matrices.



(a) Good orientation (Ori = 1.000). **(b)** Bad orientation (Ori = 0.146).

Figure 2.7: Comparison of elements based on the orientation metric.

2.3.1 Equations for Element Mass and Stiffness Matrices in Physical Domain

The integral of the element mass matrix in the physical domain is given by [5, eq. 4.25]:

$$\mathbf{M} = \int_{V^{\text{phys}}} \rho^{\text{phys}}\, \mathbf{N}^{\text{phys}\,T}\, \mathbf{N}^{\text{phys}}\, \mathrm{d}V^{\text{phys}}, \tag{2.14}$$

where ρ^{phys} is the density of the element in global coordinates, $\mathbf{N}^{\text{phys}}$ is the shape function matrix in global coordinates, and V^{phys} is the physical volume. The shape function matrix $\mathbf{N}^{\text{phys}}$ is constructed using the Kronecker product ($\otimes$) of the row vector of scalar shape functions and the identity matrix $\mathbf{I}_{\text{dim}}$. For the 3D case (dim = 3), this operation expands the scalar functions to map to the x, y, and z displacement components at each node, resulting in:

$$\mathbf{N}^{\text{phys}} = \begin{bmatrix} N_0^{\text{phys}} & 0 & 0 & \dots & N_{N_{\text{nodes}}-1}^{\text{phys}} & 0 & 0 \\ 0 & N_0^{\text{phys}} & 0 & \dots & 0 & N_{N_{\text{nodes}}-1}^{\text{phys}} & 0 \\ 0 & 0 & N_0^{\text{phys}} & \dots & 0 & 0 & N_{N_{\text{nodes}}-1}^{\text{phys}} \end{bmatrix}. \tag{2.15}$$

The integral of the element stiffness matrix in the physical domain is [5, eq. 4.19]:

$$\mathbf{K} = \int_{V^{\text{phys}}} \mathbf{B}^{\text{phys}\,T}\, \mathbf{C}^{\text{phys}}\, \mathbf{B}^{\text{phys}}\, \mathrm{d}V^{\text{phys}}, \tag{2.16}$$

where $\mathbf{C}^{\text{phys}}$ is the elasticity matrix of the element in global coordinates. The elasticity matrix $\mathbf{C}$ varies in size depending on the spatial dimensions: it is a 1×1 matrix for the 1D case, a 3×3 matrix for the 2D case, and a 6×6 matrix for the 3D case [5, Tab. 4.3].

The strain-displacement matrix $\mathbf{B}^{\text{phys}}$ is defined in global coordinates and constructed by differentiating and appropriately combining the rows of the shape function matrix $\mathbf{N}^{\text{phys}}$ [5, p. 162].

For the 1D case, $\mathbf{B}^{\text{phys}}$ is defined as [10, eq. 2.2-6]:

$$\mathbf{B}^{\text{phys}}(x) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.17}$$

For the 2D case, $\mathbf{B}^{\text{phys}}$ is defined as [36, eq. 4.10b]:

$$\mathbf{B}^{\text{phys}}(x,y) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial y} & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} \\ \frac{\partial N_0^{\text{phys}}}{\partial y} & \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.18}$$

For the 3D case, $\mathbf{B}^{\text{phys}}$ is defined as [36, eq, 6.11]:

$$\mathbf{B}^{\text{phys}}(x,y,z) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial y} & 0 & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & 0 \\ 0 & 0 & \frac{\partial N_0^{\text{phys}}}{\partial z} & \dots & 0 & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} \\ \frac{\partial N_0^{\text{phys}}}{\partial y} & \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial z} & \frac{\partial N_0^{\text{phys}}}{\partial y} & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} \\ \frac{\partial N_0^{\text{phys}}}{\partial z} & 0 & \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.19}$$

2.3.2 Equations for Element Mass and Stiffness Matrices in Isoparametric Domain

The integral equations presented in Section 2.3.1 are valid for the physical domain. However, as discussed in Section 2.1, numerical evaluation is more efficient in the isoparametric domain. To perform integration in this domain, a coordinate transformation is required.

To transform ρ^{phys} to ρ^{iso} and $\mathbf{C}^{\text{phys}}$ to $\mathbf{C}^{\text{iso}}$, the coordinate mapping functions are used [36, eq. 9.2]:

$$x(\xi,\eta,\psi) = \sum_{i=0}^{N_{\text{nodes}}-1} N_i^{\text{iso}} X_i, \quad y(\xi,\eta,\psi) = \sum_{i=0}^{N_{\text{nodes}}-1} N_i^{\text{iso}} Y_i, \quad z(\xi,\eta,\psi) = \sum_{i=0}^{N_{\text{nodes}}-1} N_i^{\text{iso}} Z_i. \tag{2.20}$$

By substituting these maps into the physical density and elasticity expressions, we obtain ρ^{iso} and $\mathbf{C}^{\text{iso}}$.

The strain-displacement matrix $\mathbf{B}$ consists of spatial derivatives. Consequently, a direct transformation via simple coordinate mapping is impossible. To transform these derivative terms, the Jacobian matrix is required. Direct multiplication of $\mathbf{B}^{\text{phys}}$ by the inverse of the Jacobian yields incorrect results because $\mathbf{B}^{\text{phys}}$ contains specific combinations of shape function derivatives. To correctly derive the physical gradients, the matrix of shape function derivatives in the isoparametric space is first multiplied by the inverse of the Jacobian matrix:

$$\nabla \mathbf{N}^{\text{phys}}(\xi,\eta,\psi) = \mathbf{J}^{-1} \nabla \mathbf{N}^{\text{iso}}(\xi,\eta,\psi), \tag{2.21}$$

where the matrix of isoparametric shape function derivatives, $\nabla \mathbf{N}^{\text{iso}}$, is defined as:

$$\nabla \mathbf{N}^{\text{iso}}(\xi,\eta,\psi) = \begin{bmatrix} \frac{\partial N_0^{\text{iso}}}{\partial \xi} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{iso}}}{\partial \xi} \\ \frac{\partial N_0^{\text{iso}}}{\partial \eta} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{iso}}}{\partial \eta} \\ \frac{\partial N_0^{\text{iso}}}{\partial \psi} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{iso}}}{\partial \psi} \end{bmatrix}. \tag{2.22}$$

For the 1D case, the resulting $\mathbf{B}^{\text{iso}}$ matrix containing the computed physical derivatives is:

$$\mathbf{B}^{\text{iso}}(\xi) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.23}$$

For the 2D case, the $\mathbf{B}^{\text{iso}}$ matrix is:

$$\mathbf{B}^{\text{iso}}(\xi,\eta) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial y} & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} \\ \frac{\partial N_0^{\text{phys}}}{\partial y} & \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.24}$$

For the 3D case, the $\mathbf{B}^{\text{iso}}$ matrix is:

$$\mathbf{B}^{\text{iso}}(\xi,\eta,\psi) = \begin{bmatrix} \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial y} & 0 & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & 0 \\ 0 & 0 & \frac{\partial N_0^{\text{phys}}}{\partial z} & \dots & 0 & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} \\ \frac{\partial N_0^{\text{phys}}}{\partial y} & \frac{\partial N_0^{\text{phys}}}{\partial x} & 0 & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} & 0 \\ 0 & \frac{\partial N_0^{\text{phys}}}{\partial z} & \frac{\partial N_0^{\text{phys}}}{\partial y} & \dots & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial y} \\ \frac{\partial N_0^{\text{phys}}}{\partial z} & 0 & \frac{\partial N_0^{\text{phys}}}{\partial x} & \dots & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial z} & 0 & \frac{\partial N_{N_{\text{nodes}}-1}^{\text{phys}}}{\partial x} \end{bmatrix}. \tag{2.25}$$

Simply transforming the coordinates is insufficient; the integrand must be multiplied by the determinant of the Jacobian matrix. This scales the differential volume (or area/length) from the physical element to the isoparametric element. To generalize this relationship for cases involving non-square Jacobian matrices, the differential volume element is defined as [10, eq. 4.4-8] [18, eqs. 4.97 and 4.98]:

$$\mathrm{d}V^{\text{phys}} = \sqrt{\det(\mathbf{J}\mathbf{J}^T)}\,\mathrm{d}V^{\text{iso}}. \tag{2.26}$$

Following these transformation steps, the integrals for the element matrices are evaluated in the isoparametric domain. The 3D formulations implemented in the code are presented below:

$$\mathbf{M} = \int_0^1\int_0^1\int_0^1 \rho^{\text{iso}}\,\mathbf{N}^{\text{iso}\,T}\,\mathbf{N}^{\text{iso}}\,\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\,\mathrm{d}\xi\mathrm{d}\eta\mathrm{d}\psi, \tag{2.27}$$

$$\mathbf{K} = \int_0^1\int_0^1\int_0^1 \mathbf{B}^{\text{iso}\,T}\,\mathbf{C}^{\text{iso}}\,\mathbf{B}^{\text{iso}}\,\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\,\mathrm{d}\xi\mathrm{d}\eta\mathrm{d}\psi. \tag{2.28}$$

2.4 Derivation of Error Bound for the Gauss Quadrature

In this section, the error formula for the required number of Gaussian points is derived step by step.

2.4.1 Derivation of the Gauss Quadrature Error Formula

The derivation of the Gauss-Legendre quadrature rule begins with the objective of approximating the integral of a function $f(x)$ over an interval $[a,b]$ using a weighted sum of function values [2, eq. 5.3.1]:

$$I_n(f) = \sum_{j=1}^{n} w_j f(x_j). \tag{2.29}$$

While standard interpolation utilizes n nodes to achieve precision for polynomials of degree $n-1$, Gaussian quadrature aims to utilize the $2n$ parameters (weights w_j and nodes x_j) to achieve precision for polynomials of degree $2n-1$ [2, ch. 5.3, case 3].

To derive the optimal nodes and weights, we utilize Hermite interpolation. We construct a polynomial $H_n(x)$ of degree $2n-1$ such that it matches both the function values and the derivatives at the nodes $x_1,\dots,x_n$ [2, eqs. 5.3.12 and 3.6.2]:

$$H_n(x) = \sum_{j=1}^{n} f(x_j)h_j(x) + \sum_{j=1}^{n} f'(x_j)\tilde{h}_j(x), \tag{2.30}$$

where the basis functions are defined as:

$$h_j(x) = [1-2l_j'(x_j)(x-x_j)][l_j(x)]^2, \quad \tilde{h}_j(x) = (x-x_j)[l_j(x)]^2, \tag{2.31}$$

and where the Lagrange basis polynomials involve the nodal polynomial $\Psi_n(x)$:

$$l_j(x) = \frac{\Psi_n(x)}{(x-x_j)\Psi'(x_j)}, \quad \Psi_n(x) = (x-x_1)\cdots(x-x_n). \tag{2.32}$$

Integrating this approximation gives [2, eq. 5.3.15]:

$$\begin{aligned}\int_a^b w(x)f(x)\,\mathrm{d}x &= \int_a^b w(x)H_n(x)\,\mathrm{d}x + \int_a^b w(x)\mathcal{E}_n(x)\,\mathrm{d}x \\ &= \sum_{j=1}^{n} f(x_j)\int_a^b w(x)h_j(x)\,\mathrm{d}x + \sum_{j=1}^{n} f'(x_j)\int_a^b w(x)\tilde{h}_j(x)\,\mathrm{d}x + \int_a^b w(x)\mathcal{E}_n(x)\,\mathrm{d}x.\end{aligned} \tag{2.33}$$

To obtain a rule that does not require the derivative term $f'(x)$, we force the sum involving derivatives to zero by requiring [2, eq. 5.3.19]:

$$\begin{aligned}0 &= \int_a^b w(x)(x-x_j)[l_j(x)]^2\,\mathrm{d}x \\ &= \int_a^b w(x)(x-x_j)\left[\frac{\Psi_n(x)}{(x-x_j)\Psi'(x_j)}\right]^2\mathrm{d}x.\end{aligned} \tag{2.34}$$

Let $\varphi_n(x)$ be the orthogonal polynomials on (a,b) with respect to the weight function $w(x)$ and having zeros $a < x_1 < \cdots < x_n < b$ [2, eq. 5.3.8]. The polynomial $\varphi_n(x)$ is represented as [2, eq. 5.3.9]:

$$\varphi_n(x) = A_n x^n + A_{n-1}x^{n-1} + \dots. \tag{2.35}$$

Since both $\varphi_n(x)$ and $\Psi_n(x)$ are of degree n and share the same zeros [2, eq. 5.3.14]:

$$\Psi_n(x) = \frac{\varphi_n(x)}{A_n}. \tag{2.36}$$

Substituting this back into the derivative integral condition yields:

$$\begin{aligned}0 &= \int_a^b w(x)(x-x_j)\left[\frac{\varphi_n(x)}{(x-x_j)\varphi'(x_j)}\right]^2\mathrm{d}x \\ &= \frac{1}{[\varphi'(x_j)]^2}\int_a^b w(x)\frac{\varphi_n^2(x)}{(x-x_j)}\,\mathrm{d}x.\end{aligned} \tag{2.37}$$

If $f(x)$ is $2n$ times continuously differentiable on $[a,b]$, then the resulting interpolation error is the integral term containing $\mathcal{E}_n(x)$ from the Hermite interpolation [2, eqs. 5.3.13, 3.2.12]:

$$\begin{aligned} E_n(f) &= \int_a^b w(x)\mathcal{E}_n(x)\,\mathrm{d}x \\ &= \int_a^b w(x)[\Psi_n(x)]^2 f[x_1,x_1,\ldots,x_n,x_n,x]\,\mathrm{d}x \\ &= f[x_1,x_1,\ldots,x_n,x_n,\xi]\int_a^b w(x)[\Psi_n(x)]^2\,\mathrm{d}x \quad (\text{for some } \xi\in[a,b]) \\ &= f[x_1,x_1,\ldots,x_n,x_n,\xi]\int_a^b w(x)\frac{\varphi_n^2(x)}{A_n^2}\,\mathrm{d}x \\ &= \frac{f^{(2n)}(\xi)}{(2n)!}\int_a^b w(x)\frac{\varphi_n^2(x)}{A_n^2}\,\mathrm{d}x, \end{aligned} \tag{2.38}$$

where the divided difference is defined as $f[x_0,\ldots,x_n,\xi]=\frac{f^{(n+1)}(\xi)}{(n+1)!}$.

2.4.2 Gauss-Legendre Quadrature Formula

In this work, Legendre polynomials $P_n(x)$ are used as the orthogonal polynomials for line, quadrilateral, and hexahedron elements. The weight function is $w(x)=1$ on the interval $[-1,1]$ [14, ch. 1.13]. The error term becomes:

$$\begin{aligned} E_n(f) &= \frac{f^{(2n)}(\xi)}{(2n)!}\int_a^b w(x)\frac{P_n^2(x)}{A_n^2}\,\mathrm{d}x \\ &= \frac{f^{(2n)}(\xi)}{(2n)!}\int_0^1 \frac{P_n^2(x)}{A_n^2}\,\mathrm{d}x \\ &= \frac{f^{(2n)}(\xi)}{A_n^2(2n)!}\int_0^1 P_n^2(x)\,\mathrm{d}x. \end{aligned} \tag{2.39}$$

The leading coefficient of the Legendre polynomial A_n will be derived from the formula adapted for the interval $[0,1]$ [28, eq. 2]:

$$P_n(x)=\sum_{k=0}^{n}(-1)^{n+k}\frac{(n+k)!}{(n-k)!(k!)^2}x^k. \tag{2.40}$$

For the term where $k=n$:

$$P_n(x)=(-1)^{2n}\frac{(2n)!}{(0)!(n!)^2}x^n=\frac{(2n)!}{(n!)^2}x^n \implies A_n=\frac{(2n)!}{(n!)^2}. \tag{2.41}$$

The norm of the Legendre polynomial for the interval $[0,1]$ is [28, eq. 3]:

$$\int_0^1 P_n^2(x)\,\mathrm{d}x=\frac{1}{2n+1}. \tag{2.42}$$

When we implement the leading coefficient and the norm of the Legendre polynomial in Equation (2.39), we get the error equation for the interval $[0,1]$ as follows:

$$E_n(f)=\frac{(n!)^4}{(2n+1)[(2n)!]^3}f^{(2n)}(\xi). \tag{2.43}$$

2.4.3 Derivation of the Error Bound for 1D Case

The function $f(\xi)$ in this thesis represents the integrand of the element Mass and Stiffness matrices, as displayed in Section 2.3.2. For error bound calculation, the function is considered as a polynomial of degree d:

$$f(\xi) = a_0 + a_1\xi + \cdots + a_d\xi^d = \sum_{k=0}^{d} a_k\xi^k. \tag{2.44}$$

Assuming $f(\xi)$ is $2n$ times continuously differentiable, the derivatives are derived as:

$$\begin{aligned}
f^{(1)}(\xi) &= a_1 + 2a_2\xi + \cdots + da_d\xi^{d-1} = \sum_{k=1}^{d} ka_k\xi^{k-1} \\
f^{(2)}(\xi) &= 2a_2 + 6a_3\xi + \cdots + d(d-1)a_d\xi^{d-2} = \sum_{k=2}^{d} k(k-1)a_k\xi^{k-2} \\
&\vdots \\
f^{(2n)}(\xi) &= (2n)!a_{2n} + (2n+1)!a_{2n+1}\xi + \cdots + \frac{d!}{(d-2n)!}a_d\xi^{d-2n} = \sum_{k=2n}^{d} \frac{k!}{(k-2n)!}a_k\xi^{k-2n}.
\end{aligned} \tag{2.45}$$

Since we do not want to calculate the errors individually, we create an upper bound for the error of the Gauss-Legendre Quadrature for the interval $[0,1]$. The ξ^{k-2n} term is neglected because its maximum value in this interval is 1. The resulting upper bound is:

$$|E_n(f)| \leq \frac{(n!)^4}{(2n+1)[(2n)!]^3} \left|f^{(2n)}(\xi)\right|, \tag{2.46}$$

which expands to:

$$|E_n(f)| \leq \frac{(n!)^4}{(2n+1)[(2n)!]^3} \sum_{k=2n}^{d} \frac{k!}{(k-2n)!} |a_k|. \tag{2.47}$$

2.4.4 Extension to Multidimensional Cases

The derivation of error bounds for multidimensional elements, such as quadrilaterals and hexahedra, relies on the property that their shape functions and integration domains can be represented as tensor products of 1D domains. Consequently, the multidimensional quadrature rule is constructed as a product of 1D Gauss-Legendre rules [17, Rem. 4.9].

Derivation of Error Bound for 2D Quadrilateral Elements

For a 2D quadrilateral element, the integration is performed over the isoparametric square domain $V^{\text{iso}} = [0,1] \times [0,1]$ in the (ξ, η) plane. Let $I(f)$ be the exact integral and $Q_{n,n}(f)$ be the numerical approximation using n points in each direction:

$$Q_{n,n}(f) = \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j f(\xi_i, \eta_j). \tag{2.48}$$

The total error $E_{2D}(f)$ is defined as the difference between the exact integral and the quadrature approximation:

$$E_{2D}(f) = \int_0^1 \int_0^1 f(\xi,\eta)\,\mathrm{d}\xi\mathrm{d}\eta - \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j f(\xi_i,\eta_j). \tag{2.49}$$

To analyze this error, we use the standard technique of adding and subtracting an intermediate "hybrid" term H. This term represents a semi-discrete integration where the ξ-direction is integrated numerically, but the η-direction is integrated exactly:

$$H = \sum_{i=1}^{n} w_i \int_0^1 f(\xi_i,\eta)\,\mathrm{d}\eta. \tag{2.50}$$

We decompose the total error into two components: the error accumulated from the ξ-integration and the error from the η-integration:

$$\begin{aligned} E_{2D}(f) &= (I(f)-H) + (H - Q_{n,n}(f)) \\ &= \left(\int_0^1 \int_0^1 f(\xi,\eta)\mathrm{d}\xi\mathrm{d}\eta - \sum_{i=1}^{n} w_i \int_0^1 f(\xi_i,\eta)\mathrm{d}\eta\right) \\ &\quad + \left(\sum_{i=1}^{n} w_i \int_0^1 f(\xi_i,\eta)\mathrm{d}\eta - \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j f(\xi_i,\eta_j)\right). \end{aligned} \tag{2.51}$$

In the first term, we pull the integration over η to the outside. The term inside the parentheses then becomes exactly the 1D quadrature error with respect to ξ, denoted as E_ξ:

$$I(f)-H = \int_0^1 \left(\int_0^1 f(\xi,\eta)\mathrm{d}\xi - \sum_{i=1}^{n} w_i f(\xi_i,\eta)\right)\mathrm{d}\eta = \int_0^1 E_\xi[f(\cdot,\eta)]\,\mathrm{d}\eta. \tag{2.52}$$

In the second term, we factor out the common summation $\sum w_i$. The term inside the parentheses becomes the 1D quadrature error with respect to η, denoted as E_η, evaluated at the fixed node ξ_i:

$$H - Q_{n,n}(f) = \sum_{i=1}^{n} w_i \left(\int_0^1 f(\xi_i,\eta)\mathrm{d}\eta - \sum_{j=1}^{n} w_j f(\xi_i,\eta_j)\right) = \sum_{i=1}^{n} w_i E_\eta[f(\xi_i,\cdot)]. \tag{2.53}$$

Combining these results gives the decomposed error formula:

$$E_{2D}(f) = \int_0^1 E_\xi[f(\cdot,\eta)]\,\mathrm{d}\eta + \sum_{i=1}^{n} w_i E_\eta[f(\xi_i,\cdot)]. \tag{2.54}$$

To obtain a loose upper bound, we apply the triangle inequality ($|A+B| \le |A| + |B|$) to the decomposed error terms:

$$|E_{2D}(f)| \le \int_0^1 \left|E_\xi[f(\cdot,\eta)]\right|\mathrm{d}\eta + \sum_{i=1}^{n} |w_i| \left|E_\eta[f(\xi_i,\cdot)]\right|. \tag{2.55}$$

Since the 1D error $E_n(g)$ for a function g is bounded by $C_n \max|g^{(2n)}|$, we can bound the terms inside the integral and summation by the maximum magnitude of the partial derivatives over the domain. We define the 1D error constant C_n derived in the previous section as:

$$C_n = \frac{(n!)^4}{(2n+1)[(2n)!]^3}. \tag{2.56}$$

Substituting this bound into the inequality yields:

$$|E_{2D}(f)| \leq \left(C_n \max_{(\xi,\eta)} \left| \frac{\partial^{2n} f}{\partial \xi^{2n}} \right| \right) \int_0^1 \mathrm{d}\eta + \left(C_n \max_{(\xi,\eta)} \left| \frac{\partial^{2n} f}{\partial \eta^{2n}} \right| \right) \sum_{i=1}^{n} |w_i|. \tag{2.57}$$

Finally, utilizing the properties that the integration interval length is $\int_0^1 \mathrm{d}\eta = 1$ and the sum of weights is $\sum w_i = 1$, we arrive at the simplified bound:

$$|E_{2D}(f)| \leq C_n \left(\left| \frac{\partial^{2n} f}{\partial \xi^{2n}}(\xi,\eta) \right| + \left| \frac{\partial^{2n} f}{\partial \eta^{2n}}(\xi,\eta) \right| \right). \tag{2.58}$$

Next, we apply this bound to a general polynomial function $f(\xi,\eta)$ with degree d_1 in ξ and d_2 in η:

$$f(\xi,\eta) = \sum_{i=0}^{d_1} \sum_{j=0}^{d_2} a_{ij} \xi^i \eta^j. \tag{2.59}$$

When we differentiate this polynomial $2n$ times with respect to ξ, any term where the power $i < 2n$ vanishes. For terms where $i \geq 2n$, the power rule applies $2n$ times, resulting in the factorial coefficient $\frac{i!}{(i-2n)!}$. The same logic applies to the derivatives with respect to η:

$$\frac{\partial^{2n} f}{\partial \xi^{2n}} = \sum_{i=2n}^{d_1} \sum_{j=0}^{d_2} a_{ij} \frac{i!}{(i-2n)!} \xi^{i-2n} \eta^j. \tag{2.60}$$

To finalize the loose upper bound, we maximize the expression by setting the variables ξ and η to 1 (since the domain is $[0,1]$). Substituting these maximum derivative values into the error bound formula gives the final result:

$$|E_{2D}(f)| \leq C_n \left[\sum_{i=2n}^{d_1} \sum_{j=0}^{d_2} |a_{ij}| \frac{i!}{(i-2n)!} + \sum_{i=0}^{d_1} \sum_{j=2n}^{d_2} |a_{ij}| \frac{j!}{(j-2n)!} \right]. \tag{2.61}$$

Derivation of Error Bound for 3D Hexahedral Elements

For a 3D hexahedral element, the integration is performed over the isoparametric cubic domain $V^{\mathrm{iso}} = [0,1]^3$. Let $I(f)$ be the exact integral and $Q_{n,n,n}(f)$ be the numerical approximation using n points in each direction:

$$Q_{n,n,n}(f) = \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{k=1}^{n} w_i w_j w_k f(\xi_i, \eta_j, \psi_k). \tag{2.62}$$

The total error $E_{3D}(f)$ is defined as the difference between the exact integral and the quadrature approximation:

$$E_{3D}(f) = \int_0^1 \int_0^1 \int_0^1 f(\xi,\eta,\psi)\, \mathrm{d}\xi \mathrm{d}\eta \mathrm{d}\psi - Q_{n,n,n}(f). \tag{2.63}$$

To analyze this error, we extend the technique of adding and subtracting intermediate hybrid terms. We introduce two such terms, H_1 and H_2, which represent stages of partial numerical

integration:

$$H_1 = \sum_{i=1}^{n} w_i \int_0^1 \int_0^1 f(\xi_i, \eta, \psi)\, \mathrm{d}\eta \mathrm{d}\psi, \tag{2.64}$$

$$H_2 = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \int_0^1 f(\xi_i, \eta_j, \psi)\, \mathrm{d}\psi. \tag{2.65}$$

We decompose the total error into three components corresponding to the error accumulation in each direction:

$$E_{3D}(f) = (I(f) - H_1) + (H_1 - H_2) + (H_2 - Q_{n,n,n}(f)). \tag{2.66}$$

The first component, $I(f) - H_1$, captures the error associated with the integration in the ξ-direction. We can rewrite $I(f)$ by pulling the exact integrations over η and ψ to the outside:

$$I(f) - H_1 = \int_0^1 \int_0^1 \left(\int_0^1 f(\xi, \eta, \psi) \mathrm{d}\xi - \sum_{i=1}^{n} w_i f(\xi_i, \eta, \psi) \right) \mathrm{d}\eta \mathrm{d}\psi. \tag{2.67}$$

The term inside the parentheses is exactly the 1D quadrature error with respect to ξ, denoted as $E_\xi[f(\cdot, \eta, \psi)]$. Thus, the first component becomes $\int \int E_\xi \, \mathrm{d}\eta \mathrm{d}\psi$.

The second component, $H_1 - H_2$, captures the error in the η-direction. Here, we factor out the summation over ξ and the integration over ψ:

$$H_1 - H_2 = \sum_{i=1}^{n} w_i \int_0^1 \left(\int_0^1 f(\xi_i, \eta, \psi) \mathrm{d}\eta - \sum_{j=1}^{n} w_j f(\xi_i, \eta_j, \psi) \right) \mathrm{d}\psi. \tag{2.68}$$

The term inside the parentheses represents the 1D quadrature error with respect to η, evaluated at the fixed node ξ_i. This component simplifies to $\sum w_i \int E_\eta \, \mathrm{d}\psi$.

The third component, $H_2 - Q_{n,n,n}(f)$, captures the error in the ψ-direction. We factor out the summations for both ξ and η:

$$H_2 - Q_{n,n,n}(f) = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \left(\int_0^1 f(\xi_i, \eta_j, \psi) \mathrm{d}\psi - \sum_{k=1}^{n} w_k f(\xi_i, \eta_j, \psi_k) \right). \tag{2.69}$$

The term inside the parentheses is the 1D quadrature error with respect to ψ. This component simplifies to $\sum \sum w_i w_j E_\psi$.

Combining these results gives the fully decomposed error formula:

$$E_{3D}(f) = \int_0^1 \int_0^1 E_\xi \, \mathrm{d}\eta \mathrm{d}\psi + \sum_{i=1}^{n} w_i \int_0^1 E_\eta \, \mathrm{d}\psi + \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j E_\psi. \tag{2.70}$$

To obtain a loose upper bound, we apply the triangle inequality ($|A + B| \leq |A| + |B|$) and substitute the 1D error bound $C_n \max |g^{(2n)}|$. Utilizing the properties that the integration length is 1 and the sum of weights is 1, we obtain:

$$|E_{3D}(f)| \leq C_n \left(\left| \frac{\partial^{2n} f}{\partial \xi^{2n}} \right| + \left| \frac{\partial^{2n} f}{\partial \eta^{2n}} \right| + \left| \frac{\partial^{2n} f}{\partial \psi^{2n}} \right| \right). \tag{2.71}$$

Finally, applying this bound to a general polynomial function $f(\xi, \eta, \psi) = \sum \sum \sum a_{ijk} \xi^i \eta^j \psi^k$ and maximizing the expression by setting the variables ξ, η, ψ to 1 (since the domain is

$[0,1]^3$), we arrive at the final loose upper bound for the hexahedral element:

$$|E_{3D}(f)| \leq C_n \Bigg[\sum_{i=2n}^{d_1} \sum_{j=0}^{d_2} \sum_{k=0}^{d_3} |a_{ijk}| \frac{i!}{(i-2n)!} + \sum_{i=0}^{d_1} \sum_{j=2n}^{d_2} \sum_{k=0}^{d_3} |a_{ijk}| \frac{j!}{(j-2n)!} + \sum_{i=0}^{d_1} \sum_{j=0}^{d_2} \sum_{k=2n}^{d_3} |a_{ijk}| \frac{k!}{(k-2n)!} \Bigg]. \tag{2.72}$$

2.4.5 Monomial Ordering

To efficiently manage polynomial operations within the codebase, a consistent method for organizing terms is required. Polynomials are constructed from sums of monomials. Let a polynomial be defined as $p(\mathbf{x}) = c_\alpha \mathbf{x}^\alpha + c_\beta \mathbf{x}^\beta + \ldots$, where a monomial is given by $\mathbf{x}^\alpha = x^{\alpha_x} y^{\alpha_y} z^{\alpha_z}$ and the exponent of the monomial is the multi-index $\alpha = (\alpha_x, \alpha_y, \alpha_z)$ [11, p. 55].

To establish a proper and standard representation, monomial ordering is essential. Since a polynomial is a sum of monomials, the terms must be arranged unambiguously in ascending or descending order. In this thesis, ascending order is preferred. To achieve this, we must be able to compare every pair of monomials to establish their relative positions. Consequently, for every pair of monomials $\mathbf{x}^\alpha$ and $\mathbf{x}^\beta$, the statement $\mathbf{x}^\beta > \mathbf{x}^\alpha$ must be true [11, p. 55].

Once monomials are ordered by their total degree (exponent), a secondary ordering is required to distinguish between monomials of the same degree. This necessitates the introduction of lexicographical ordering. Let $\alpha = (\alpha_1, \alpha_2, \alpha_3)$ and $\beta = (\beta_1, \beta_2, \beta_3)$ be in $\mathbb{Z}^n_{\geq 0}$. We say $\alpha >_{\text{lex}} \beta$ if the leftmost nonzero entry of the vector difference $\alpha - \beta \in \mathbb{Z}^n_{\geq 0}$ is positive. We write $\mathbf{x}^\alpha >_{\text{lex}} \mathbf{x}^\beta$ if $\alpha >_{\text{lex}} \beta$ [11, p. 56]. For example: $(0,1,2) >_{\text{lex}} (0,0,3)$ since the difference is $\alpha - \beta = (0,1,-1)$.

In practice, when working with polynomials in two or three variables, we denote the variables as x, y, z. We assume the alphabetical order $x > y > z$ is used to define the lexicographic ordering unless explicitly stated otherwise. Lex order is analogous to the ordering of words in a dictionary [11, p. 57]. This ordering is independent of the coordinate system used, whether physical (x,y,z) or local (ξ,η,ψ), allowing the implementation to function consistently across different dimensional spaces.

To streamline operations, full symbolic information is not stored. Instead, the implementation defines a base for monomial operations and accepts a vector of coefficients as input. Polynomials are represented as vectors of coefficients a_α corresponding to monomials $\mathbf{x}^\alpha$, arranged in lexicographic order with ascending total monomial order as described in previous paragraphs. To maintain a standardized format for the Gaussian quadrature algorithm, even 1D and 2D polynomials are represented as 3D vectors. For example, a 2D polynomial $p_0(x,y)$ is mapped to a vector containing 3D basis indices as follows:

$$p_0(x,y) = a_{00} + a_{01}y + a_{20}x^2 + a_{11}xy + a_{02}y^2 \rightarrow [a_{00}, 0, a_{01}, 0, a_{20}, a_{11}, 0, a_{02}, 0, 0]. \tag{2.73}$$

2.4.6 Implementation of Polynomial Operations

The addition of two polynomials $p_1 = \sum_\alpha a_\alpha \mathbf{x}^\alpha$ and $p_2 = \sum_\alpha b_\alpha \mathbf{x}^\alpha$ is performed by combining the coefficients of identical monomials. Because the implementation uses the coefficient

vector indexing described in Subsection 2.4.5, this is a linear operation where the sum is defined as:

$$(p_1 + p_2) = \sum_{\alpha} (a_\alpha + b_\alpha)\mathbf{x}^\alpha. \tag{2.74}$$

Multiplication of a polynomial p_1 by a scalar constant k scales every individual coefficient in the representation. This operation is expressed as:

$$k \cdot p_1 = \sum_{\alpha} (k \cdot a_\alpha)\mathbf{x}^\alpha. \tag{2.75}$$

Provided the constant $c \neq 0$, this operation scales the leading coefficient without altering the multidegree of the polynomial.

Polynomial multiplication distributes over addition. To represent the product of p_1 and p_2, we must account for the interaction between every term in the first polynomial and every term in the second. Using distinct indices α and β to represent the monomials of p_1 and p_2 respectively, the product is defined as:

$$p_1 \cdot p_2 = \left(\sum_{\alpha} a_\alpha \mathbf{x}^\alpha\right) \cdot \left(\sum_{\beta} b_\beta \mathbf{x}^\beta\right) = \sum_{\alpha,\beta} (a_\alpha \cdot b_\beta)\mathbf{x}^{\alpha+\beta}. \tag{2.76}$$

This operation utilizes the fundamental property that multiplying two monomials $\mathbf{x}^\alpha$ and $\mathbf{x}^\beta$ results in $\mathbf{x}^{\alpha+\beta}$.

2.5 Numerical Helper Functions

Directly implementing analytical equations is often computationally expensive, particularly when dealing with symbolic or high-degree polynomial entries. To optimize performance, we utilize specific numerical helper functions as efficient replacements for standard operations. This section details the mathematical formulations and approximations used throughout the codebase.

2.5.1 Inverse of a Matrix

To calculate the stiffness matrix integrand, the inverse of the Jacobian matrix, $\mathbf{J}$, is required as seen in Subsection 2.3.2. Since the entries of $\mathbf{J}$ are polynomials, direct symbolic inversion is computationally prohibitive. Instead, we employ alternative formulations based on the matrix dimensions.

For square Jacobian matrices, we utilize the adjugate-determinant formulation. This allows us to handle polynomial entries by computing the adjugate matrix and the determinant separately [32, sec. 4]:

$$\mathbf{J}^{-1} = \frac{\text{adj}(\mathbf{J})}{\det(\mathbf{J})}. \tag{2.77}$$

In cases where the Jacobian is non-square, the standard inverse, adjoint, and determinant are not defined. To resolve this, we implement the Moore-Penrose pseudoinverse, $\mathbf{J}^+$. By first forming a square symmetric matrix through the product $\mathbf{J}\mathbf{J}^T$, we can apply the inversion identity:

$$(\mathbf{J}\mathbf{J}^T)^{-1} = \frac{\text{adj}(\mathbf{J}\mathbf{J}^T)}{\det(\mathbf{J}\mathbf{J}^T)}. \tag{2.78}$$

The left-hand pseudoinverse is then constructed as follows [4, eq. 14]:

$$\mathbf{J}^{+} = \mathbf{J}^{T}(\mathbf{J}\mathbf{J}^{T})^{-1}. \tag{2.79}$$

2.5.2 Series Expansion for Reciprocal Polynomial

As discussed in Section 2.5.1, the inversion formulas require the calculation of $1/\det(\mathbf{J})$. Given that $\det(\mathbf{J})$ is a multivariate polynomial, it takes the form:

$$\det(\mathbf{J}) = a_0 + a_{100}x + a_{010}y + a_{001}z + a_{200}x^2 + a_{110}xy + a_{101}xz + \dots \tag{2.80}$$

We employ a Taylor series expansion to approximate the reciprocal using the geometric series identity [25, p. 40]:

$$\frac{1}{1-r} = \sum_{n=0}^{\infty} r^n. \tag{2.81}$$

To apply this identity, we must transform the determinant expression into the form $1-r$. First, we normalize the polynomial by its constant term a_0 to ensure the leading term is 1:

$$\frac{\det(\mathbf{J})}{a_0} = 1 + \tilde{a}_{100}x + \tilde{a}_{010}y + \tilde{a}_{001}z + \tilde{a}_{200}x^2 + \tilde{a}_{110}xy + \dots \tag{2.82}$$

Second, we define the term r such that the normalized determinant matches the denominator $1-r$ in Equation 2.81. By setting $r = 1 - \frac{\det(\mathbf{J})}{a_0}$, we obtain:

$$r = (-1)\left(\tilde{a}_{100}x + \tilde{a}_{010}y + \tilde{a}_{001}z + \tilde{a}_{200}x^2 + \tilde{a}_{110}xy + \dots\right). \tag{2.83}$$

Substituting this into the expansion yields:

$$\frac{1}{1-r} = \frac{1}{1-\left(1-\frac{\det(\mathbf{J})}{a_0}\right)} = \frac{1}{\frac{\det(\mathbf{J})}{a_0}}. \tag{2.84}$$

To reach the final expression for $1/\det(\mathbf{J})$, we scale the result by the constant $1/a_0$:

$$\frac{1}{\det(J)} = \frac{1}{a_0}\frac{1}{1-r} = \frac{1}{a_0}\frac{1}{\frac{\det(\mathbf{J})}{a_0}} = \frac{1}{a_0}\sum_{n=0}^{\infty} r^n. \tag{2.85}$$

By inserting Equation 2.83 into the summation, the reciprocal is expressed as:

$$\frac{1}{\det(J)} = \frac{1}{a_0}\sum_{n=0}^{\infty}(-1)^n\left(\tilde{a}_{100}x + \tilde{a}_{010}y + \dots\right)^n. \tag{2.86}$$

For computational efficiency, we define recursive variables to represent the powers of the polynomial expansion:

$$S_0 = 1 \tag{2.87}$$

$$S_1 = [\tilde{a}_{100}x + \tilde{a}_{010}y + \dots]^1 \tag{2.88}$$

$$S_2 = S_1 S_1 \tag{2.89}$$

$$S_3 = S_1 S_2 \tag{2.90}$$

$$\vdots$$

$$S_N = S_1 S_{N-1}. \tag{2.91}$$

The choice of the number of terms N in the expansion determines the approximation accuracy. In this implementation, we utilize $N = 5$ terms (corresponding to a polynomial degree of 4), though this parameter can be adjusted based on the required precision. The truncated Taylor expansion for the reciprocal is thus:

$$\frac{1}{\det(J)} \approx \frac{1}{a_0}(1 - S_1 + S_2 - S_3 + S_4). \tag{2.92}$$

2.5.3 Series Expansion for Square Root

In addition to the reciprocal, we must calculate the square root of the polynomial $\det(\mathbf{J}\mathbf{J}^T)$. Let this polynomial be defined as:

$$\det(\mathbf{J}\mathbf{J}^T) = b_0 + b_{100}x + b_{010}y + b_{001}z + \dots \tag{2.93}$$

We approximate this term using the Generalized Binomial Theorem [29, eq. 1]:

$$\sqrt{1+s} = \sum_{n=0}^{\infty} \binom{1/2}{n} s^n. \tag{2.94}$$

To transform the determinant into the required format, we first factor out the constant term b_0:

$$\det(\mathbf{J}\mathbf{J}^T) = b_0 \left(1 + \tilde{b}_{100}x + \tilde{b}_{010}y + \tilde{b}_{001}z + \dots\right). \tag{2.95}$$

Taking the square root of the normalized expression gives:

$$\sqrt{\det(\mathbf{J}\mathbf{J}^T)} = \sqrt{b_0}\sqrt{1 + \tilde{b}_{100}x + \tilde{b}_{010}y + \tilde{b}_{001}z + \dots} \tag{2.96}$$

We define the s from Equation 2.94 as:

$$s = \tilde{b}_{100}x + \tilde{b}_{010}y + \tilde{b}_{001}z + \dots \tag{2.97}$$

Substituting s into the generalized binomial expansion, we obtain:

$$\sqrt{\det(\mathbf{J}\mathbf{J}^T)} = \sqrt{b_0}\sqrt{1+s} = \sqrt{b_0}\sum_{n=0}^{\infty} \binom{1/2}{n} s^n. \tag{2.98}$$

Since an infinite expansion is not computationally feasible, the series is truncated. For this thesis, we utilize $N = 5$ terms (corresponding to an order of 4), resulting in the following implemented formula:

$$\sqrt{\det(\mathbf{J}\mathbf{J}^T)} \approx \sqrt{b_0}\left(1 + \frac{1}{2}s - \frac{1}{8}s^2 + \frac{1}{16}s^3 - \frac{5}{128}s^4\right). \tag{2.99}$$

Chapter 3

Implementation of the Code

In this chapter, the software implementation is explained in detail. The codebase is designed with strict modularity to facilitate future open-source contributions. This design choice ensures that the software can be easily extended to support the parsing of additional formats, such as Ansys mesh files or diverse result file structures.

Furthermore, the modular architecture permits the expansion of analytical capabilities, such as implementing alternative mesh quality metrics, determining the necessary Gaussian integration points for damping matrices or elements other than lines, quads, and hexahedra, or generating output files in formats other than VTU. Although the current implementation utilizes Gauss-Legendre error estimation, the framework is designed to accommodate alternative error calculation methods in future iterations.

3.1 Code Overview

The execution logic, contained primarily within `main.rs`, orchestrates the translation and analysis pipeline. The process begins with identifying the input mesh format. By analyzing the file extension, the software determines whether the input is a COMSOL Multiphysics Output (`.mphtxt`) or an Abaqus Output (`.inp`) and dispatches the appropriate parser. Concurrently, if auxiliary result files are provided via command-line arguments, the software parses nodal field data from `.txt` files. Following parsing, these heterogeneous data sources are unified and transformed into the VTU (Unstructured Grid) format, enabling direct visualization in ParaView.

The second phase of execution, illustrated in Figure 3.1, focuses on mesh quality analysis. The software analyzes the parsed topology using existing quality metrics. Currently, this analysis is performed on the undeformed mesh configuration, though the architecture supports future implementation of quality analysis on displaced meshes (resultant deformation). Crucially, the calculated quality metrics are injected directly into the output VTU file as scalar fields, allowing users to visually inspect element quality distribution within ParaView.

The final stage of the pipeline determines the optimal number of Gaussian quadrature points for line, quadrilateral, and hexahedral elements, including their quadratic, biquadratic, and triquadratic versions. Further error bound formulas can be implemented to analyze other element types. The system first identifies the target integration type, specifically, whether the computation is intended for an element mass matrix (M) or an element stiffness matrix (K). Using user-defined material properties and an error tolerance, the software calculates the upper error bound to identify the optimal number of Gaussian points. The results are stored and exported to the VTU file in three distinct fields for each element: the theoretical limit for exact integration, the optimal number of Gaussian points, and the difference between these

two.

To maintain code cleanliness and reusability, common data structures (such as `MeshData`, `Node`, and `Element`) and mathematical functions (such as monomial operations) are encapsulated within the `structs_and_impls` module. All error handling and custom exception types, such as formatting errors or I/O interruptions, are managed centrally within the `error` module.s

3.2 Parsing Module

The parsing logic operates as a dispatch system orchestrated by the `main.rs` controller. This controller identifies input file types and delegates processing to specific modules. For mesh generation, detection of the ".mphtxt" extension invokes the `mphtxt.rs` module, while the ".inp" extension triggers the `inp.rs` module.

Regarding field data, parsing is governed by command-line flags. The flag `-node-values`, when accompanied by a ".txt" file, activates the `txt_node.rs` module. In this scenario, the software interprets the provided file as nodal data, ensuring the correct mapping of field values to the corresponding mesh nodes.

3.2.1 Parsing of COMSOL Multiphysics Mesh Output (.mphtxt)

The `mphtxt.rs` module handles the parsing of COMSOL Multiphysics mesh exports. A specific challenge with this format is that node and element ordering is implicit; the file does not explicitly index entities, requiring the parser to maintain internal counters during the sequential read of the file.

The parser scans the file line by line using a buffered reader to extract global metadata. It specifically searches for tags identifying the spatial dimension ("`# sdim`") and the total count of nodes ("`# number of mesh vertices`"). Additionally, it identifies the "`# lowest mesh vertex index`" to determine the starting offset (0-based or 1-based indexing).

Once metadata is established, the parser searches for the "`# Mesh vertex coordinates`" tag. This trigger delegates control to a helper function, which consumes the subsequent lines to populate the global node vector. To ensure data integrity, the parser validates that the number of coordinate values per line exactly matches the previously parsed spatial dimension ($sdim$).

The parsing of elements is handled via a pattern-matching approach. The system identifies specific string identifiers in the file, such as "edg", "tet2", or "prism2", and maps them to the internal `ElementType` enum (e.g., mapping "tet2" to `ElementType::QuadraticTetra`). To systematically manage the complexity of mixed meshes, the software utilizes an internal `ElementTypeInfo` structure to track the metadata for each block. As the parser encounters different element groups (marked by the "`# Type #`" tag), it first verifies that the number of nodes provided for each element matches the expected count for that type (e.g., ensuring 10 nodes are present for a quadratic tetrahedron). Subsequently, it calculates the global element ID based on the `element_id_counter` to ensure continuous numbering across mixed element types. As shown in Figure 3.2, the parser strictly targets mesh topology. Ancillary sections such as "`# Parameters`", "`# Domains`", and the geometric topology definitions for "`# Up/down`" pairs are intentionally skipped to optimize processing speed, as they are not required for the generation of the VTU output.

Finally, a node reordering step is performed immediately after parsing. This is necessary because the internal node numbering convention utilized by COMSOL Multiphysics often

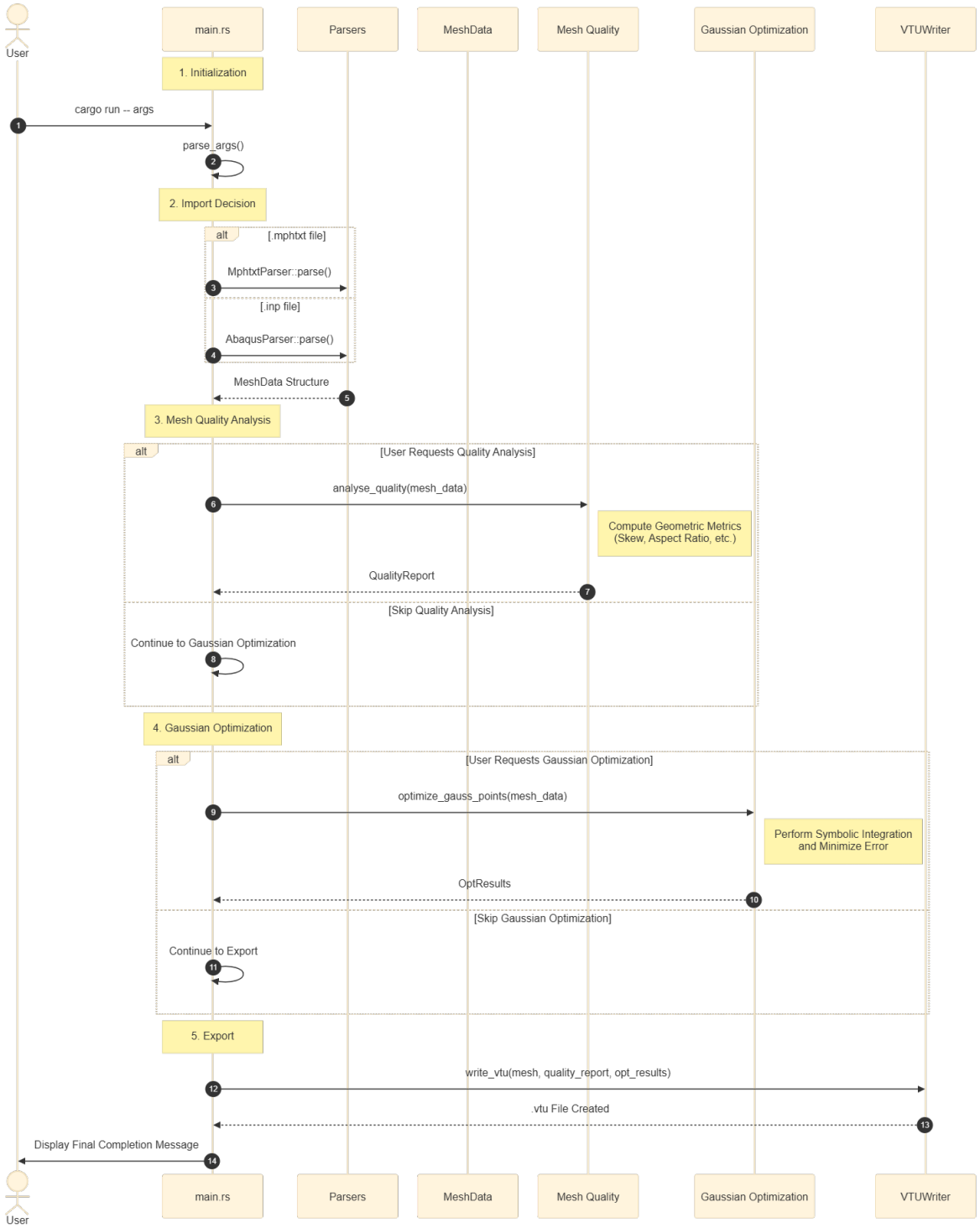


Figure 3.1: Execution sequence and system data flow. The pipeline is orchestrated by ‘main.rs‘, beginning with argument validation. Depending on the input file extension, the system dynamically dispatches the appropriate parser (‘MphtxtParser‘ or ‘AbaqusParser‘) to unify legacy formats into a standardized ‘MeshData‘ structure. The workflow features two conditional branches controlled by user flags: the **Mesh Quality** module, which computes geometric metrics such as skewness and aspect ratio, and the **Gaussian Optimization** module, which performs symbolic integration to determine optimal quadrature points. The process concludes with the ‘VTUWriter‘ serializing the mesh topology and computed results into a ‘.vtu‘ file for visualization.

differs from the VTK standard [8, 19, 20]. A detailed explanation of these node reordering algorithms is provided in Section 3.3.1.

```
# Created by COMSOL Multiphysics
...
3 # sdim
1503 # number of mesh points
0 # lowest mesh point index
# Mesh point coordinates
0 0 0
...
12.5 30 13.125
7 # number of element types

# Type #0
4 tet2 # type name
10 # number of nodes per element
318 # number of elements
# Elements
926 18 13 17 971 958 61 967 66 60
...
30 # number of parameter values per element
0 # number of parameters
# Parameters
...
318 # number of domains
# Domains
...
0 # number of up/down pairs
# Up/down
```

Figure 3.2: Abbreviated structure of a COMSOL Multiphysics .mphtxt file [9]. The parser processes topological data while ignoring auxiliary sections like Parameters, Domains, and Up/down pairs.

3.2.2 Parsing of Abaqus Input Files (.inp)

The `abaqus_inp.rs` module handles the parsing of Abaqus mesh data. Unlike the implicit, sequential structure of COMSOL files, the Abaqus format relies on an explicit ID-mapping approach where every node and element is assigned a unique integer identifier (see Figure 3.3). The parser scans the file line-by-line, utilizing a state-machine approach to identify keyword blocks such as `*NODE` and `*ELEMENT`.

A fundamental discrepancy addressed by this module is the indexing convention. Abaqus utilizes a 1-based indexing system, whereas the internal architecture of this software and the standard VTK format rely on 0-based indexing. Consequently, the parser iterates through every parsed node ID and element connectivity list, mathematically subtracting one from all indices. This ensures that the internal representation is continuous and compatible with standard visualization arrays.

Abaqus employs a complex naming convention where the element name encodes both the geometry and the physics of the simulation (e.g., `C3D8R` for a reduced-integration brick-/hexahedron). To extract the core topology, the parser implements a hierarchical string analysis method within `ElementType::from_str_abaqus`. It is critical that this hierarchy is processed in a specific order, prioritizing dimensionality, to avoid substring collisions. The

parser first checks for 1D signatures, mapping “C1D2” to `ElementType::Line` and “C1D3” to `ElementType::QuadraticEdge`. Subsequently, it prioritizes 3D roots over 2D roots to prevent false positives. For instance, the string “C3D4” (a 4-node tetrahedron) contains the substring “D4”. If the parser searched for 2D membranes (“D4”) first, it would incorrectly identify the tetrahedron as a 2D element. By checking for 3D tags (e.g., “3D4”, “3D8”, “3D20”) before general 2D tags (e.g., “S4”, “D4”), the system ensures accurate identification of the element topology.

Certain element classes are intentionally excluded from the parsing process due to topological incompatibilities. Axisymmetric elements (e.g., `CAX4`, `CAX8`) are defined in the file as 2D planar geometries but physically represent a 3D volume of revolution. These elements are omitted to prevent analysis errors. Similarly, linear cylindrical elements (e.g., `CL9`) are excluded because they lack a direct topological equivalent in the VTK standard. Mapping these elements would require either data interpolation or the omission of nodes. A critical issue with node omission is that it effectively transforms the topology into a shape indistinguishable from an existing standard element type. This creates ambiguity within the mesh quality analysis module, as it becomes impossible to determine whether a given element is a degraded cylindrical element with missing data or a valid, error-free instance of the standard type.

While Abaqus connectivity standards generally align closer to VTK than COMSOL, specific exceptions require intervention [12, 19, 20]. A detailed explanation of these node reordering algorithms is provided in Section 3.3.1.

3.2.3 Parsing of Nodal Results (.txt)

The processing of nodal simulation results is handled by the `txt_node.rs` module. This component is strictly dedicated to parsing data fields defined at mesh nodes, such as displacement vectors. Unlike the rigid schemas required for mesh topology (such as .mphtxt or .inp), this parser implements a generalized, user-configurable framework for columnar data. This design choice recognizes that result data is frequently generated by diverse sources, ranging from commercial software exports to custom in-house academic codes, or may be manually converted into plain text for interoperability. By standardizing on a flexible column-based structure, the parser serves as a universal interface, allowing users to define their own data layout without being constrained by a specific solver’s proprietary format.

The core intelligence of the parser lies in its dynamic column mapping capability, implemented in the `parse_headers` function. Rather than expecting a fixed input schema, the system scans the file metadata to discover the data layout dynamically. It employs a string analysis routine to tokenize column headers and map them to the internal `ValueType` enum. Crucially, the parser automatically reconstructs higher-order physical quantities from flat scalar data. For example, if the parser detects separate columns labeled “displacement_x”, “displacement_y”, and “displacement_z”, it aggregates these distinct fields into a single 3D vector entity within the `NodeValueData` structure. This aggregation ensures that the visualization pipeline receives semantically correct vector and tensor fields rather than a collection of disjointed scalar arrays.

The numerical data is processed via a buffered reader that consumes the file line-by-line, populating the component vectors defined during the header analysis. A key distinction in this phase is the handling of data association. This module operates on a direct nodal mapping principle, assuming that the sequence of data rows in the text file corresponds precisely to the global node ordering of the parsed mesh. Unlike the mesh parsing stage, which requires complex reordering to correct element connectivity, nodal results do not require permutation provided the node IDs are consistent. As long as the result file adheres to the node sequence

```
*HEADING
Two-dimensional overhead hoist frame
SI Units
1-axis horizontal, 2-axis vertical
*PREPRINT, ECHO=YES, MODEL=YES, HISTORY=YES
**
** Model definition
**
*NODE
101, 0., 0., 0.
102, 1., 0., 0.
103, 2., 0., 0.
104, 0.5, 0.866, 0.
105, 1.5, 0.866, 0.
*ELEMENT, TYPE=T2D2, ELSET=FRAME
11, 101,102
12, 102,103
13, 101,104
14, 102,104
15, 102,105
16, 103,105
17, 104,105
*SOLID SECTION, ELSET=FRAME, MATERIAL=STEEL
1.963E-5,
*MATERIAL, NAME=STEEL
*ELASTIC
200.E9, 0.3
**
** History data
**
*STEP, PERTURBATION
10kN central load
*STATIC
*BOUNDARY
101, ENCASTRE
103, 2
*CLOAD
102, 2, -10.E3
*NODE PRINT
U,
RF,
*EL PRINT
S,
*END STEP
```

Figure 3.3: Structure of an Abaqus .inp file showing explicit node definitions (*NODE) and element connectivity (*ELEMENT) [13].

of the source mesh, the values are correctly associated with their corresponding coordinates, allowing for immediate visualization of the nodal fields.

3.3 Node Reordering and Shape Functions for VTK-Compatible Output

A core challenge in developing a multi-format mesh parser is ensuring that the processed data is suitable for both accurate visualization and subsequent numerical analysis. This work uses the VTK Unstructured Grid format (".vtu") as the canonical output, enabling visualization in ParaView and serving as the foundation for mesh quality evaluation and finite element computations. Consequently, two critical processing steps are required: Reordering element connectivity to match VTK's canonical definitions, and implementing the corresponding shape functions and their derivatives for analysis routines.

3.3.1 Element Connectivity Reordering and Topology Matching

During parsing, the raw node ordering from source formats (e.g., Abaqus, COMSOL) can differ from the sequence prescribed by the VTK standard for topologically equivalent elements. If unaddressed, this leads to geometrically distorted or topologically invalid elements in the output. The reordering process resolves these discrepancies and is applied immediately after reading the element connectivity list.

The first step addresses index basis correction. Since VTK and internal data structures use 0-based indexing, all node IDs from 1-based formats (like Abaqus) are decremented by one upon ingestion.

Subsequent reordering falls into two primary categories based on the relationship between the source and target element definitions. The first category involves the permutation of compatible topologies, where elements share the same node count but different ordering. The second category involves topology modification via node omission, where the source element contains more nodes than the target VTK element. While interpolating nodes represents a theoretical alternative, this implementation adopts the omission strategy to maintain topological validity with minimal complexity and computational overhead. It is important to note that elements subjected to node omission are excluded from the mesh quality analysis and Gaussian quadrature optimization routines within this work. This exclusion is necessary because the data loss at the omitted node prevents accurate numerical assessment; consequently, their modification serves solely to enable correct topological reconstruction for visualization purposes.

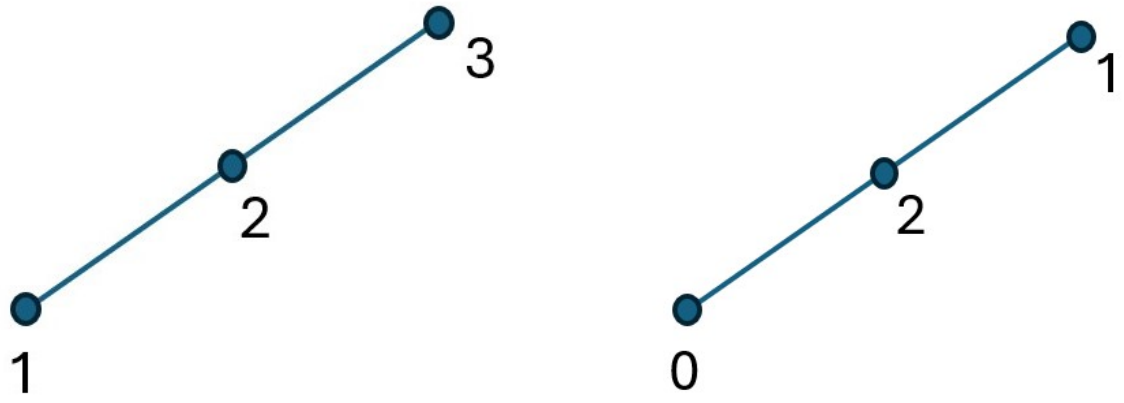
Connectivity Permutation

The first category applies to elements where the geometric definition (e.g., endpoints, corners, mid-edges) maps one-to-one between the source and VTK formats, but the enumeration sequence differs. The parser applies a hardcoded permutation vector to swap indices into the correct order.

A primary example is the Abaqus 3-node link (`C1D3`). Abaqus defines this element geometrically by listing the first end node, followed by the middle node, and finally the second end node. In contrast, the VTK standard requires a sequential ordering defined by the two endpoints followed by the middle node, as illustrated in Figure 3.4.

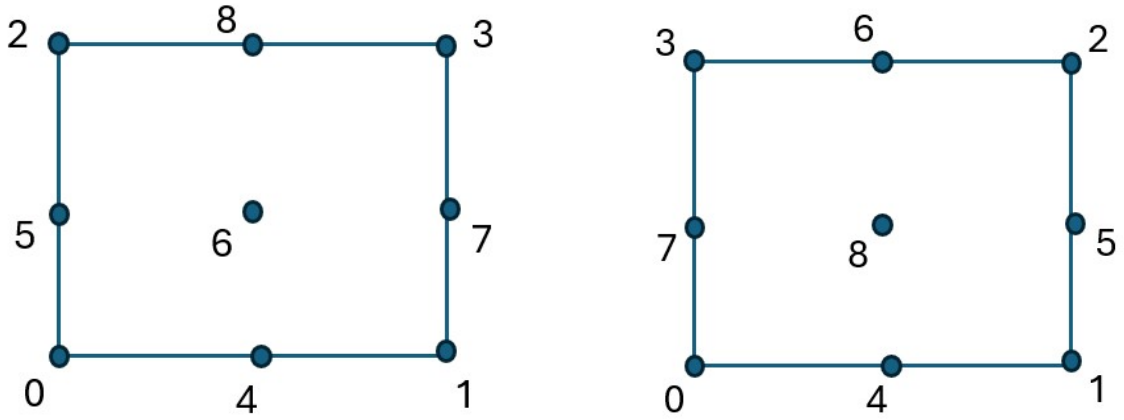
This logic extends to higher-order 2D elements, such as the COMSOL `quad2`. The element connectivity is permuted to match the specific definition of the VTK Biquadratic Quadrilateral

(Type 28), as shown in Figure 3.5. In this transformation, the corner nodes 0, 1, and 4 remain unchanged. However, the mid-side and internal nodes require swapping to align with the VTK format: nodes 2 and 3 are exchanged, nodes 5 and 7 are exchanged, and nodes 6 and 8 are exchanged.



(a) Abaqus 3-node link (C1D3). **(b)** VTK Quadratic Edge (Type 21).

Figure 3.4: Quadratic Edge Element comparison: Abaqus vs. VTK.



(a) COMSOL quad2 element. **(b)** VTK Biquadratic Quad (Type 28).

Figure 3.5: Biquadratic Quad Element comparison: COMSOL vs. VTK.

Topology Modification via Node Omission

The second category addresses high-order elements with fundamental differences in node count or placement.

In these scenarios, the parser explicitly omits specific nodes to achieve a matching topology. A notable instance is the COMSOL `pyr2`, defined with 14 nodes (5 corners, 8 mid-edges, and 1 quadrilateral face center). The standard `VTK_QUADRATIC_PYRAMID` (Type 27) supports only the 13-node configuration. To resolve this, the parser discards the face-center node (specifically index 7 in the COMSOL definition) and permutes the remaining nodes. In this reordering process, nodes 0, 1, 4, 5, and 12 remain unchanged. The connectivity is adjusted as follows: nodes 2 and 3 are swapped, and nodes 6 and 8 are swapped. Furthermore, the indices are shifted to close the gap left by the omitted node: the original node 9 becomes

7, node 10 becomes 9, node 11 becomes 10, and node 13 becomes 11. This transformation is visually depicted in Figure 3.8. This ensures the element is topologically valid within the visualization pipeline, albeit with a minor loss of geometric information at the face center. Consequently, no numerical analysis is performed on these modified quadratic pyramids due to the data loss inherent in the omission process.

Conversely, elements that lack a direct VTK equivalent and cannot be mapped via simple omission, such as Abaqus linear cylindrical elements (e.g., `CL9`), are currently excluded to avoid significant data degradation or ambiguity in the analysis.

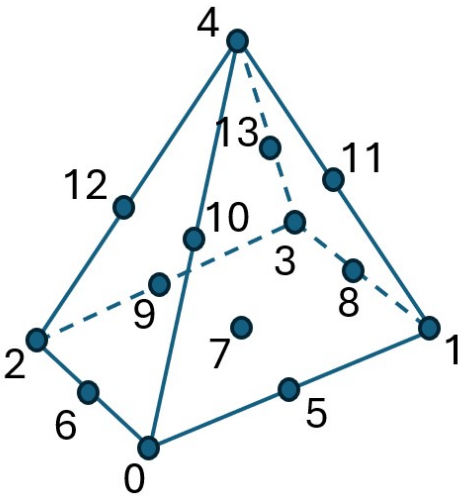


Figure 3.6: COMSOL pyr2 element.

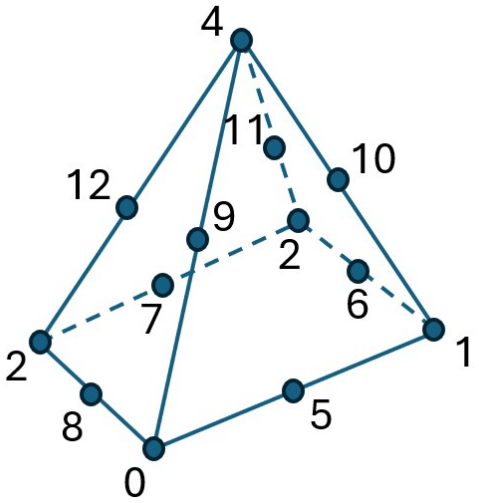


Figure 3.7: VTK Quadratic Pyramid (Type 27).

Figure 3.8: Quadratic Pyramid Element comparison: COMSOL vs. VTK.

3.3.2 Implementation of Shape Functions

Mesh quality analysis and Gaussian quadrature optimization require precise definitions of shape functions and their derivatives. To ensure consistency with the visualization output, this implementation hardcodes shape functions and their derivatives based directly on the definitions provided in the VTK source code [21].

Monomial Basis and Coefficient Mapping

To optimize performance and avoid the overhead of runtime symbolic parsing, polynomial operations are implemented using a monomial basis. Polynomials are transformed into coefficient vectors ordered according to a graded lexicographic sequence. To maintain a standardized data structure across the codebase, functions are padded to a full 3-dimensional basis vector $\mathbf{P}$, as explained in Section 2.4.5.

Considering a monomial basis in graded lexicographic order, the full basis vector used for this mapping is:

$$\mathbf{P} = [1, \xi, \eta, \psi, \xi^2, \xi\eta, \xi\psi, \eta^2, \eta\psi, \psi^2, \xi^3, \xi^2\eta, \xi^2\psi, \xi\eta^2, \xi\eta\psi, \xi\psi^2, \eta^3, \eta^2\psi, \eta\psi^2, \psi^3, \\ \xi^4, \xi^3\eta, \xi^3\psi, \xi^2\eta^2, \xi^2\eta\psi, \xi^2\psi^2, \xi\eta^3, \xi\eta^2\psi, \xi\eta\psi^2, \xi\psi^3, \eta^4, \eta^3\psi, \eta^2\psi^2, \eta\psi^3, \psi^4]. \quad (3.1)$$

As an example, utilizing the shape functions and derivatives defined in the VTK source code [21], the biquadratic shape function N_0 is defined as:

$$N_0 = 4\eta^2\xi^2 - 6\eta^2\xi + 2\eta^2 - 6\eta\xi^2 + 9\eta\xi - 3\eta + 2\xi^2 - 3\xi + 1. \tag{3.2}$$

This polynomial is encoded as the following coefficient vector:

$$\begin{aligned}\mathbf{c}_{N_0} = [&1.0, -3.0, -3.0, 0.0, 2.0, 9.0, 0.0, 2.0, 0.0, 0.0, 0.0, -6.0, 0.0, -6.0, 0.0, 0.0,\\ &0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0].\end{aligned} \tag{3.3}$$

The derivative with respect to ξ is calculated as:

$$\frac{\partial N_0}{\partial \xi} = 9\eta + 4\xi - 12\eta\xi + 8\eta^2\xi - 6\eta^2 - 3, \tag{3.4}$$

which is encoded as:

$$\mathbf{c}_{\frac{\partial N_0}{\partial \xi}} = [-3.0, 4.0, 9.0, 0.0, 0.0, -12.0, 0.0, -6.0, 0.0, 0.0, 0.0, 0.0, 0.0, 8.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]. \tag{3.5}$$

The derivative with respect to η is:

$$\frac{\partial N_0}{\partial \eta} = 4\eta + 9\xi - 12\eta\xi + 8\eta\xi^2 - 6\xi^2 - 3, \tag{3.6}$$

encoded as:

$$\mathbf{c}_{\frac{\partial N_0}{\partial \eta}} = [-3.0, 9.0, 4.0, 0.0, -6.0, -12.0, 0.0, 0.0, 0.0, 0.0, 0.0, 8.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]. \tag{3.7}$$

The derivative with respect to ψ is zero for this 2D element:

$$\frac{\partial N_0}{\partial \psi} = 0, \tag{3.8}$$

encoded as a zero-padded vector:

$$\mathbf{c}_{\frac{\partial N_0}{\partial \psi}} = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]. \tag{3.9}$$

3.4 Mesh Quality Analysis Module

Following the parsing and reordering of the mesh topology, the software executes a comprehensive geometric diagnostic performed on an element-by-element basis. This functionality is encapsulated within the `GeometricAnalysis` struct. This module implements metrics derived from the determinant of the Jacobian matrix, alongside an algebraic quality assessment framework based on the factorization of the Jacobian matrix. This approach enables the mathematical isolation of specific geometric distortions, such as skewness, length ratio, and orientation, for every individual cell, thereby providing a rigorous quality certification of the entire discretized domain.

3.4.1 Analysis Architecture and Data Flow

The high-level orchestration is handled by the function `analyse_mesh_quality`. This function acts as both an iterator and an aggregator. It accepts an immutable reference to `MeshData` and iterates through the defined element types. A specific filter is applied at this stage to skip certain element classes. `Vertex` elements are bypassed as they are 0-dimensional entities possessing no intrinsic geometric quality such as volume or length ratio. Furthermore, `QuadraticPyramid` elements are excluded due to the specific node omission required during the parsing phase, as detailed in Section 3.3.1. Since the face-center node is removed to adhere to VTK topology, any geometric defects associated with that specific node, such as severe depression or elevation relative to the face, are lost. Analyzing the remaining nodes would result in a misleading quality report that fails to detect actual distortions present in the original mesh. For all other parsed elements, the function delegates the computational load to the core logic encapsulated in `calculate_element_quality`.

A critical architectural detail is the efficient data interchange between topological and geometric representations. The `Element` structs stored in memory contain only topological references (node IDs). To perform geometric analysis, the `calculate_element_quality` function must first reconstruct the physical element. It invokes `get_element_nodes`, which acts as a lookup service, retrieving the distinct spatial coordinates (x, y, z) for the element's nodes from the global node vector.

3.4.2 Jacobian Polynomial Construction

Rather than evaluating the Jacobian matrix at arbitrary points immediately, the software employs a polynomial approach. Using the `calculate_jacobian_monomial` function (imported from the `gaussian_quadrature.rs` module), the software constructs the Jacobian matrix symbolically as a set of coefficient vectors for monomial polynomials.

Once the Jacobian matrix is established in this monomial polynomial format, the function `find_min_max_jacobian_determinant` evaluates the determinant at specific, critical locations within the element. The software retrieves a set of pre-defined evaluation points (typically corners and centroids) specific to the element type via `get_evaluation_points`. It calculates the determinant at each point to identify the "worst-case" location within the element domain.

The worst-case location is identified by calculating the minimum value of $\det(\mathbf{J})$ for square Jacobian matrices, or $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$ for non-square cases (e.g., 2D elements in 3D space). Crucially, if any evaluation point yields a non-positive determinant, indicating an inverted element or a singularity, the element is immediately flagged as invalid. In such cases, all quality metrics are reset to zero, and further algebraic factorization is aborted to conserve computational resources.

If the element is topologically valid, the software proceeds to compute specific quality metrics. The `find_min_max_jacobian_determinant` function returns the coordinates of the worst-case evaluation point, alongside the minimum and maximum values of the determinant. These extrema are subsequently utilized to compute the Jacobian Ratio metric.

3.4.3 Algebraic Matrix Factorization

The evaluation point identified as the worst-case location is utilized to compute the numeric Jacobian matrix. At this stage, the software implements a conditional logic to establish the basis for Knupp's algebraic metrics [22]. If the Jacobian matrix is square, the software utilizes

the evaluated Jacobian directly as the `metric_jacobian`. Conversely, when the Jacobian is non-square, the system constructs a square metric Jacobian by multiplying the matrix by its transpose ($\mathcal{M} = \mathbf{J}\mathbf{J}^T$).

Once the valid square `metric_jacobian` is established, it is decomposed into three constituent matrices, specifically R (Orientation), Q (Skew), and D (Length Ratio). The function `jacobian_factorization` performs this decomposition by adapting the algorithm based on the dimensionality of the matrix. Following factorization, these matrices, along with the original metric Jacobian, are inverted to facilitate the calculation of the specific norm-based quality metrics.

3.4.4 Quality Metrics

Once the factorization matrices (R, Q, D) and their inverses are computed via `inverse_matrix`, the specific metrics are derived using Frobenius norms and matrix traces. The Shape Metric, calculated via `calculate_shape_metric`, divides the element dimension by the product of the Frobenius norms of the metric Jacobian and its inverse. This value normalizes general distortion, yielding 1.0 for a perfect element and approaching 0.0 as the geometry degrades. Similarly, angular distortion is evaluated using the Skewness Metric. Calculated via `calculate_skewness_metric` using the Skew matrix (Q), this index yields 1.0 for an orthogonal element and decreases toward 0.0 as the element shears. To address aspect ratio disparities, the Length Ratio Metric utilizes the Length Ratio matrix (D). This metric quantifies the uniformity of edge lengths, assigning a value of 1.0 to equilateral elements and lower values to those that are stretched or compressed.

Complementary to these shape-based factors, the software calculates metrics for alignment and scale. The Orientation Metric is derived from the trace of the Orientation matrix (R) via `calculate_orientation_metric`. It quantifies the element's alignment with the global coordinate system, where a value of 1.0 indicates perfect alignment. Finally, the Volume Metric corresponds directly to the minimum determinant value ($\min(\det(\mathbf{J}))$) identified during the evaluation phase, representing the physical size of the element at its most compressed integration point.

3.4.5 Utility Functions

Several lower-level utility functions support these high-level algorithms. `calculate_determinant` handles algebraic determinant calculation for 1×1, 2×2, and 3×3 matrices, including logic for handling non-square Jacobian matrices. Similarly, `inverse_matrix` computes the matrix inverse using the adjoint method ($A^{-1} = \text{adj}(A)/\det(A)$).

3.5 Gaussian Quadrature and Numerical Optimization

The final analytical component of the software is an optimization module designed to determine the optimal number of Gaussian points for finite element matrices, adhering to a user-defined tolerance for the estimated error bound. Although a theoretical minimum number of Gaussian points exists for the exact integration of polynomial fields, utilizing this maximum count is not always computationally efficient or necessary. The objective of this module is to

determine if a reduced integration order can satisfy the required accuracy, thereby checking if computational costs can be spared without compromising simulation fidelity.

A critical design choice in this module is the reliance on an analytical error bound rather than the calculation of the exact error. Computing the exact error would typically require performing the integration at a significantly higher order for comparison, a process that would consume excessive computational resources and negate the performance benefits of the optimization. Consequently, an upper bound estimate is employed to prioritize runtime efficiency. This implemented bound is intentionally conservative, rather than tight, to strictly ensure that the simulation remains on the safe side regarding accuracy limits. However, the system's modular architecture is designed to be extensible, allowing for the future implementation of tighter error bounds should a user require more aggressive optimization for specific applications.

This module, encapsulated within the `GaussianQuadrature` struct, implements a parallelized and iterative error minimization algorithm. It leverages the `rayon` crate for data parallelism and employs efficient monomial polynomial arithmetic to evaluate the convergence of the integration error dynamically.

3.5.1 Architectural Flow and Parallelism

The entry point for the module is the public function `find_optimal_gauss_points_number_mesh`. This function serves as the high-level coordinator, accepting the parsed mesh geometry, the target integration type (Mass or Stiffness), the user-defined material properties, and the error tolerance. In this implementation, parallelism is utilized to drastically reduce computational time. The function first flattens the hierarchical mesh structure into a linear collection of processable element units, then utilizes a parallel iterator to distribute the workload across available CPU cores. Each element is processed independently to find its local optimal integration order. This design is particularly effective for large-scale meshes where sequential processing would create a significant bottleneck. The module includes robust error handling and fallback mechanisms. If an element fails analysis, the system returns zero for the theoretical and optimal number of Gaussian points rather than halting the entire analysis pipeline.

3.5.2 Optimization Logic

The core optimization logic is encapsulated within `find_optimal_gauss_points`, which orchestrates the decision-making process for selecting the integration order. A crucial preliminary step involves determining whether an element is eligible for adaptive optimization. The current error bound derivation is implemented specifically for tensor-product element families (Lines, Quadrilaterals, Hexahedra). For unsupported topologies (Triangles, Tetrahedra, Wedges), the system defaults to the safe theoretical limit to ensure numerical stability.

For the supported element types, the system first calculates the theoretical limit for exact integration. This serves as a deterministic upper bound derived from the polynomial degree of the integrand matrix. The software inspects the symbolic representation of the integrand to identify the maximum polynomial degree, denoted as d_{poly}, among all matrix components. The theoretical minimum number of Gaussian points required for exact integration is then derived:

$$n_{GP,\text{theoretical}} = \text{ceil}\left(\frac{d_{poly}+1}{2}\right). \tag{3.10}$$

This value establishes the upper boundary of the search space. The algorithm then enters an iterative loop, starting from a single Gaussian point ($n = 1$) and incrementing up to this theoretical limit. In each iteration, the function `calculate_error` is invoked to estimate the upper bound of the integration error. The error metric is computed using a specialized Gauss-Legendre error estimator. If the estimated error falls below the user-specified tolerance, the current order n is flagged as the `optimal_number`, and the loop terminates early. This early exit strategy ensures that the system identifies the most efficient integration scheme that strictly satisfies the accuracy requirement.

3.5.3 Symbolic Integrand Construction

A distinguishing feature of this implementation is the exclusive use of symbolic polynomial arithmetic in a monomial basis. Rather than approximating the integrand numerically at every integration point, the software constructs the exact polynomial representation of the integrand matrix via the `MonomialPolynomial` struct. This data structure utilizes a Graded Lexicographic Order to flatten multivariate polynomials ($x^i y^j z^k$) into linear coefficient vectors. Crucially, this approach significantly enhances computational performance by circumventing the high overhead associated with direct symbolic manipulation. By reducing complex algebraic operations to efficient and vectorized arithmetic on coefficient arrays, the software achieves the precision of analytical derivation without the performance penalty typically associated with symbolic engines.

The construction of the Jacobian matrix $\mathbf{J}$ relies on the partial derivatives of the shape functions with respect to the parametric coordinates. To further optimize performance, these derivatives are not differentiated dynamically at runtime but are instead retrieved from a library of hard-coded and pre-computed derivative polynomials. The software then implements logic to handle both standard and embedded elements during determinant calculation. For elements where the spatial and topological dimensions match, the determinant $\det(\mathbf{J})$ is calculated using the standard algebraic formula. Conversely, for structural elements embedded in higher dimensions, such as a 2D shell in 3D space, the Jacobian is non-square. In these instances, the software computes the metric determinant, defined as $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$. Since the exact square root of a polynomial is rarely a finite polynomial, the system approximates this term using a binomial series expansion, ensuring that the resulting differential area or length element remains compatible with the monomial arithmetic engine while correctly scaling the geometry.

Subsequent operations necessitate the calculation of the inverse Jacobian, particularly for the Stiffness matrix where derivatives must be mapped to the physical frame. Since standard numerical inversion techniques are computationally inefficient for symbolic polynomial matrices, the software employs the adjoint method. For square matrices, the inverse is formulated as $\mathbf{J}^{-1} = \text{adj}(\mathbf{J})/\det(\mathbf{J})$. However, for non-square Jacobian matrices arising from embedded elements, standard determinants and inverses are undefined. Consequently, the system implements the Moore-Penrose pseudoinverse combined with the adjoint method, governed by the formulation $\mathbf{J}^+ = \mathbf{J}^T(\text{adj}(\mathbf{J}\mathbf{J}^T)/\det(\mathbf{J}\mathbf{J}^T))$. As the exact inverse of the resulting polynomial determinant is a rational function, which is computationally intractable for the current monomial engine, the software utilizes a Taylor Series approximation. The reciprocal determinant term is expanded as a polynomial series, enabling the system to approximate the inverse Jacobian as a polynomial matrix while preserving compatibility with the arithmetic engine.

Concurrently, the module manages user-defined spatially varying material properties by transforming them from the physical domain (x, y, z) to the parametric domain (ξ, η, ψ). This transformation is executed by substituting the element's coordinate map directly into

the material property definitions, ensuring that functionally graded materials are accurately represented. The software constructs this coordinate map by expressing the physical coordinates as dependent variables of the parametric coordinates utilizing the element shape functions, formulated as $x(\xi, \eta, \psi) = \sum_{i=0}^{n-1} N_i^{\text{iso}} X_i$, with similar expansions for y and z depending on the dimension. These symbolic expressions then replace the spatial variables x, y, and z in the material polynomials, effectively mapping the material properties into the integration domain.

The final integrand construction differentiates between Mass and Stiffness matrices, applying specific optimizations for each case. For Mass matrix integration, the user-defined density polynomial ρ is first evaluated. To minimize computational cost, the software checks if the density is constant; if not, it transforms the scalar field from physical to isoparametric coordinates using the coordinate map. The integrand is then computed as the outer product of the shape functions ($\mathbf{N}^T\mathbf{N}$), scaled by the density and the Jacobian determinant. Since the Mass matrix is inherently symmetric, the software computes only the upper triangular part and mirrors the results, reducing polynomial operations by half.

The Stiffness matrix construction is significantly more complex and begins with the assembly of the strain-displacement matrix $\mathbf{B}^{\text{iso}}$. This requires transforming the parametric shape function derivatives ($\mathrm{d}\mathbf{N}/\mathrm{d}\xi$) into physical derivatives ($\mathrm{d}\mathbf{N}/\mathrm{d}\mathbf{x}$) by multiplying them with the previously calculated inverse Jacobian $\mathbf{J}^{-1}$. Concurrently, the elasticity tensor $\mathbf{C}$ is processed. Similar to the density field, the software detects if the material properties are constant to bypass unnecessary transformations. Crucially, the system also checks the symmetry of the physical elasticity matrix. If $\mathbf{C}^{\text{phys}}$ is symmetric, the transformed isoparametric elasticity matrix remains symmetric, guaranteeing that the resulting Stiffness matrix $\mathbf{K}$ is also symmetric. In this case, the software optimizes the triple matrix product $\mathbf{B}^{\text{iso}\,T}\mathbf{C}^{\text{iso}}\mathbf{B}^{\text{iso}}$ by calculating only the upper triangular entries. Conversely, if the material exhibits non-symmetric behavior, the full matrix multiplication is performed to ensure numerical correctness.

3.5.4 Gauss-Legendre Error Estimation

The error estimation logic, implemented in `gauss_legendre_error`, avoids the computational expense of performing a full numerical integration benchmark. Instead, it analyzes the polynomial coefficients of the integrand directly. The system specifically utilizes the Gauss-Legendre error bound, as this rule is the default for the supported tensor-product elements. To guarantee high performance during the iterative optimization loop, the computationally expensive factorials required for the error terms are precomputed and stored in a static `Lazy` array. This prevents redundant multiplications during the evaluation of high-order polynomials.

For a polynomial integrand, the n-point Gauss-Legendre quadrature integrates all terms up to degree $2n-1$ exactly. The error is consequently driven by terms of degree $2n$ and higher. The software implements specific error estimators for 1D, 2D, and 3D cases. The base error constant C_n is defined as:

$$C_n = \frac{n!^4}{(2n+1)((2n)!)^3}. \tag{3.11}$$

For a 1D polynomial $P(\xi) = \sum a_k \xi^k$, the error is estimated by summing the weighted magnitudes of the truncated coefficients:

$$E_{1D} = C_n \sum_{k=2n}^{d_{poly}} |a_k| \frac{k!}{(k-2n)!}. \tag{3.12}$$

For a 2D polynomial $P(\xi,\eta) = \sum\sum a_{ij}\xi^i\eta^j$, the implementation calculates a conservative upper bound by summing the error contributions along each independent axis. This approach is not a tight bound but ensures numerical safety by capturing the worst-case truncation in both directions:

$$E_{2D} = C_n \left(\sum_{i=2n}^{d_x} \sum_{j=0}^{d_y} |a_{ij}| \frac{i!}{(i-2n)!} + \sum_{i=0}^{d_x} \sum_{j=2n}^{d_y} |a_{ij}| \frac{j!}{(j-2n)!} \right). \tag{3.13}$$

Similarly, for a 3D polynomial, the error is computed as the additive sum of the truncation errors along the ξ, η, and ψ axes:

$$E_{3D} = C_n \left(\sum |a_{ijk}| \frac{i!}{(i-2n)!} + \sum |a_{ijk}| \frac{j!}{(j-2n)!} + \sum |a_{ijk}| \frac{k!}{(k-2n)!} \right). \tag{3.14}$$

The module's architecture is designed to be highly modular. While the current implementation focuses on Gauss-Legendre quadrature for tensor-product elements, the structure allows for the seamless integration of alternative error formulas in the future. This extensibility facilitates support for other quadrature rules (e.g., Gauss-Lobatto) or distinct element topologies such as Triangles or Pyramids.

3.6 Writer Implementation for Visualization in ParaView

The final stage of the pre-processing pipeline involves serializing the parsed mesh data, simulation results, and analysis reports into a format compatible with scientific visualization software. This implementation utilizes the VTK Unstructured Grid (.vtu) XML format, which is natively supported by ParaView. The writer module is encapsulated within the `VTUWriter` struct and leverages the `vtkio` crate to construct the strict XML schema required by the VTK standard. The implementation is divided into three primary functionalities: basic geometry export, nodal field value export, and element quality reporting.

3.6.1 Geometry Serialization and Coordinate Padding

The fundamental task of the writer is to translate the internal `MeshData` structure into VTK geometry. This begins with the serialization of nodal coordinates. During development, it was observed that the visualization software frequently failed to render 1D and 2D meshes correctly when provided with raw, lower-dimensional coordinate data. To ensure a universal and stable implementation capable of handling any mesh dimension, the writer enforces a strict three-component coordinate format. The code iterates through the node list and flattens the coordinates into a single contiguous vector. During this process, the iterator utilizes a chaining mechanism to append infinite zeros to the coordinate sequence, taking exactly three components for every node. This ensures that a 2D node (x, y) is consistently serialized as $(x, y, 0.0)$, preventing visualization errors and guaranteeing stability across all supported element types.

3.6.2 Topology and Connectivity Assembly

Following coordinate definition, the writer reconstructs the mesh topology using the standard VTK connectivity format. This requires the generation of three synchronized data structures

that collectively define the mesh structure. The first is the "Connectivity Array", a linear sequence of node indices that defines the vertices of every element flattened into a single list. To navigate this linear list, the writer constructs the "Offsets Array", which stores a cumulative count of nodes, effectively marking the ending index of each element within the connectivity array. Finally, the "Cell Types Array" is populated with integer values specifying the geometric classification (e.g., `VTK_QUAD`, `VTK_HEXAHEDRON`) for each corresponding element. The code iterates through the `element_type_info` groups to efficiently map the internal element types to these VTK integer constants. This grouping strategy ensures that mixed meshes, such as those combining triangles and quadrilaterals, are handled correctly without the need for expensive manual sorting.

3.6.3 Nodal Attribute Export and Vector Padding

The function `write_vtu_with_field_values` is responsible for mapping simulation results, such as displacement or velocity, onto the mesh nodes. A significant challenge here is properly typing the data for visualization filters (e.g., Warp by Vector). The writer implements a helper function, `is_vector_quantity`, to distinguish between scalar fields (like Temperature or Pressure) and vector fields. Similar to the geometry export, this section implements strict attribute padding to ensure 3D compatibility. For 1D vectors, the scalar value u is padded to $(u, 0, 0)$. For 2D vectors, the components (u, v) are expanded to $(u, v, 0)$. Furthermore, for 2D tensors (e.g., plane stress components xx, yy, xy, yx), the writer reconstructs the full 9-component 3D tensor by placing the 2D components into their correct indices and padding the z-direction terms with zeros. This rigorous formatting allows ParaView to automatically recognize and visualize directionality and tensor characteristics without user intervention.

3.6.4 Cell Data and Geometric Analysis Reporting

The final component, `write_vtu_with_geometric_analysis_and_gaussian_points_number`, handles data associated with elements (Cells) rather than nodes (Points). This function is critical for visualizing the results of the mesh quality assessment and the Gaussian quadrature optimization. The writer iterates through the `MeshQualityReport` and populates independent scalar arrays for each metric, including Volume, Shape, Skewness, and Jacobian Ratio. Additionally, the optimization results are exported as cell attributes named "Optimal_Gaussian_Points" and "Theoretical_Gaussian_Points". A derived field, "Gaussian_Point_Difference", is also calculated and written to the file. This allows the user to visually identify regions where the adaptive algorithm successfully reduced the integration order compared to the theoretical limit, providing immediate visual feedback on the efficiency gains across the mesh domain.

Chapter 4

Validation and Results

This chapter presents the validation of the software implementation. First, the parsing capabilities for COMSOL Multiphysics mesh files (`.mphtxt`), Abaqus input files (`.inp`), and nodal result files (`.txt`) are demonstrated. The resulting topologies are visualized in ParaView to verify connectivity and element identification. Second, the implemented mesh quality metrics are validated against reference geometries to ensure numerical accuracy. Third, the Gauss-Legendre quadrature optimization is evaluated through unit tests to confirm the efficiency of the adaptive integration scheme. Finally, the approximation errors introduced by polynomial series expansions for non-linear mappings are quantified.

4.1 Validation of Mesh Parsing

The primary prerequisite for any analysis is the accurate interpretation of input data. This section validates the parser's capability to ingest mixed-dimensional topologies from different source formats.

To ensure robustness, a comprehensive unit test suite was developed for both COMSOL and Abaqus formats. The test cases are categorized by the spatial dimension of the mesh domain (1D, 2D, and 3D). It is noted that this categorization refers to the dimensionality of the domain itself, not the exclusive dimensionality of the elements within it. For example, the "3D Mesh Domain" test case validates the software's ability to simultaneously handle 1D edges, 2D surfaces, and 3D volumes within a single file.

Furthermore, a specific technical challenge regarding the visualization of lower-dimensional elements in ParaView was addressed. The writer module implements coordinate padding to normalize all nodal data to three components (x, y, z), ensuring that 1D and 2D elements render correctly alongside volumetric domains.

4.1.1 Validation of COMSOL (.mphtxt) File Parsing

The objective of this test suite was to validate the parsing of all standard element types available in the COMSOL library within a single `.mphtxt` file. The following results demonstrate the successful import of vertex, edge, triangular, quadrilateral, tetrahedral, pyramidal, prismatic, and hexahedral elements.

Parsing of the 1D Mesh Domain

Figure 4.1 illustrates the successful parsing of the 1D mesh domain. The parser correctly identifies the connectivity and node count for vertex and edge elements. Specifically, the

element labeled as 1 corresponds to the COMSOL element type `vtx` and represents a single-node vertex. The element labeled as 2 is identified as `edg` and represents a 2-node linear edge element. The element labeled as 3 is the quadratic counterpart `edg2` and features a midpoint node.

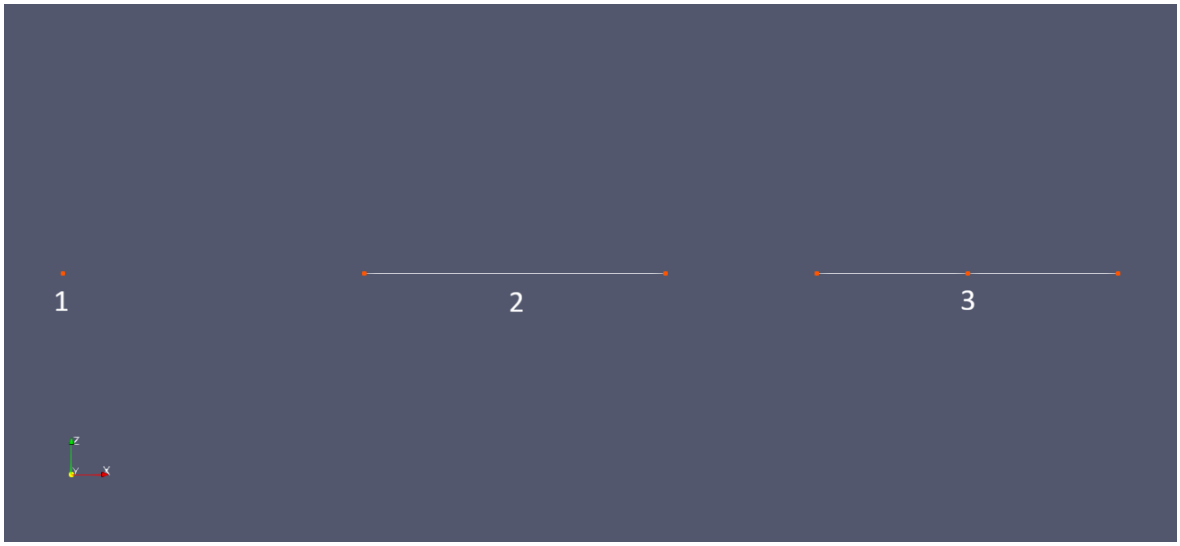


Figure 4.1: Visualization of the one-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. Labels 1, 2, and 3 correspond to vertex, linear edge, and quadratic edge elements.

Parsing of the 2D Mesh Domain

The results for the 2D mesh domain are presented in Figure 4.2. This domain aggregates the previously validated 1D elements (1–3) alongside planar geometries.
The 2D elements are identified as follows: The element labeled 4 is the 3-node linear triangular element `tri`. The element labeled 5 is the 6-node quadratic triangular element `tri2`. Element 6 is the 4-node linear quadrilateral element `quad`. Finally, the element labeled 7 corresponds to the 9-node quadratic quadrilateral element `quad2`.

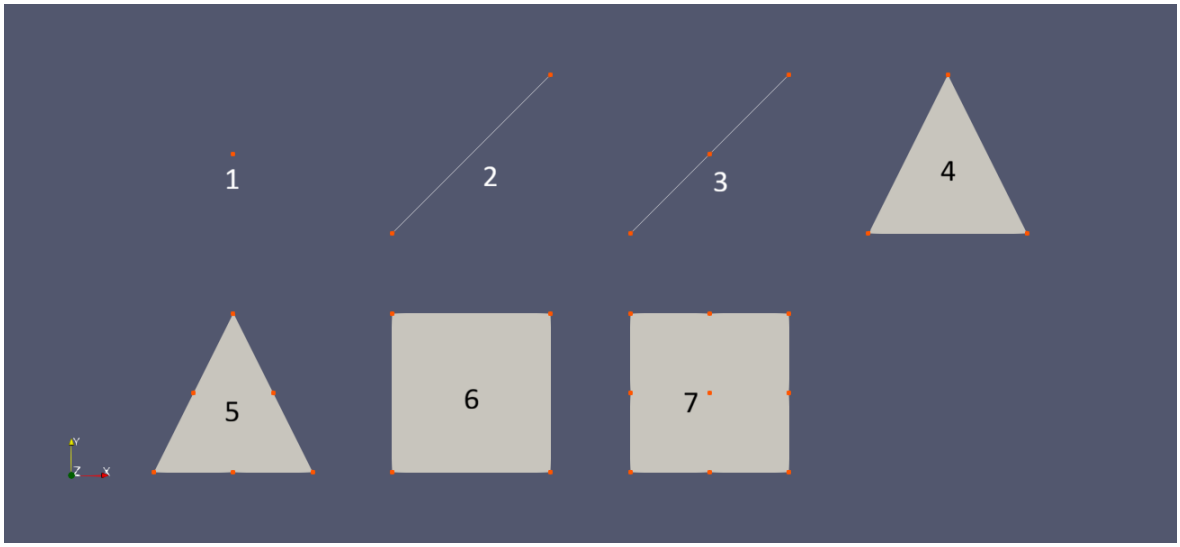


Figure 4.2: Visualization of the two-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. The domain comprises the 1D elements (1–3) alongside the 2D triangular and quadrilateral elements (4–7).

Parsing of the 3D Mesh Domain

Figure 4.3 demonstrates the parsing results for the complete 3D mesh domain, validating the integration of volumetric elements with lower-dimensional entities.
The domain correctly parses the lower-dimensional entities (1–7). The 3D volumetric elements are identified as follows: Element 8 is `tet` and represents a 4-node linear tetrahedron. Element 9 corresponds to the 10-node quadratic tetrahedron `tet2`. Element 10 is the 5-node linear pyramid `pyr`. Element 11 represents the 14-node quadratic pyramid `pyr2`.
Prismatic and hexahedral geometries are also validated. Element 12 is the 6-node linear prism `prism`. Element 13 corresponds to the 18-node quadratic prism `prism2`. Finally, element 14 is the 8-node linear hexahedron `hex`, and element 15 represents the 27-node quadratic hexahedron `hex2`.

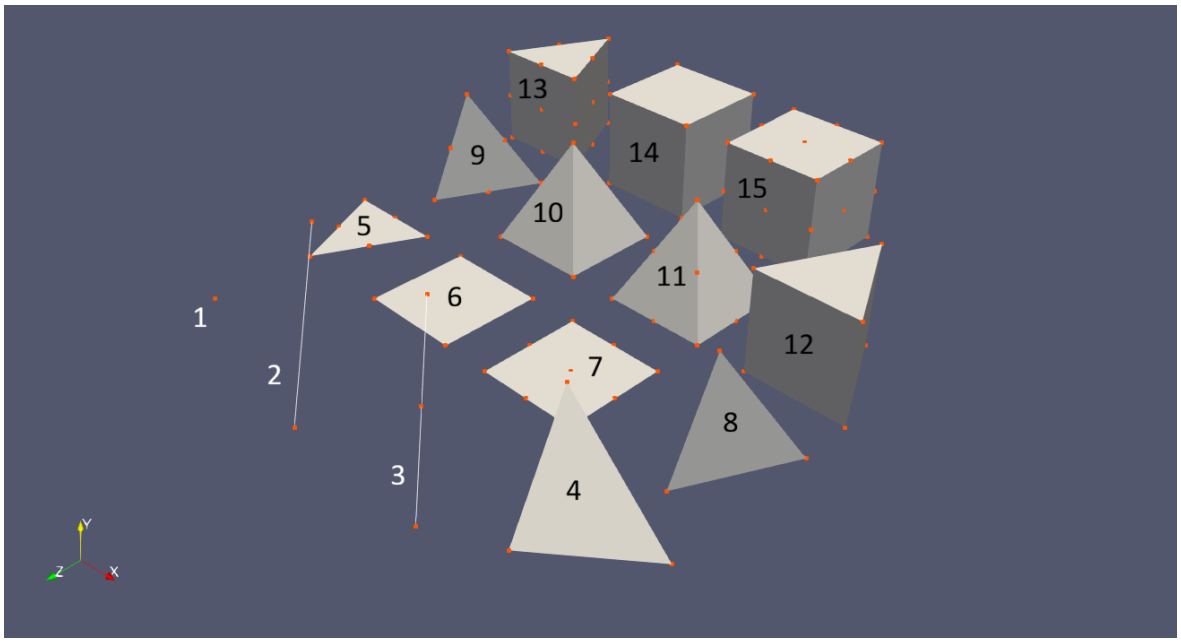


Figure 4.3: Visualization of the three-dimensional mesh topology parsed from the COMSOL `.mphtxt` file. The scene integrates 1D lines and 2D surfaces alongside the 3D volumetric elements labeled 8 to 15.

4.1.2 Validation of Abaqus (.inp) File Parsing

Similarly, the Abaqus parser was validated using a suite containing all standard element types from the Abaqus library, with the exception of linear cylindrical and axis-symmetric elements. These tests confirm the correct interpretation of the explicit node and element indexing unique to the `.inp` format.

Parsing of the 1D Mesh Domain

Figure 4.4 illustrates the successful parsing of the 1D mesh domain. The parser correctly identifies the connectivity and node count for both linear and quadratic link elements. Specifically, the element labeled as 1 corresponds to the Abaqus element type `AC1D2`, which is a 2-node link element. The element labeled as 2 is identified as `DC1D3E`, representing a 3-node link element.

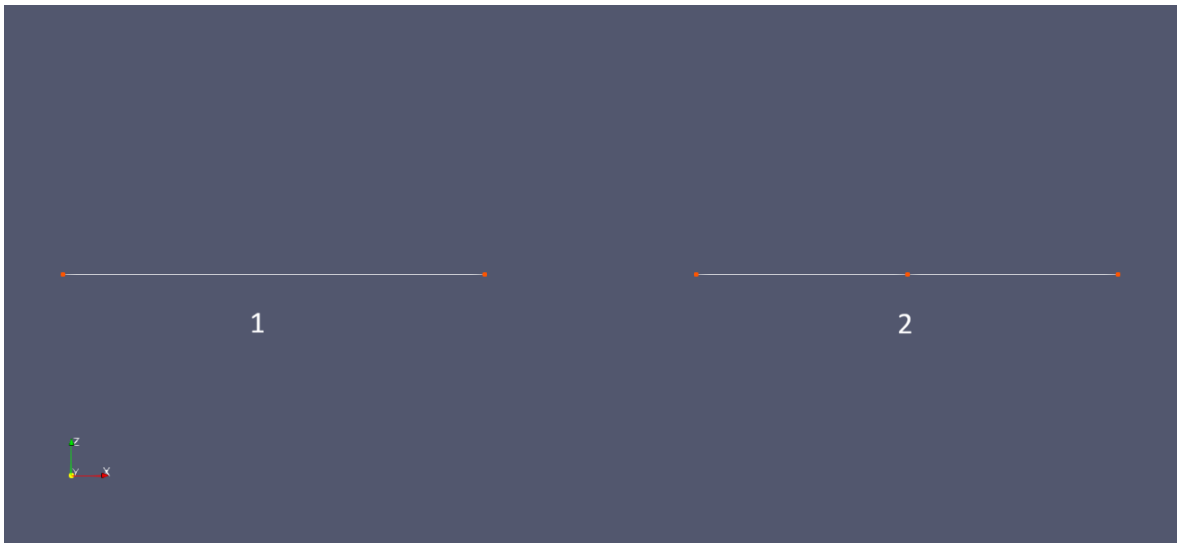


Figure 4.4: Visualization of the one-dimensional mesh topology parsed from the Abaqus `.inp` file. Labels 1 and 2 correspond to linear and quadratic link elements, respectively.

Parsing of the 2D Mesh Domain

The validation of the 2D mesh domain is presented in Figure 4.5. This test case confirms the parser's ability to handle mixed-dimensional topologies that include both 1D and 2D elements.

As seen in the figure, the domain includes the previously validated 1D elements: element 1 (`AC1D2`) and element 2 (`DC1D3E`). The 2D elements are identified as follows: The element displayed as 3 is the 3-node linear triangular element `DS3`. The element labeled 4 is the 6-node quadratic triangular element `CPEG6MH`. Moving to quadrilateral geometries, the element displayed as 5 is the 4-node bilinear quadrilateral element `M3D4`. Finally, the element labeled 6 corresponds to the 8-node biquadratic quadrilateral element `CPE8P`.

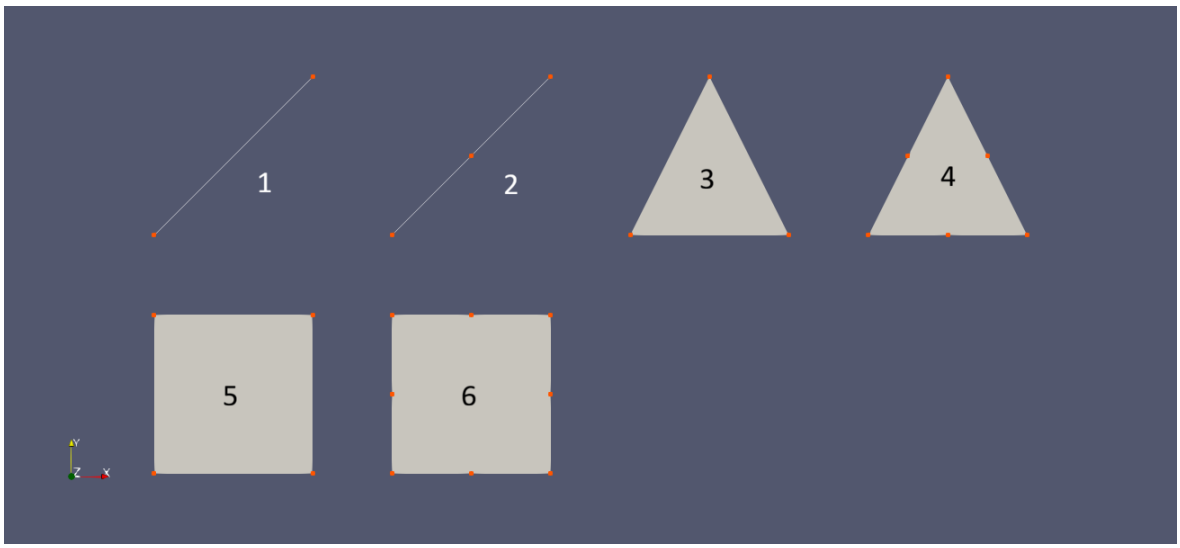


Figure 4.5: Visualization of the two-dimensional mesh topology parsed from the Abaqus `.inp` file. The domain comprises the 1D link elements (1–2) alongside the 2D triangular and quadrilateral elements (3–6).

Parsing of the 3D Mesh Domain

Figure 4.6 demonstrates the parsing results for the 3D mesh domain. This represents the most comprehensive test case, validating the simultaneous presence of 1D, 2D, and 3D elements within a single file.

The domain correctly parses the lower-dimensional entities: the 1D elements 1 and 2, and the 2D elements 3 through 6.

The 3D elements are identified as follows: Element 7 is `EMC3D4`, a 4-node linear tetrahedron, while element 8 is its quadratic counterpart, `Q3D10M`. Element 9 represents a 5-node linear pyramid, type `C3D5H`. Prismatic and hexahedral elements include element 10, the 6-node linear prism `C3D6PHT`; element 11, the 15-node quadratic prism `C3D15`; element 12, the 8-node linear brick `CIN3D8`; and element 13, the 20-node quadratic brick `Q3D20R`. Finally, the test included specialized quadratic cylindrical elements: element 14, type `CCL18`, and element 15, type `CCL24RH`.

It is noted that while this validation suite covers a broad spectrum of element formulations, axisymmetric elements and standard linear cylinder elements are explicitly excluded from the parsing logic.

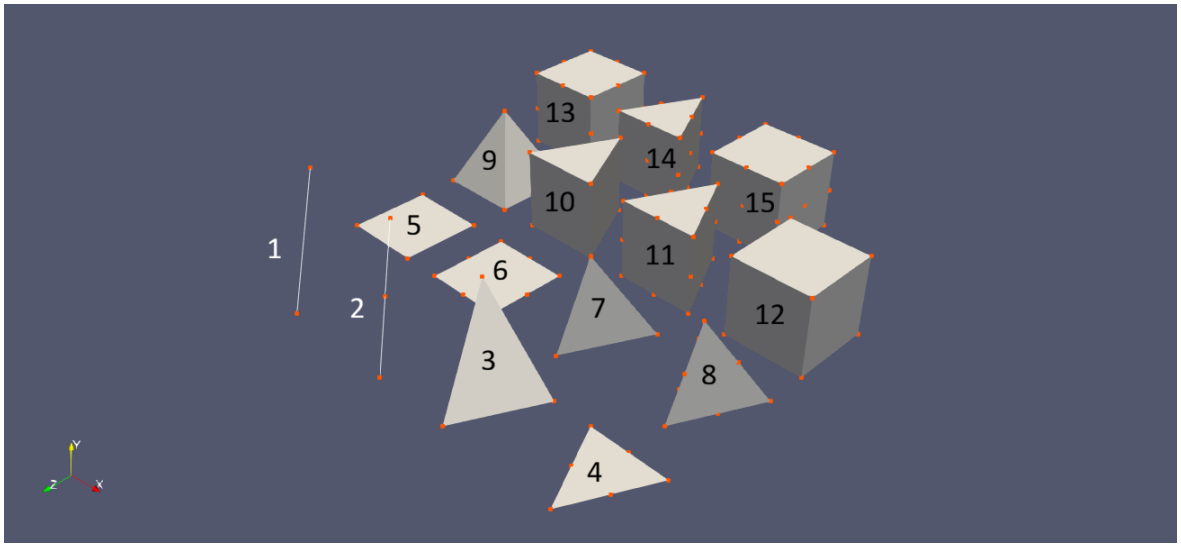


Figure 4.6: Visualization of the three-dimensional mesh topology parsed from the Abaqus `.inp` file. The scene integrates 1D links (1–2) and 2D surfaces (3–6) alongside the 3D volumetric elements (7–15).

4.1.3 Validation of Nodal Results and Mesh Deformation

Following the verification of the mesh parsing algorithms for `.mphtxt` and `.inp` formats, the subsequent phase focused on validating the parsing of simulation results (nodal values). Specifically, this test ensures that nodal data in `.txt` format is correctly mapped to the mesh nodes and that the resulting mesh deformation is accurately calculated. The validation procedure involves parsing the displacement field from the result file and applying it to the undeformed mesh geometry to generate the deformed configuration. This process was verified for 1D, 2D, and 3D mesh cases.

Validation of Displacement in the 1D Mesh Domain

Figure 4.7 presents the validation result for a 1D domain, where the underlying mesh consists of a COMSOL `edg` element (a linear 1D edge element). The validation process compares the

reference coordinates with the parsed displacement application to verify accuracy. In the undeformed configuration, depicted in orange, the original nodes are labeled 0a (located at $x = 0.0$) and 1a (located at $x = 2.0$). The parser subsequently reads the displacement values from the source `.txt` file, identifying a displacement of $u_0 = -1.0$ for the first node and $u_1 = 0.5$ for the second. Applying these values results in the deformed configuration shown in green, where Node 0 moves to position 0b ($0.0 - 1.0 = -1.0$) and Node 1 moves to position 1b ($2.0 + 0.5 = 2.5$). The exact correspondence between the expected coordinates and the visualized green nodes confirms that the parser correctly interprets and applies scalar or vector fields to the mesh topology.



Figure 4.7: Visualization of 1D mesh deformation. The orange nodes (0a, 1a) represent the initial undeformed state of the `edg` element. The green nodes (0b, 1b) depict the deformed state after applying the displacement values parsed from the `.txt` file.

Validation of Displacement in the 2D Mesh Domain

The validation was extended to two-dimensional domains using a quadratic quadrilateral element. Figure 4.8 illustrates the results for a single 9-node quadratic quadrilateral element (COMSOL type `quad2`). The undeformed geometry, represented by the orange wireframe, is a square defined by corner nodes 0a, 1a, 2a, and 3a at coordinates $(0.0, 0.0)$, $(2.0, 0.0)$, $(0.0, 2.0)$, and $(2.0, 2.0)$ respectively, with mid-side and central nodes completing the definition.

The parser validates the deformation field by reading nodal displacement vectors (u_x, u_y) from the input file and mapping them to the corresponding mesh vertices. For instance, the corner nodes are subjected to symmetric horizontal expansion: Node 0 moves by $(-0.5, 0.0)$ to a new position 0b at $(-0.5, 0.0)$, while Node 1 shifts by $(0.5, 0.0)$ to 1b at $(2.5, 0.0)$. Vertical deformation is observed in the horizontal mid-side nodes; specifically, Node 4 at the bottom edge is displaced by $(0.0, -0.5)$ to position 4b, and Node 8 at the top edge is displaced by $(0.0, 0.5)$ to position 8b. In contrast, the vertical mid-side nodes, 5 and 7, remain stationary as their defined displacement is $(0.0, 0.0)$, preserving the element's width at the centerline. Additionally, the central node (Node 6) experiences a slight vertical shift of 0.25 units. The visual agreement between the calculated green nodal positions and the expected geometric distortion confirms the parser's accuracy in handling multi-dimensional vector fields for quadratic elements.

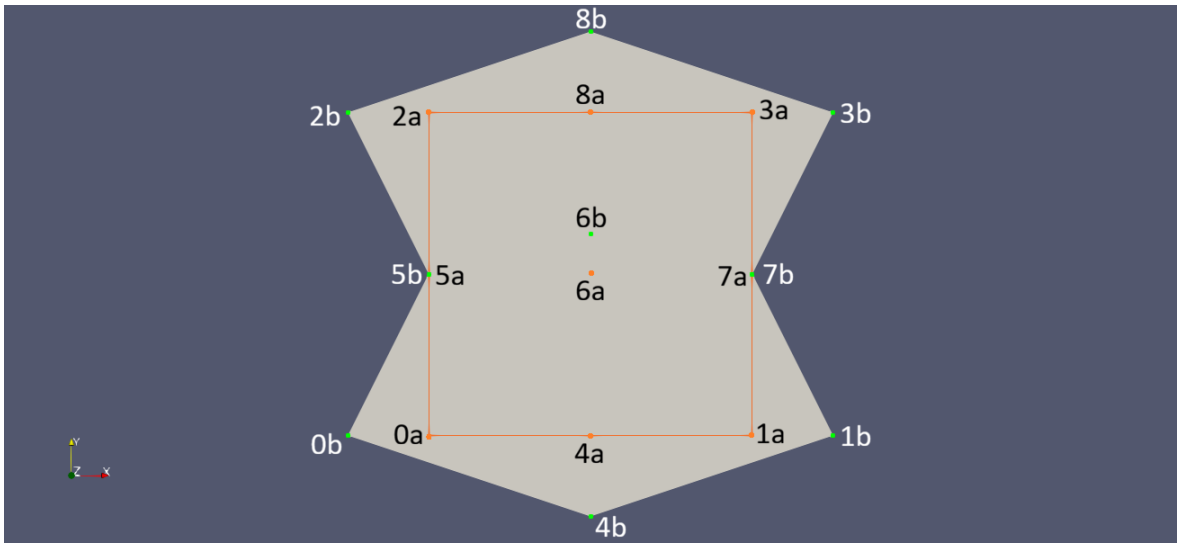


Figure 4.8: Visualization of 2D mesh deformation on a 9-node `quad2` element. The orange mesh (nodes labeled with suffix 'a') shows the initial unit square configuration. The green mesh (nodes labeled with suffix 'b') depicts the deformed state, characterized by symmetric horizontal expansion at the corners and vertical displacement of the horizontal mid-side nodes, while the vertical mid-side nodes (5 and 7) remain stationary.

Validation of Displacement in the 3D Mesh Domain

The final validation phase addresses the 3D domain. Figure 4.9 demonstrates the parsing results for a composite mesh consisting of two distinct volumetric element types. The geometry is constructed from a 14-node quadratic pyramid (`pyr2`) stacked upon an 8-node linear hexahedron (`hex`).

The undeformed configuration is shown in orange. The pyramid is defined by a square base with corner nodes 0a to 3a. It rises to an apex at node 4a. The hexahedron shares these base nodes as its top surface, extending downwards to nodes 14a through 17a.

The parser correctly processes the 3D vector field (u_x, u_y, u_z), updating the topology for every node in the mesh. The displacement of the shared interface nodes 0a to 3a is characterized by a symmetric expansion along the x-axis. Node 0a shifts from $(0.0, 0.0, 0.0)$ to 0b at $(-0.5, 0.0, 0.0)$. Node 1a moves from $(2.0, 0.0, 0.0)$ to 1b at $(2.5, 0.0, 0.0)$. Similarly, Node 2a at $(0.0, 2.0, 0.0)$ is displaced to 2b at $(-0.5, 2.0, 0.0)$. Node 3a at $(2.0, 2.0, 0.0)$ is displaced to 3b at $(2.5, 2.0, 0.0)$.

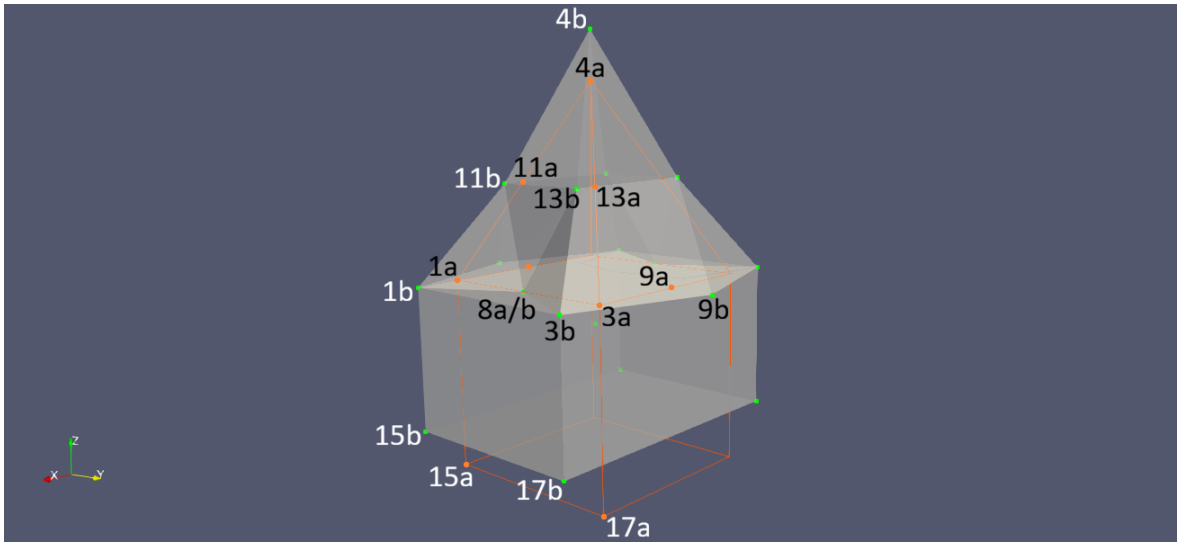
The pyramid apex and mid-side nodes exhibit distinct deformation modes. The apex is Node 4a. It undergoes a pure vertical translation from $(1.0, 1.0, 2.0)$ to 4b at $(1.0, 1.0, 2.5)$. The base mid-side nodes 5a and 9a show y-direction shifts. Node 5a moves from $(1.0, 0.0, 0.0)$ to 5b at $(1.0, -0.5, 0.0)$. Node 9a moves from $(1.0, 2.0, 0.0)$ to 9b at $(1.0, 2.5, 0.0)$. The remaining mid-side nodes 6a and 8a remain stationary at $(0.0, 1.0, 0.0)$ and $(2.0, 1.0, 0.0)$ respectively.

A specific exception applies to Node 7a. This node exists in the mesh vertex list with coordinates $(1.0, 1.0, 0.0)$. It has an associated displacement vector of $(0.0, 0.0, -0.5)$. However, it is excluded from the element connectivity of the 14-node pyramid formulation used in VTK. This is explained in Section 3.3.1. This node remains independent from the pyramid topology; consequently, it is neglected during the visualization of the deformed element.

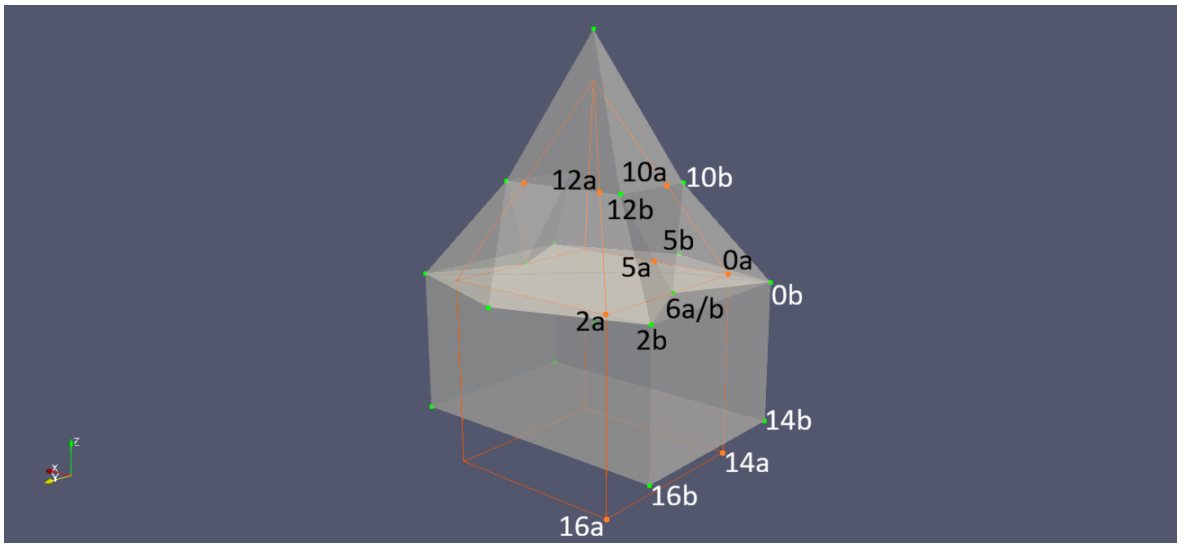
The quadratic mid-edge nodes on the pyramid slope experience localized x-direction shifts. Node 10a moves from $(0.5, 0.5, 1.0)$ to 10b at $(0.25, 0.5, 1.0)$. Node 11a moves from $(1.5, 0.5, 1.0)$ to 11b at $(1.75, 0.5, 1.0)$. Node 12a shifts from $(0.5, 1.5, 1.0)$ to 12b at $(0.25, 1.5, 1.0)$. Node 13a shifts from $(1.5, 1.5, 1.0)$ to 13b at $(1.75, 1.5, 1.0)$.

Finally, the base nodes of the hexahedron undergo a combined transformation involving

x-expansion and z-elevation. Node 14a moves from $(0.0, 0.0, -2.0)$ to 14b at $(-0.5, 0.0, -1.5)$. Node 15a moves from $(2.0, 0.0, -2.0)$ to 15b at $(2.5, 0.0, -1.5)$. Node 16a moves from $(0.0, 2.0, -2.0)$ to 16b at $(-0.5, 2.0, -1.5)$. Node 17a moves from $(2.0, 2.0, -2.0)$ to 17b at $(2.5, 2.0, -1.5)$. This exhaustive verification confirms the precision of the parser.



(a) First perspective view



(b) Second perspective view

Figure 4.9: Visualization of 3D mesh deformation on a composite domain containing a `pyr2` and a `hex` element. Subfigures (a) and (b) present alternate perspective views. Orange indicates the initial state, and green indicates the final deformed state.

4.2 Validation of Mesh Quality Metrics

This section details the validation of the implemented mesh quality algorithms. As the mathematical definitions of these metrics are well-established in the literature, the primary objective here is to verify the correct implementation of the software's calculation and visualization modules. To achieve this, a comprehensive suite of unit tests was developed using

single-element meshes. This approach allows for the isolation of specific geometric distortions, such as skewness, length ratio, or Jacobian determinants, ensuring that the software correctly identifies and quantifies each defect.

To provide a holistic view of the element quality, the full set of calculated metrics (Shape Metric, Skewness Metric, Length Ratio Metric, Orientation Metric, Jacobian Ratio, and Volume Metric) is reported for every test case. This allows for verification that specific geometric changes only affect the relevant metrics.

The verification process utilizes a standardized color-coded legend to visually represent the numerical quality of the elements. The grading scale is defined as follows: Blue represents a metric value between 0.8 and 1.0. Green corresponds to the range of 0.6 to 0.8, followed by yellow (0.4–0.6) and orange (0.2–0.4). Finally, red denotes values between 0.0 and 0.2. While this scale provides a general visual overview, specific acceptance criteria depend on the metric in question. For example, regarding the Jacobian Ratio, values below $0.0\overline{3}$ are considered indicative of poor element quality in ANSYS [6].

4.2.1 Validation of 1D Elements in 1D Meshes

The first set of unit tests focuses on the 1D Quadratic Edge element in a 1D mesh. The validation results, presented in Figure 4.10, compare a geometrically perfect element against instances of Jacobian distortion and topological inversion.

The reference case (Figure 4.10a) demonstrates a perfect element where the mid-side node is exactly centered. As expected, the software calculates perfect scores across the board: Shape Metric, Skewness Metric, Length Ratio Metric, Orientation Metric, Jacobian Ratio, and Volume Metric are all exactly 1.0.

In contrast, Figure 4.10b illustrates a case where the mid-side node is displaced significantly toward one end (coordinate 0.26 instead of 0.5). While the element remains valid in terms of connectivity, this distortion severely impacts the mapping quality. The Jacobian Ratio plummets to 0.0204, indicating that the element is suffering from parametric distortion.

Finally, Figure 4.10c depicts an invalid element where the determinant of the Jacobian matrix is non-positive at a validation point. In this scenario, the software correctly flags the element as invalid and assigns a default value of 0.0 to all metrics.

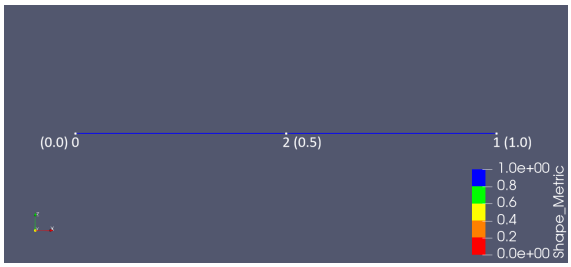
4.2.2 Validation of 2D Elements in 2D Meshes

The robustness of the metrics for two-dimensional elements in a two-dimensional mesh was tested using a series of quadrilateral elements. Each test case, shown in Figure 4.11, applies a specific form of geometric degradation.

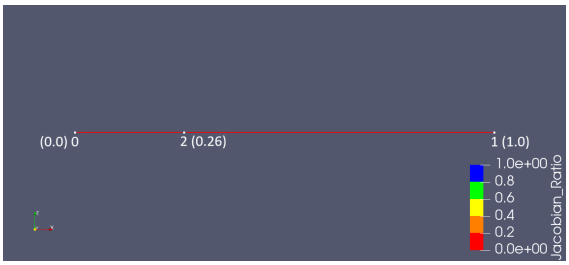
The reference configuration is a unit square (Figure 4.11a). This ideal geometry yields a perfect score of 1.0 for all calculated metrics.

When the element is subjected to shear deformation (Figure 4.11b), the metrics accurately reflect the angular distortion. The Shape Metric drops to 0.1905 and the Skewness Metric to 0.2425. The Length Ratio also degrades slightly to 0.7854, and the Jacobian Ratio falls to 0.25. However, the Orientation Metric remains 1.0, confirming that the element is not rotated relative to the global axes.

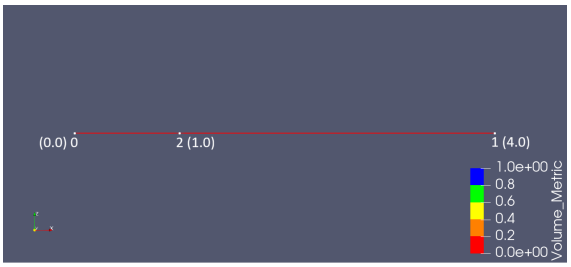
Anisotropic scaling is applied to create a high length ratio element in Figure 4.11c. Here, the Length Ratio Metric clearly isolates the defect with a value of 0.198. The Shape Metric also reflects this elongation. Conversely, the Skewness Metric, Orientation Metric, and Jacobian Ratio all remain at 1.0, proving that the element is orthogonal and well-mapped despite its slenderness.



(a) Perfect element: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 1.0.



(b) Bad Jacobian Ratio: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 0.0204, and Volume Metric = 0.04.



(c) Inverted element: Shape Metric = 0.0, Skewness Metric = 0.0, Length Ratio = 0.0, Orientation Metric = 0.0, Jacobian Ratio = 0.0, and Volume Metric = 0.0.

Figure 4.10: Visual validation of quality metrics for 1D Quadratic Edge elements in 1D mesh.

Rotational invariance is tested in Figure 4.11d. Although the element is geometrically a perfect square, its 135-degree rotation results in a specific Orientation Metric of 0.146. Crucially, all internal quality metrics remain perfect at 1.0, verifying that the software distinguishes between poor alignment and poor geometry.

Severe distortion is analyzed in Figure 4.11e, where a node displacement creates a near-concave geometry. This impending collapse causes widespread degradation: Jacobian Ratio (0.03), Shape Metric (0.0579), Length Ratio (0.102), and Volume Metric (0.09) all indicate failure. Only the Orientation Metric (1.0) remains unaffected.

Finally, the inverted element case (Figure 4.11f) represents a "bow-tie" self-intersection. The software assigns a value of 0.0 to all metrics for this invalid topology. It is noted that the visualization artifact marked with an 'X' is caused by rendering limitations for self-intersecting polygons.

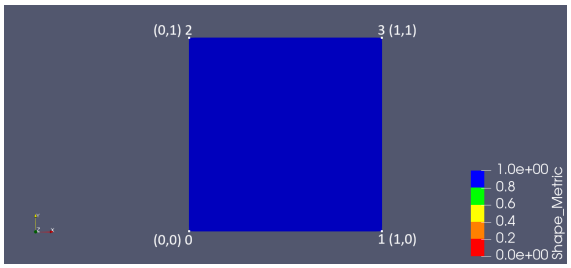
4.2.3 Validation of 2D Elements in 3D Meshes

Validation extends to two-dimensional surface elements embedded in three-dimensional space using triangular elements. Figure 4.12 displays the test cases, subjecting a standard triangle to various geometric transformations.

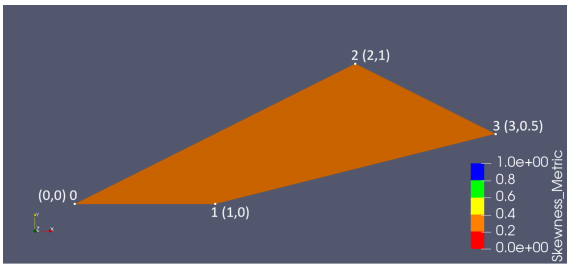
The reference configuration (Figure 4.12a) is a right-angled triangle. This geometry produces a perfect score of 1.0 for all metrics.

Shear deformation (Figure 4.12b) significantly impacts the element quality. The Skewness Metric drops to 0.0397, and the Shape Metric falls to 0.0309. The Orientation Metric also decreases to 0.730.

Anisotropic scaling creates a slender element in Figure 4.12c. The Length Ratio Metric (0.1245) and Shape Metric (0.1245) successfully isolate this defect. Skewness Metric and



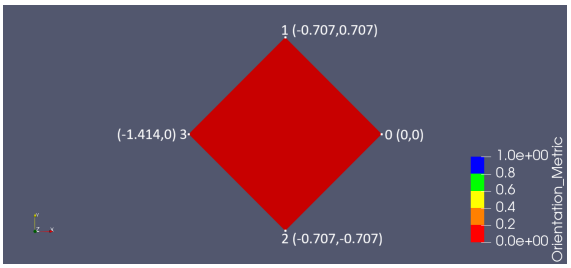
(a) Perfect element: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 1.0.



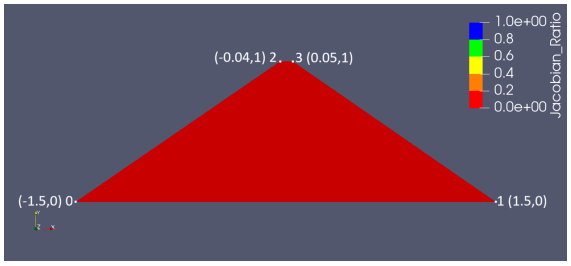
(b) Bad Skewness: Shape Metric = 0.1905, Skewness Metric = 0.2425, Length Ratio = 0.7854, Orientation Metric = 1.0, Jacobian Ratio = 0.25, and Volume Metric = 0.5.



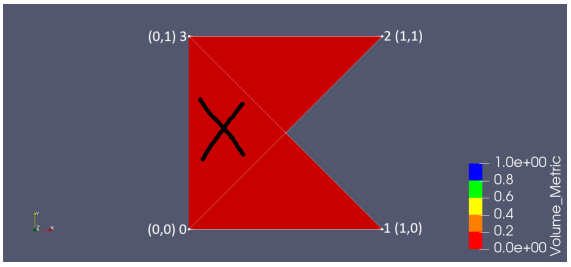
(c) Bad Length Ratio: Shape Metric = 0.198, Skewness Metric = 1.0, Length Ratio = 0.198, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 10.0.



(d) Bad Orientation: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 0.146, Jacobian Ratio = 1.0, and Volume Metric = 0.9997.



(e) Bad Jacobian Ratio: Shape Metric = 0.0579, Skewness Metric = 0.568, Length Ratio = 0.102, Orientation Metric = 1.0, Jacobian Ratio = 0.03, and Volume Metric = 0.09.



(f) Inverted element: Shape Metric = 0.0, Skewness Metric = 0.0, Length Ratio = 0.0, Orientation Metric = 0.0, Jacobian Ratio = 0.0, and Volume Metric = 0.0.

Figure 4.11: Visual validation of quality metrics for a 2D Quadrilateral element within a 2D mesh domain.

Orientation Metric remain 1.0, correctly identifying the element as orthogonal despite the high length ratio.

The rotational test (Figure 4.12d) highlights a critical distinction between 2D-in-2D and 2D-in-3D analysis. The element is rotated, yet the Orientation Metric remains 1.0. This behavior stems from the mapping formulation; the Jacobian matrix for a 2D element in 3D space is non-square (2×3), requiring the analysis of the metric tensor $\mathbf{J}\mathbf{J}^T$. This mathematical operation results in the loss of explicit element orientation information [23].

Near-collapse is analyzed in Figure 4.12e. Here, mid-side nodes are displaced toward the corners, creating a near-singular mapping. Although the geometric area remains large, the local Jacobian approaches zero. The Jacobian Ratio reflects this with a value of 0.00052.

The inverted topology (Figure 4.12f) forces a self-intersection. Consistent with previous tests, the software assigns a value of 0.0 to all metrics, flagging the invalid geometry. Furthermore, the determinant calculation relies on $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$, which causes the loss of sign information due to the squaring of terms. Consequently, the determinant sign cannot be used to verify element validity in this context.

4.2.4 Validation of 3D Elements in 3D Meshes

The validation was extended to three-dimensional geometries using a series of linear wedge (prism) elements. Each test case, shown in Figure 4.13, applies a specific form of geometric degradation to a volumetric element.

The reference configuration is a regular prism (Figure 4.13a). This ideal geometry yields a perfect score of 1.0 for all calculated metrics.

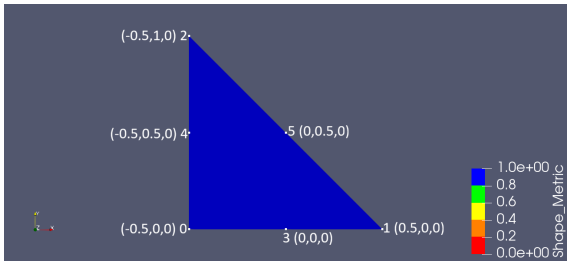
When the element is subjected to severe shear deformation (Figure 4.13b), the Skewness Metric drops significantly to 0.0629, and the Shape Metric reduces to 0.25. Unlike the 2D case, this complex 3D distortion also affects the Orientation Metric (0.6581) and the Length Ratio Metric (0.5976). However, the Jacobian Ratio remains 1.0, indicating that the mapping validity is preserved despite the poor shape.

Anisotropic scaling is applied to create a high length ratio element in Figure 4.13c. Here, the Length Ratio Metric clearly isolates the defect with a value of 0.2095. The Shape Metric also reflects this elongation. Conversely, the Skewness Metric, Orientation Metric, and Jacobian Ratio all remain at 1.0, proving that the element is orthogonal and well-mapped despite its slenderness.

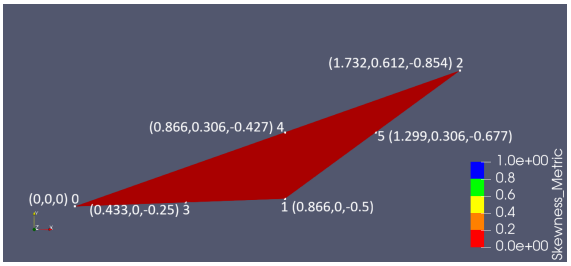
Rotational invariance is tested in Figure 4.13d. Although the element is geometrically a perfect prism, its rotation results in a specific Orientation Metric of 0.125. Crucially, all internal quality metrics remain perfect at 1.0, verifying that the software distinguishes between poor alignment and poor geometry.

Severe distortion is analyzed in Figure 4.13e, where the top face of the prism is scaled down to near-collapse. This tapering causes the Jacobian Ratio to plummet to 0.001024. The Shape Metric and Length Ratio Metric also degrade to 0.0678, while the Skewness Metric remains 1.0, accurately reflecting that the element is tapered but not sheared.

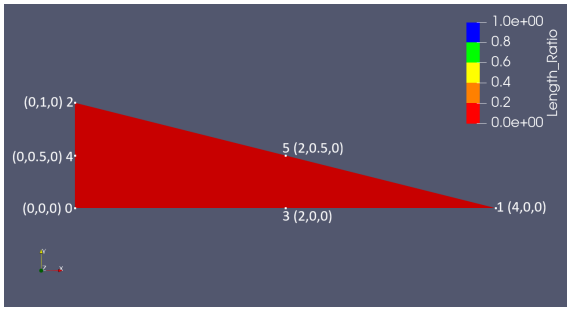
Finally, the inverted element case (Figure 4.13f) represents a topological inversion where nodes are displaced to create a negative volume. The software assigns a value of 0.0 to all metrics for this invalid topology.



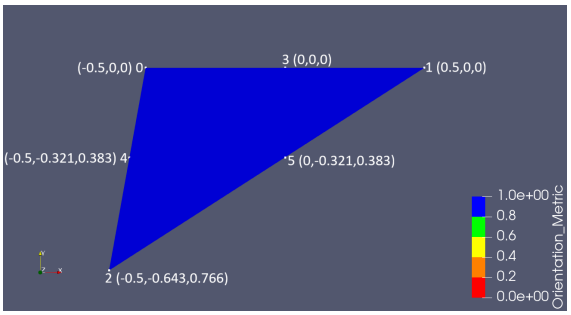
(a) Perfect element: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 1.0.



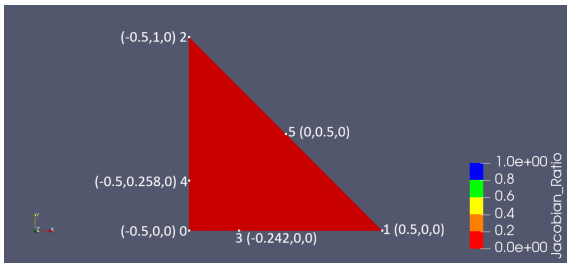
(b) Bad Skewness: Shape Metric = 0.0309, Skewness Metric = 0.0397, Length Ratio = 0.779, Orientation Metric = 0.730, Jacobian Ratio = 1.0, and Volume Metric = 0.6249.



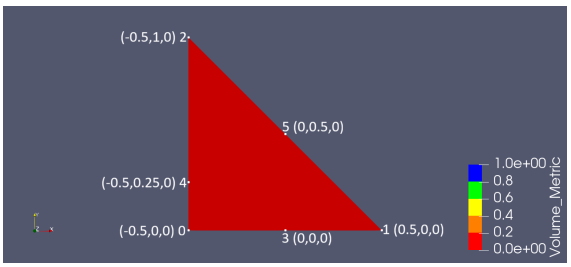
(c) Bad Length Ratio: Shape Metric = 0.1245, Skewness Metric = 1.0, Length Ratio = 0.1245, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 4.0.



(d) Bad Orientation: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 1.0.

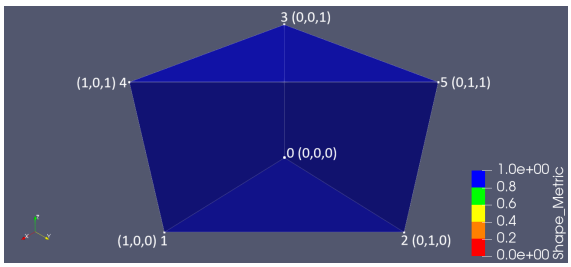


(e) Bad Jacobian Ratio: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 0.00052, and Volume Metric = 0.00102.

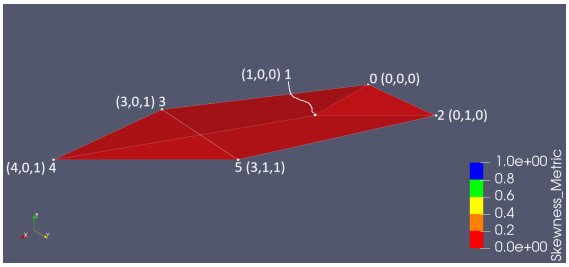


(f) Inverted element: Shape Metric = 0.0, Skewness Metric = 0.0, Length Ratio = 0.0, Orientation Metric = 0.0, Jacobian Ratio = 0.0, and Volume Metric = 0.0.

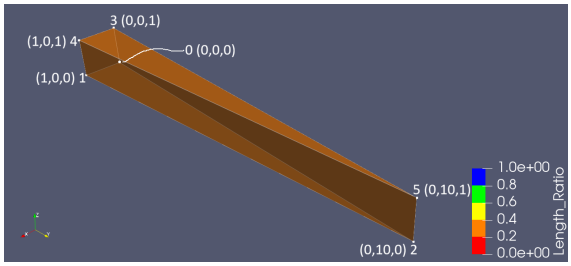
Figure 4.12: Visual validation of 2D Triangular element quality metrics embedded in 3D space.



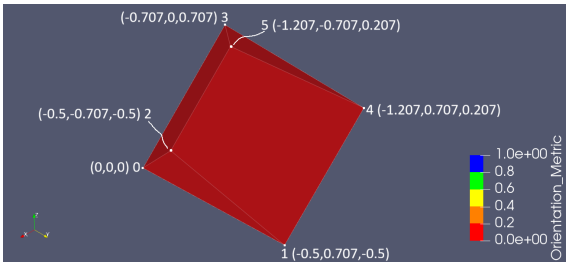
(a) Perfect element: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 1.0.



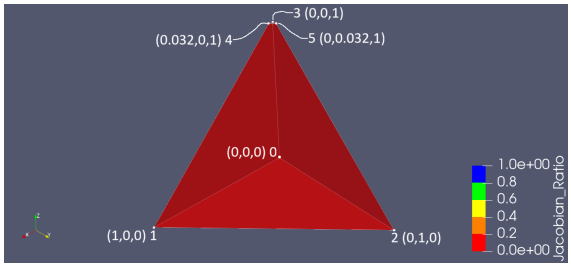
(b) Bad Skewness: Shape Metric = 0.25, Skewness Metric = 0.0629, Length Ratio = 0.5976, Orientation Metric = 0.6581, Jacobian Ratio = 1.0, and Volume Metric = 1.0.



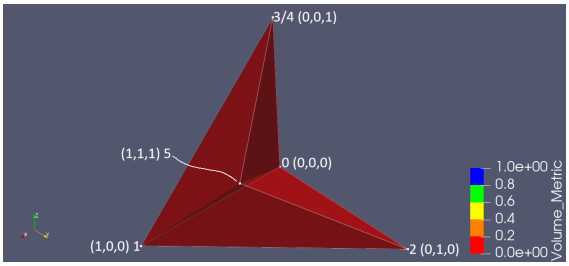
(c) Bad Length Ratio: Shape Metric = 0.2095, Skewness Metric = 1.0, Length Ratio = 0.2095, Orientation Metric = 1.0, Jacobian Ratio = 1.0, and Volume Metric = 10.0.



(d) Bad Orientation: Shape Metric = 1.0, Skewness Metric = 1.0, Length Ratio = 1.0, Orientation Metric = 0.125, Jacobian Ratio = 1.0, and Volume Metric = 0.9997.



(e) Bad Jacobian Ratio: Shape Metric = 0.0678, Skewness Metric = 1.0, Length Ratio = 0.0678, Orientation Metric = 1.0, Jacobian Ratio = 0.001024, and Volume Metric = 0.001024.



(f) Inverted element: Shape Metric = 0.0, Skewness Metric = 0.0, Length Ratio = 0.0, Orientation Metric = 0.0, Jacobian Ratio = 0.0, and Volume Metric = 0.0.

Figure 4.13: Visual validation of quality metrics for 3D Wedge (Prism) elements.

4.3 Validation of Quadrature Optimization

Following the verification of mesh quality metrics, the validation process proceeds to the numerical integration module. The primary objective of this phase is to determine the optimal number of integration points (Gaussian points) required to approximate the finite element governing equations within a specified error tolerance (10^{-4}). Optimization of this parameter is critical for computational efficiency; utilizing fewer points than the theoretical upper bound ($d_{\text{poly}} \leq 2n_{gp} - 1$), without compromising accuracy, can significantly reduce solution time.

This validation focuses specifically on the evaluation of the mass matrix, which involves integrating the product of shape functions and the material density, ρ, over the element domain. It should be noted that the analysis of the mass matrix for non-square Jacobian cases (where the element dimension differs from the mesh dimension), as well as the stiffness matrix for all Jacobian cases, is excluded from this section. These scenarios are omitted due to specific limitations in the implemented approximation method, which are discussed in detail in Section 4.4.

The following subsections analyze two key factors influencing the required quadrature order: the polynomial degree of the material property and the geometric quality of the element. It is important to note that isolating a specific geometric distortion is not always possible, as a single distortion often affects multiple quality metrics simultaneously. For the purposes of this analysis, we define several key terms. *Max Degree* refers to the maximum polynomial degree present in the integrand matrix (d_{poly}). *Theoretical GP* denotes the number of Gaussian points theoretically required to integrate the function without error, governed by the rule $d_{\text{poly}} = 2n_{gp,\text{theoretical}} - 1$. *Optimal GP* represents the minimum number of Gaussian points required to achieve an error lower than or equal to the specified tolerance (10^{-4}). Finally, a *Perfect Element* is defined as a reference element with no geometric distortion, possessing an ideal shape and linear mapping.

4.3.1 Analysis of 1D Quadratic Line Element in 1D Mesh

Influence of Material Polynomial Degree

Material Function $\rho(x)$	Degree of ρ	Max Degree in Matrix	Theoretical GP	Optimal GP	Difference
1.0	Constant	4	3	3	0
$1+2x$	Linear	5	3	3	0
$1-2x+3x^2$	Quadratic	6	4	4	0

Table 4.1: Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 1D Quadratic Edge).

The first study examines a 1D quadratic element to isolate the impact of material property complexity on integration requirements. Table 4.1 presents the relationship between the polynomial degree of the density function, $\rho(x)$, and the resulting quadrature points.

As shown in Table 4.1, the theoretical number of Gaussian points strictly matches the optimal number determined by the algorithm; no reduction in points was possible for the tolerance of 10^{-4}. Notably, both the constant and linear material properties require the same number of Gaussian points (3), despite the increase in the integrand's maximum polynomial degree from 4 to 5.

This behavior is mathematically consistent with the Gaussian quadrature rule, which states that n points can exactly integrate a polynomial of degree up to $2n-1$. For 3 Gaussian points, the maximum integrable degree is 5 ($2 \cdot 3 - 1$). Since both the constant case (degree

4) and the linear case (degree 5) fall within this limit, the required number of points remains constant at 3. However, the quadratic case raises the total degree to 6, which exceeds the capacity of 3 points. Consequently, the requirement jumps to 4 points, which are capable of integrating polynomials up to degree 7.

Influence of Geometric Distortion

Test Case	Element Quality Metrics						Integrand	Gaussian Points		
	Shape	Skew	Len. Rat.	Orient.	Jac. Rat.	Vol.	Max Deg.	Theo.	Opt.	Diff.
Perfect Element	1.0	1.0	1.0	1.0	1.0	1.0	4	3	3	0
Bad Jacobian Ratio	1.0	1.0	1.0	1.0	0.0204	0.04	5	3	3	0

Table 4.2: Effect of geometric distortion on Gaussian quadrature requirements. The distortion is introduced by shifting the mid-side node, lowering the Jacobian Ratio. (Material: Constant $\rho(x) = 1.0$, Tolerance: 10^{-4}).

Table 4.2 compares a *Perfect Element* against one with a *Bad Jacobian Ratio*. While a perfect element implies a linear mapping between the reference and global coordinates, shifting the mid-side node (indicated by a low Jacobian Ratio of 0.0204) introduces a non-linear mapping.

The distortion of the element geometry has a direct impact on the algebraic structure of the integral. The non-linear mapping increases the complexity of the Jacobian determinant, raising the maximum polynomial degree of the integrand from 4 (in the perfect case) to 5. However, similar to the material analysis above, this increase does not impact the computational cost. Because 3 Gaussian points provide exact integration for polynomials up to degree 5, the algorithm correctly identifies that no additional points are required despite the geometric distortion. This confirms that the solver remains efficient even when handling elements with significant Jacobian degradation, provided the polynomial degree remains within the limit of the quadrature rule.

4.3.2 Analysis of 2D Biquadratic Quad Elements in 2D Mesh

Influence of Material Polynomial Degree

Material Function $\rho(x,y)$	Degree of ρ	Max Degree in Matrix	Theoretical GP	Optimal GP	Difference
1.0	Constant	8	5	3	2
$1+2x$	Linear	9	5	3	2
$1+2x-y$	Bilinear	9	5	3	2
$1+2x^2+3y-xy$	Quadratic	10	6	4	2

Table 4.3: Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 2D Biquadratic Quad).

The analysis was extended to 2D Biquadratic elements to evaluate the solver's performance in multi-dimensional domains. Table 4.3 highlights a significant efficiency gain compared to the 1D case. Across all tested material properties, the algorithm identified that only 3 or 4 Gaussian points (per direction) were necessary to meet the error tolerance, whereas the theoretical bound suggested 5 or 6.

This represents a consistent saving of 2 points per dimension, validating the effectiveness of the adaptive point selection strategy. Unlike the 1D case where the theoretical and optimal values matched perfectly, the 2D results suggest that the theoretical upper bound is overly conservative for the specific algebraic structure of the mass matrix in higher dimensions.

Influence of Geometric Distortion

Test Case	Element Quality Metrics						Integrand	Gaussian Points		
	Shape	Skew	Len. Rat.	Orient.	Jac. Rat.	Vol.	Max Deg.	Theo.	Opt.	Diff.
Perfect Element	1.0	1.0	1.0	1.0	1.0	1.0	9	5	3	2
Bad Skewness	0.2	0.2	1.0	1.0	1.0	0.2	9	5	3	2
Bad Length Ratio	0.198	1.0	0.198	1.0	1.0	10.0	15	8	5	3
Bad Orientation	1.0	1.0	1.0	0.179	1.0	1.0	9	5	3	2
Bad Jacobian Ratio	1.0	1.0	1.0	1.0	0.0002	0.0004	17	9	5	4

Table 4.4: Effect of geometric distortion on Gaussian quadrature requirements for a Bilinear material property ($\rho = 1 + 2x - y$, Tolerance: 10^{-4}).

Table 4.4 details the impact of element quality on integration complexity. Notably, geometric distortions involving high Aspect Ratio (*Bad Length Ratio*) and severe Jacobian distortion (*Bad Jacobian Ratio*) exhibited a dramatic increase in the integrand's maximum polynomial degree, rising to 15 and 17 respectively.

This algebraic complexity forced the algorithm to increase the optimal Gaussian points from 3 to 5 to maintain accuracy. However, it is crucial to observe that the efficiency gain, represented by the difference between theoretical and optimal points, actually increased for these complex cases. For the *Bad Jacobian Ratio* scenario, while the theoretical bound demanded 9 points to handle the degree of 17, the adaptive solver converged with only 5 points. This results in a saving of 4 points per dimension, demonstrating that the adaptive strategy becomes increasingly valuable as the element geometry degrades and the theoretical upper bounds become excessively conservative.

4.3.3 Analysis of 3D Hexahedron Elements in 3D Mesh

Influence of Material Polynomial Degree

Material Function $\rho(x,y,z)$	Degree of ρ	Max Degree in Matrix	Theoretical GP	Optimal GP	Difference
1.0	Constant	6	4	2	2
$1-x$	Linear	7	4	2	2
$1-x+2z$	Bilinear	7	4	2	2
$1-x+10y+2z$	Trilinear	7	4	2	2
$1-x^2+10y+2z$	Quadratic	8	5	3	2

Table 4.5: Influence of material property polynomial degree on Gaussian quadrature requirements. (Tolerance: 10^{-4}, Geometry: Perfect 3D Hexahedron).

Finally, the module was validated for 3D Hexahedral elements. Similar to the 2D case, Table 4.5 demonstrates a consistent reduction of 2 Gaussian points between the theoretical and optimal values for all undistorted cases. This confirms that for standard hexahedral elements, the theoretical upper bound is strictly conservative; the adaptive algorithm is able to exploit the specific structure of the shape functions to achieve the same accuracy with significantly fewer integration points.

Influence of Geometric Distortion

Test Case	Element Quality Metrics						Integrand	Gaussian Points		
	Shape	Skew	Len. Rat.	Orient.	Jac. Rat.	Vol.	Max Deg.	Theo.	Opt.	Diff.
Perfect Element	1.0	1.0	1.0	1.0	1.0	1.0	8	5	3	2
Bad Skewness	0.24	0.15	1.0	0.86	1.0	0.2	8	5	3	2
Bad Length Ratio	0.21	1.0	0.21	1.0	1.0	10.0	8	5	3	2
Bad Orientation	1.0	1.0	1.0	0.15	1.0	1.0	8	5	3	2
Bad Jacobian Ratio	0.02	0.02	0.99	0.83	0.03	0.03	12	7	4	3

Table 4.6: Effect of geometric distortion on Gaussian quadrature requirements for a Quadratic material property ($\rho = 1 - x^2 + 10y + 2z$). Note that the parametric distortion in the Jacobian case significantly increases the polynomial degree from 8 to 12.

The results in Table 4.6 indicate that volumetric distortion can be computationally expensive. While Skewness, Length Ratio, and Orientation distortions did not alter the optimal point count (remaining at 3 points), the *Bad Jacobian Ratio* case increased the integrand's maximum degree from 8 to 12.

As noted in the introduction, geometric defects in 3D are often coupled; the Jacobian distortion case also exhibited degradation in Shape, Skewness, and Orientation metrics. Despite this complex distortion profile, the solver correctly adapted by increasing the integration points from 3 to 4. Notably, the efficiency gain also increased in this scenario: while the theoretical bound for degree 12 is 7 points, the optimizer found that 4 points were sufficient. This results in a difference of 3 points (up from the standard 2), effectively demonstrating that the adaptive strategy maximizes computational savings even when handling highly nonlinear element mappings.

4.4 Verification and Error Analysis of Polynomial Approximations

A fundamental design choice of the implemented software is the exclusive reliance on symbolic monomial arithmetic. The primary objective is to work with exact values derived directly from the algebraic structure of the finite element formulation, rather than relying on numerical interpolations at arbitrary evaluation points. This approach ensures high precision and provides a deterministic basis for error estimation.

However, a specific challenge arises from the error bound formulas utilized in the optimization module. These formulas are strictly dependent on the polynomial degree of the integrand. While this methodology works perfectly for standard elements, certain geometric configurations introduce non-polynomial terms, specifically square roots (arising in metric determinants of embedded elements) and rational functions (arising in inverse Jacobians).

To maintain compatibility with the polynomial-based error bounds, these non-polynomial terms are approximated using Binomial and Taylor series expansions. While this preserves the symbolic pipeline, it leads to a specific side effect: the expansions generate high-order polynomials. These *artificial* higher degrees inflate the input to the error bound formula, resulting in wrong estimates for the required number of Gaussian points.

It is important to note that for cases where elements have constant determinant terms, the software handles the integration exactly without these approximations. In such *valid* cases, the square root or inverse terms resolve to scalars, allowing direct symbolic integration. The following subsections verify the behavior of the solver in both valid (exact) and invalid (approximated) scenarios.

4.4.1 Mass Integration: Non-Square Jacobian with Constant Metric Determinant

This example illustrates the complete process for determining the optimal number of Gaussian points for a 1D quadratic edge element embedded in a 2D mesh. This is the *valid* case where the mapping remains linear. The element coordinates are defined as:

$$\mathbf{x}_0 = (0.0, 0.0), \quad \mathbf{x}_1 = (2.0, 0.0), \quad \text{and} \quad \mathbf{x}_2 = (1.0, 0.0).$$

The shape functions $\mathbf{N}$ and their parametric derivatives $\nabla\mathbf{N}$ for a quadratic edge element are:

$$\mathbf{N} = \begin{bmatrix} 1-3\xi+2\xi^2, & -\xi+2\xi^2, & 4\xi-4\xi^2 \end{bmatrix} \quad \text{and} \quad \nabla\mathbf{N} = \begin{bmatrix} -3+4\xi, & -1+4\xi, & 4-8\xi \end{bmatrix}.$$

The Jacobian matrix for a 1D element in 2D space is computed using the parametric derivatives and the matrix of nodal coordinates, $\mathbf{X}$:

$$\mathbf{J} = \nabla\mathbf{N} \cdot \mathbf{X} = \begin{bmatrix} -3+4\xi, & -1+4\xi, & 4-8\xi \end{bmatrix} \cdot \begin{bmatrix} 0.0 & 0.0 \\ 2.0 & 0.0 \\ 1.0 & 0.0 \end{bmatrix} = \begin{bmatrix} 2.0 & 0.0 \end{bmatrix}.$$

Since the Jacobian is non-square (1×2), we compute the metric determinant using $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$:

$$\begin{aligned} \mathbf{J}\mathbf{J}^T &= \begin{bmatrix} 2.0 & 0.0 \end{bmatrix} \cdot \begin{bmatrix} 2.0 \\ 0.0 \end{bmatrix} = 4.0, \\ \det(\mathbf{J}\mathbf{J}^T) &= 4.0, \\ \sqrt{\det(\mathbf{J}\mathbf{J}^T)} &= 2.0. \end{aligned}$$

With material density $\rho = 1.0$, the integrand for the mass matrix computation becomes:

$$\mathbf{M} = \int_0^1 \rho\, \mathbf{N}^T\mathbf{N}\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\, \mathrm{d}\xi = \int_0^1 \mathbf{N}^T\mathbf{N} \cdot 2.0\, \mathrm{d}\xi.$$

Expanding the integrand symbolically yields:

$$\mathbf{M} = \int_0^1 \begin{bmatrix} 8\xi^4-24\xi^3+26\xi^2-12\xi+2 & \cdots & \cdots \\ 8\xi^4-16\xi^3+10\xi^2-2\xi & 8\xi^4-8\xi^3+2\xi^2 & \cdots \\ -16\xi^4+40\xi^3-32\xi^2+8\xi & -16\xi^4+24\xi^3-8\xi^2 & \cdots \end{bmatrix} \mathrm{d}\xi.$$

The maximum polynomial degree in the integrand is 4. The theoretical minimum number of Gaussian points required for exact integration is given by:

$$n_{gp,\text{theoretical}} = \left\lceil \frac{d_{\text{poly}}+1}{2} \right\rceil = \left\lceil \frac{4+1}{2} \right\rceil = \lceil 2.5 \rceil \to 3,$$

where $d_{\text{poly}} = 4$ is the maximum polynomial degree. Since this case involved no approximation (the square root of the constant 4.0 is exact), the polynomial degree remains exact, ensuring computational efficiency.

4.4.2 Mass Integration: Non-Square Jacobian with Non-Constant Metric Determinant

This example demonstrates the Binomial expansion technique required when the Jacobian determinant becomes non-linear. Consider the same 1D quadratic edge element, but with the mid-node shifted to introduce curvature:

$$\mathbf{x}_0 = (0.0, 0.0), \quad \mathbf{x}_1 = (2.0, 0.0), \quad \text{and} \quad \mathbf{x}_2 = (1.0, 1.0).$$

The shape functions and derivatives remain the same, but the Jacobian calculation changes:

$$\mathbf{J} = \nabla\mathbf{N} \cdot \mathbf{X} = \begin{bmatrix} -3+4\xi, & -1+4\xi, & 4-8\xi \end{bmatrix} \cdot \begin{bmatrix} 0.0 & 0.0 \\ 2.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 2.0, & 4-8\xi \end{bmatrix}.$$

The metric determinant is obtained from $\mathbf{J}\mathbf{J}^T$:

$$\begin{aligned} \mathbf{J}\mathbf{J}^T &= (2.0)^2 + (4-8\xi)^2 = 64\xi^2 - 64\xi + 20, \\ \det(\mathbf{J}\mathbf{J}^T) &= 64\xi^2 - 64\xi + 20, \\ \sqrt{\det(\mathbf{J}\mathbf{J}^T)} &= \sqrt{64\xi^2 - 64\xi + 20}. \end{aligned}$$

With material density $\rho = 1.0$, the integrand for the mass matrix computation becomes:

$$\mathbf{M} = \int_0^1 \rho\ \mathbf{N}^T\mathbf{N}\sqrt{\det(\mathbf{J}\mathbf{J}^T)}\ \mathrm{d}\xi = \int_0^1 \mathbf{N}^T\mathbf{N} \cdot \sqrt{64\xi^2 - 64\xi + 20}\ \mathrm{d}\xi.$$

Formulating the exact integrand symbolically yields the product of the shape function matrix and the non-constant determinant:

$$\mathbf{M} = \int_0^1 \begin{bmatrix} 4\xi^4 - 12\xi^3 + 13\xi^2 - 6\xi + 1 & \dots & \dots \\ 4\xi^4 - 8\xi^3 + 5\xi^2 - 1\xi & 4\xi^4 - 4\xi^3 + 1\xi^2 & \dots \\ -8\xi^4 + 20\xi^3 - 16\xi^2 + 4\xi & -8\xi^4 + 12\xi^3 - 4\xi^2 & \dots \end{bmatrix} \sqrt{64\xi^2 - 64\xi + 20}\ \mathrm{d}\xi.$$

The maximum degree from $\mathbf{N}^T\mathbf{N}$ is 4. The degree of the determinant polynomial $(64\xi^2 - \dots)$ is 2. Although we cannot calculate the degree of a square root exactly in the polynomial sense, we can approximate the degree contribution of this term as half of the inner polynomial degree, which is 1. Adding this contribution to the matrix degree $(4+1)$, the maximum effective degree for the exact case is approximately 5.

Binomial Expansion Method

Since the symbolic engine cannot process the term $\sqrt{64\xi^2 - 64\xi + 20}$ directly, we apply a Binomial expansion as explained in Section 2.5.3. We define the variable s:

$$s = \frac{64\xi^2 - 64\xi + 20}{20} - 1 = 3.2\xi^2 - 3.2\xi.$$

The square root is then expanded using the binomial series where the expansion parameter is set to $N = 5$ (resulting in terms up to s^4):

$$\begin{aligned}\sqrt{64\xi^2 - 64\xi + 20} &= \sqrt{20} \cdot \sqrt{1+s}, \\ &\approx \sqrt{20} \sum_{n=0}^{N-1} \binom{1/2}{n} s^n \approx \sqrt{20}\left(1 + \frac{1}{2}s - \frac{1}{8}s^2 + \frac{1}{16}s^3 - \frac{5}{128}s^4\right).\end{aligned}$$

Substituting the expression for s and expanding yields an 8th-degree polynomial approximation:

$$\begin{aligned}\sqrt{64\xi^2 - 64\xi + 20} \approx &-18.3\xi^8 + 73.3\xi^7 - 100.7\xi^6 + 45.8\xi^5 + 3.4\xi^4 + 2.3\xi^3 + 1.4\xi^2 \\ &- 7.2\xi + 4.5.\end{aligned}$$

Integrand Comparison

To integrate this, we multiply the shape function matrix (degree 4) by the approximated determinant term (degree 8). This yields a significantly higher degree polynomial than the theoretical exact case:

$$\mathbf{M}_{\text{binom}} = \int_0^1 \begin{bmatrix} 4.0\xi^4 - \dots & \dots & \dots \\ \dots & \dots & \dots \\ \dots & \dots & 16.0\xi^4 - \dots \end{bmatrix} \cdot \left(-18.3\xi^8 + \dots\right) \mathrm{d}\xi.$$

The resulting integrand has a maximum polynomial degree of $4 + 8 = 12$.

Gaussian Point Requirements

Method	Max Degree	Theoretical GP
Ideal (Exact)	5	3
Binomial Expansion	12	7

Table 4.7: Gaussian point inflation due to Binomial Expansion vs. Ideal Exact function.

As shown in Table 4.7, the Binomial expansion artificially increases the polynomial degree from 5 to 12. Since the error bound formula used in the software depends strictly on this degree, the solver conservatively prescribes 7 Gaussian points (instead of the 3 that would theoretically suffice for the exact geometry). This confirms the need to implement another approximation method which will maintain the real degree of the polynomial to avoid this artificial inflation.

4.4.3 Stiffness Integration: Square Jacobian with Constant Determinant

This example demonstrates the calculation of the stiffness matrix integrand for a 1D quadratic edge element in 1D space, where the Jacobian determinant is constant. The element geometry is defined by three nodes along a straight line:

$$\mathbf{x}_0 = 0.0, \quad \mathbf{x}_1 = 2.0, \quad \text{and} \quad \mathbf{x}_2 = 1.0.$$

The Jacobian for this configuration is a scalar, computed as the product of the parametric derivatives and the vector of nodal coordinates, $\mathbf{X}$:

$$J = \nabla \mathbf{N} \cdot \mathbf{X} = \begin{bmatrix} -3+4\xi, & -1+4\xi, & 4-8\xi \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 2.0 \\ 1.0 \end{bmatrix} = 2.0.$$

To compute the physical derivatives of the shape functions, we invert the Jacobian:

$$\begin{aligned} \frac{d\mathbf{N}}{dx} &= \nabla \mathbf{N} \cdot J^{-1} = \begin{bmatrix} -3+4\xi, & -1+4\xi, & 4-8\xi \end{bmatrix} \cdot 0.5, \\ &= \begin{bmatrix} 2.0\xi - 1.5, & 2.0\xi - 0.5, & 2.0 - 4.0\xi \end{bmatrix}. \end{aligned}$$

For a material constant $C = 1.0$, the stiffness matrix integrand is computed using the gradient matrix $\mathbf{B} = d\mathbf{N}/dx$:

$$\begin{aligned} \mathbf{K} &= \int_0^1 \mathbf{B}^T C \mathbf{B} \det(J) \, \mathrm{d}\xi \\ &= \int_0^1 \left(\frac{d\mathbf{N}}{dx} \right)^T \cdot 1.0 \cdot \left(\frac{d\mathbf{N}}{dx} \right) \cdot 2.0 \, \mathrm{d}\xi \\ &= \int_0^1 2.0 \cdot \mathbf{B}^T \mathbf{B} \, \mathrm{d}\xi. \end{aligned}$$

Expanding the integrand symbolically yields:

$$\mathbf{K} = \int_0^1 \begin{bmatrix} 8\xi^2 - 12\xi + 4.5 & \dots & -16\xi^2 + 20\xi - 6.0 \\ \dots & 8\xi^2 - 4\xi + 0.5 & \dots \\ -16\xi^2 + 20\xi - 6.0 & \dots & 32\xi^2 - 32\xi + 8.0 \end{bmatrix} \mathrm{d}\xi.$$

The integrand is a polynomial of degree 2. The theoretical number of Gaussian points required is:

$$n_{gp,\text{theoretical}} = \left\lceil \frac{d_{\text{poly}} + 1}{2} \right\rceil = \left\lceil \frac{2+1}{2} \right\rceil = \lceil 1.5 \rceil \to 2.$$

The adaptive algorithm correctly identifies that 2 Gaussian points are sufficient. This confirms that for exact polynomial inputs, the symbolic engine provides the optimal quadrature order.

4.4.4 Stiffness Integration: Square Jacobian with Non-Constant Determinant

This final example demonstrates the application of Taylor series expansion to handle a non-constant Jacobian determinant in stiffness matrix integration. Consider the same 1D quadratic edge element, but with an off-center mid-node at $\mathbf{x}_2 = 0.6$. The Jacobian becomes a linear function of ξ:

$$J = 3.2\xi + 0.4.$$

The exact inverse Jacobian contains rational terms, which cannot be represented in the monomial polynomial engine:

$$J^{-1} = \frac{1}{\det(J)} = \frac{1}{3.2\xi + 0.4}.$$

In this case, the exact integrand for the stiffness matrix is a rational function. We can determine its effective polynomial degree by analyzing the degrees of the numerator and denominator separately:

$$\mathbf{K} = \int_0^1 \mathbf{B}^T \mathbf{B} \det(J)\, d\xi = \int_0^1 (\nabla \mathbf{N} \det(J)^{-1})^T (\nabla \mathbf{N} \det(J)^{-1}) \det(J)\, d\xi = \int_0^1 \frac{(\nabla \mathbf{N})^T (\nabla \mathbf{N})}{\det(J)}\, d\xi.$$

The numerator $(\nabla \mathbf{N})^T (\nabla \mathbf{N})$ is the product of two linear shape function derivative matrices. Since $\deg(\nabla \mathbf{N}) = 1$, the numerator has degree 2. The denominator J is a linear function (degree 1). Thus, the effective algebraic degree is calculated as:

$$\deg(\mathbf{K}_{\text{exact}}) = \deg(\text{Numerator}) - \deg(\text{Denominator}) = 2 - 1 = 1.$$

Thus, the effective polynomial degree of the exact physics is 1.

Taylor Series Expansion

To obtain a polynomial approximation suitable for Gaussian quadrature, we apply a Taylor series expansion around the constant term $a_0 = 0.4$. We define the variable r:

$$r = 1 - \frac{3.2\xi + 0.4}{0.4} = -8\xi.$$

The inverse Jacobian is then expanded using the geometric series formula $\frac{1}{1-r} = \sum r^n$. For an expansion limit of $N = 5$ (terms up to r^4):

$$J^{-1} = \frac{1}{0.4} \cdot \frac{1}{1-r} \approx 2.5 \sum_{n=0}^{4} r^n \approx 2.5\left(1 + r + r^2 + r^3 + r^4\right).$$

Substituting $r = -8\xi$ yields a 4th-degree polynomial approximation for the inverse Jacobian:

$$J^{-1}_{\text{taylor}} \approx -10240.0\xi^4 - 1280.0\xi^3 - 160.0\xi^2 - 20.0\xi + 2.5.$$

Integrand Comparison and Quadrature Requirements

The approximation calculates the physical shape function derivatives using this expanded inverse. Since $\nabla \mathbf{N}$ is degree 1 and J^{-1}_{taylor} is degree 4, the approximated $\mathbf{B}$ matrix becomes degree 5:

$$\mathbf{B}_{\text{taylor}} = \nabla \mathbf{N} \cdot J^{-1}_{\text{taylor}} \quad (\deg = 1 + 4 = 5).$$

The final stiffness integrand is computed as:

$$\mathbf{K}_{\text{taylor}} = \mathbf{B}_{\text{taylor}}^{T} \mathbf{B}_{\text{taylor}} \det(J).$$

Multiplying two degree 5 polynomials results in degree 10, and multiplying by the linear Jacobian $\det(J)$ adds 1 more degree:

$$\mathbf{K}_{\text{taylor}} = \begin{bmatrix} 5368709120.0\xi^{11} - \dots & \dots & \dots \\ \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots \end{bmatrix}.$$

The resulting integrand has a maximum polynomial degree of $d_{\text{poly}} = 5 + 5 + 1 = 11$.

Method	Effective Degree	Theoretical GP
Ideal (Rational)	1	1
Taylor Expansion	11	6

Table 4.8: Gaussian point inflation due to Taylor Series approximation vs. Ideal Rational function.

Table 4.8 highlights the significant computational cost of the symbolic constraint. The Taylor expansion successfully converts a rational integral into a polynomial one, but it artificially inflates the effective degree from 1 to 11. Consequently, the solver calculates the Gaussian point requirement based on this inflated degree:

$$n_{gp,\text{approx}} = \left\lceil \frac{11+1}{2} \right\rceil = \lceil 6.0 \rceil \to 6.$$

While a rational function of degree 1 might theoretically be integrated with fewer points, the jump to 6 points is largely an artifact of the high-order polynomial approximation required to fit the symbolic engine.

Chapter 5

Summary and Outlook

5.1 Summary

This research presents the development of a high-performance, modular software framework implemented in the Rust programming language, designed to address critical interoperability and pre-processing challenges in computational mechanics. The primary objective was to bridge the gap between proprietary simulation formats and open-standard visualization tools while providing rigorous, mathematically derived mesh quality diagnostics and numerical integration optimization.

A defining characteristic of the developed framework is its strict modularity, explicitly designed to facilitate open-source contribution and extensibility. The architecture decouples the core logic into distinct, interchangeable components. The input pipeline operates as a dispatch system; while the current implementation supports COMSOL Multiphysics (`.mphtxt`) and Abaqus (`.inp`) formats, the structure allows developers to easily implement and plug in parsers for other formats, such as Ansys or Nastran, without modifying the core analysis engine. Furthermore, the framework permits the expansion of analytical capabilities, allowing users to implement alternative mesh quality metrics or extend the Gaussian quadrature optimization to support different element topologies or physical matrices. Although the current software utilizes a Gauss-Legendre error bound, the error calculation module is encapsulated, enabling future researchers to inject alternative error formulas or quadrature rules seamlessly.

In terms of data ingestion, the software successfully resolves the discrepancies between disparate mesh formats. It handles the implicit, sequential ordering of COMSOL files and the explicit, ID-mapped structure of Abaqus files. A critical contribution is the standardization of this data into the VTK Unstructured Grid (`.vtu`) format. This required the implementation of node reordering algorithms to map source-specific element connectivity to the canonical VTK standard. Additionally, the writer module implements coordinate and vector padding, ensuring that lower-dimensional entities, such as 1D edges and 2D surfaces, are correctly visualized in 3D space within ParaView.

The framework implements a rigorous algebraic quality assessment based on the factorization of the Jacobian matrix. By constructing the Jacobian as a symbolic monomial polynomial, the software evaluates the determinant at critical element locations to identify the "worst-case" distortion. The system isolates specific defects using Knupp's algebraic metrics, providing independent scalar fields for Shape, Skewness, Length Ratio, Orientation, and Volume. Unit tests confirmed that these metrics accurately identify isolated geometric defects and correctly flag invalid elements containing non-positive determinants, with all results injected directly into the VTK output for visual inspection.

The thesis also introduces a numerical optimization module that determines the optimal

number of Gaussian points required to satisfy a user-defined error tolerance. This feature relies on a specialized symbolic monomial arithmetic engine to construct exact polynomial representations of the Mass and Stiffness integrands. Results demonstrated that for standard geometries, the adaptive algorithm significantly reduces the integration order compared to theoretical upper bounds, often saving 2 to 4 integration points per dimension in 3D meshes. Moreover, the optimizer dynamically accounts for spatially varying material properties, increasing the integration order only when the polynomial degree of the material density or stiffness specifically demands it.

Finally, a critical analysis of the symbolic engine revealed a trade-off regarding non-linear element mappings. The error bound formulas strictly depend on the polynomial degree of the integrand. For elements with constant determinants, the symbolic integration is exact and highly efficient. However, for distorted elements, the Jacobian determinant involves square roots, or the inverse of the Jacobian involves rational functions. To maintain compatibility with the polynomial engine, the software employs Binomial and Taylor series expansions. The validation process revealed that these expansions artificially inflate the polynomial degree, for example increasing an effective degree of 1 to 11. This leads to misleading error estimates, causing the optimizer to prescribe a higher number of Gaussian points than theoretically necessary for distorted elements.

In conclusion, this work provides a robust, verifiable toolchain for mesh analysis and visualization. Its modular design ensures it can evolve to handle new formats and physics, while its rigorous mathematical approach provides transparency into the numerical quality of finite element meshes.

5.2 Future Developments

Based on the findings and limitations identified in this work, several avenues for future research and development are proposed. The primary area for improvement lies in the refinement of the mathematical approximation techniques used in the error estimation module. The current reliance on Taylor and Binomial series to handle non-square Jacobians and stiffness matrix inversions results in artificially inflated polynomial degrees. Future iterations should implement alternative approximation methods that will not cause degree inflation. Due to the modular design of the `GaussianQuadrature` struct, these mathematical kernels can be replaced or updated without necessitating a refactoring of the overarching architecture. Furthermore, the error estimation framework should be generalized to support element topologies beyond the current tensor-product families, and to accommodate alternative numerical integration schemes, such as Gauss-Lobatto or pixel-based integration, which may offer superior convergence rates for specific distorted geometries.

The analytical scope of the framework can be significantly broadened by extending the supported physical formulations. While the current implementation optimizes Mass and Stiffness matrices, the architecture is sufficiently generic to support the calculation of Damping matrices. Implementing the specific symbolic integrands for damping would allow for a complete dynamic analysis suite. Concurrently, the parsing module, which is currently limited to nodal field data, should be extended to support Element Value parsing. This would allow the software to ingest and visualize element-centric data, such as stress tensors, providing a more comprehensive view of simulation results.

Enhancements to the topological analysis capabilities constitute another critical direction for future work. Currently, the quality analysis is performed on the undeformed configuration. By leveraging the parsed displacement fields, the software could reconstruct the deformed geometry to perform quality assessment on the final configuration, allowing for

the identification of elements that degrade during large-deformation processes. Additionally, the current strategy of node omission for topology matching (e.g., converting 14-node pyramids to 13-node VTK standards) incurs data loss. A more robust approach would involve implementing shape function interpolation to mathematically map the original nodes to the target topology, ensuring that the visualized element retains the geometric fidelity of the source. The system is also primed for the inclusion of new algebraic quality metrics to diagnose specific phenomena such as planarity or orthogonality defects more precisely.

Finally, the software's modular architecture invites the expansion of its interoperability layer. The dispatch-based parsing engine can be readily extended to support additional proprietary formats, such as Ansys or Nastran mesh files, thereby increasing the tool's utility across different engineering workflows. Similarly, the writer module can be adapted to support alternative output standards beyond the current VTK XML format, facilitating integration with a wider ecosystem of post-processing and visualization software.

Appendix A

Source Code Repository

The source code developed for this thesis is available at the following GitHub repository: https://github.com/tiagomrns/meshrs/tree/main

Bibliography

[1] Amaral, I. B. do, Ferreira, R. C., and Goldman, A. *Rust vs. C for Python Libraries: Evaluating Rust-Compatible Bindings Toolchains*. 2025. arXiv: 2507.00264 [cs.PL]. URL: https://arxiv.org/abs/2507.00264.

[2] Atkinson, K. E. *An Introduction to Numerical Analysis*. Second. New York: John Wiley & Sons, 1989. ISBN: 0471500232. URL: http://www.worldcat.org/isbn/0471500232.

[3] Ayachit, U. *The ParaView Guide: A Parallel Visualization Application*. Kitware, Inc., 2015. ISBN: 978-1-930934-30-6.

[4] Barata, J. C. A. and Hussein, M. S. "The Moore–Penrose pseudoinverse: A tutorial review of the theory". In: *Brazilian Journal of Physics* 42.1 (2012), pp. 146–165.

[5] Bathe, K. J. *Finite element procedures*. 1996.

[6] Bucki, M., Lobos, C., Payan, Y., and Hitschfeld, N. "Jacobian-based repair method for finite element meshes after registration". In: *Engineering with Computers* 27.3 (2011), pp. 285–297.

[7] Chen, X., Liu, J., Pang, Y., Chen, J., Chi, L., and Gong, C. "Developing a new mesh quality evaluation method based on convolutional neural network". In: *Engineering Applications of Computational Fluid Mechanics* 14.1 (2020), pp. 391–400.

[8] COMSOL AB. *COMSOL Multiphysics Mesh Import and Export Guide*. Version 6.1. COMSOL AB. 2022.

[9] COMSOL AB. *COMSOL Multiphysics Reference Manual*. Version 6.1. COMSOL AB. 2022. Chap. 10.

[10] Cook, R. D. *Finite element modeling for stress analysis*. eng. New York: Wiley, 1995. ISBN: 0471107743.

[11] Cox, D. A., Little, J., and O'Shea, D. *Ideals, Varieties, and Algorithms*. Fourth. Undergraduate Texts in Mathematics. Cham: Springer, 2015. ISBN: 978-3-319-16720-6. DOI: 10.1007/978-3-319-16721-3.

[12] Dassault Systèmes. *Abaqus 2016 Analysis User's Guide*. Section: 28.1. Accessed: 2026-01-18. Dassault Systèmes Simulia Corp. 2016. URL: http://130.149.89.49:2080/v2016/books/usb/default.htm?startat=pt06ch28s01alm01.html.

[13] Dassault Systèmes Simulia Corp. *Getting Started with Abaqus: Interactive Edition (Version 6.6)*. Accessed: 2026-01-18. Washington University in St. Louis. URL: https://classes.engineering.wustl.edu/2009/spring/mase5513/abaqus/docs/v6.6/books/gss/default.htm.

[14] Davis, P. J. and Rabinowitz, P. *Methods of Numerical Integration*. New York: Academic Press, 1975. ISBN: 0-12-206350-3.

[15] Deligiannis, P., Lal, A., Mehrotra, N., Poddar, R., and Rastogi, A. "RustAssistant: Using LLMs to Fix Compilation Errors in Rust Code". In: *47th International Conference on Software Engineering (ICSE)*. Apr. 2025. URL: https://www.microsoft.com/en-us/research/publication/rustassistant-using-llms-to-fix-compilation-errors-in-rust-code/.

[16] Geuzaine, C. and Remacle, J.-F. "Gmsh: A 3-D Finite Element Mesh Generator with Built-in Pre- and Post-Processing Facilities". In: *International Journal for Numerical Methods in Engineering* 79 (Sept. 2009), pp. 1309–1331. DOI: 10.1002/nme.2579.

[17] Hiemstra, R. R., Calabrò, F., Schillinger, D., and Hughes, T. J. "Optimal and reduced quadrature rules for tensor product and hierarchically refined splines in isogeometric analysis". In: *Computer Methods in Applied Mechanics and Engineering* 316 (2017). Special Issue on Isogeometric Analysis: Progress and Challenges, pp. 966–1004. ISSN: 0045-7825. DOI: https://doi.org/10.1016/j.cma.2016.10.049. URL: https://www.sciencedirect.com/science/article/pii/S004578251631489X.

[18] Jung, M. and Langer, U. *Methode der finiten Elemente für Ingenieure: Eine Einführung in die numerischen Grundlagen und Computersimulation*. 2., überarbeitete und erweiterte Auflage. Wiesbaden, Germany: Springer Fachmedien Wiesbaden, 2013. ISBN: 978-3-658-01100-0.

[19] Kitware Inc. *IsoparametricCellsDemo*. Accessed: 2026-01-18. VTK Examples. URL: https://examples.vtk.org/site/Cxx/GeometricObjects/IsoparametricCellsDemo/.

[20] Kitware Inc. *LinearCellsDemo*. Accessed: 2026-01-18. VTK Examples. URL: https://examples.vtk.org/site/Cxx/GeometricObjects/LinearCellsDemo/.

[21] Kitware, Inc. *vtkBiQuadraticQuad.cxx*. https://github.com/Kitware/VTK/blob/master/Common/DataModel/vtkBiQuadraticQuad.cxx. VTK source code, accessed: 2025-09-01. 2025.

[22] Knupp, P. M. "Algebraic mesh quality metrics". In: *SIAM journal on scientific computing* 23.1 (2001), pp. 193–218.

[23] Knupp, P. M. "Algebraic mesh quality metrics for unstructured initial meshes". In: *Finite Elements in Analysis and Design* 39.3 (2003), pp. 217–241. ISSN: 0168-874X. DOI: https://doi.org/10.1016/S0168-874X(02)00070-7. URL: https://www.sciencedirect.com/science/article/pii/S0168874X02000707.

[24] Kwok, W. and Chen, Z. "A Simple and Effective Mesh Quality Metric for Hexahedral and Wedge Elements." In: *IMR*. 2000, pp. 325–333.

[25] Lang, S. *Complex Analysis*. Fourth. Vol. 103. Graduate Texts in Mathematics. New York: Springer-Verlag, 1999. DOI: 10.1007/978-1-4757-3083-8.

[26] Martins, E. M., Faé, L. G., Hoffmann, R. B., Bianchessi, L. S., and Griebler, D. *NPB-Rust: NAS Parallel Benchmarks in Rust*. 2025. arXiv: 2502.15536 [cs.DC]. URL: https://arxiv.org/abs/2502.15536.

[27] Miller, M. *Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape*. Presentation at BlueHat IL. Microsoft Security Response Center (MSRC), Feb. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/blob/master/presentations/2019_02_BlueHatIL/2019_01%20-%20BlueHatIL%20-%20Trends%2C%20challenge%2C%20and%20shifts%20in%20software%20vulnerability%20mitigation.pdf.

[28] Mohammadi, F. and Hosseini, M. “A new Legendre wavelet operational matrix of derivative and its applications in solving the singular ordinary differential equations”. In: *Journal of the Franklin Institute* 348.8 (2011), pp. 1787–1796. ISSN: 0016-0032. DOI: https://doi.org/10.1016/j.jfranklin.2011.04.017. URL: https://www.sciencedirect.com/science/article/pii/S0016003211001074.

[29] Nyblom, M. “On a generalization of the binomial theorem”. In: *The Fibonacci Quarterly* 37.1 (1999), pp. 3–13.

[30] Sande, V., Pena-Brage, F., and Constenla, I. *FEconv: Finite Element Converter*. Version 2.0. Computer software. 2020. URL: https://github.com/victorsndvg/feconv.

[31] Schlömer, N. *meshio: I/O for mesh files*. Version 5.3.5. Computer software. 2024. URL: https://github.com/nschloe/meshio.

[32] Shi, Z. and Chen, G. “Formal Verification of Executable Matrix Inversion via Adjoint Matrix and Gaussian Elimination”. In: *Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming*. 2024, pp. 1–13.

[33] Sorgente, T., Biasotti, S., Manzini, G., and Spagnuolo, M. “A survey of indicators for mesh quality assessment”. In: *Computer graphics forum*. Vol. 42. 2. Wiley Online Library. 2023, pp. 461–483.

[34] Sullivan, C. and Kaszynski, A. “PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK)”. In: *Journal of Open Source Software* 4.37 (2019), p. 1450.

[35] Vander Stoep, J. *Memory Safe Languages in Android 13*. Google Security Blog. Accessed: 2026-01-26. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[36] Zienkiewicz, O. *The Finite Element Method, ; Zienkiewicz, OC, Taylor, RL, Eds*. 2000.

Disclaimer

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Garching, January 26, 2026

(Signature)